University of Michigan Oct. 6 2021

AE567

Statistical Inference, Estimation, and Learning

Project 1:

Stochastic Modeling, Monte Carlo Methods, and Variance Reduction

---

Deniz Dost

UM ID: 82337583

1. Warm-up: Life without a CLT

For the given PDF for X, the CDF is calculated by simply integrating the given function as shown below:

$$\int_1^x \frac{a}{x^{a+1}}\,dx = \int_1^x a\,x^{-(a+1)}\,dx = a\left[\frac{1}{-a}\,x^{-a}\right]_1^\infty = -x^{-a}\Big|_1^x = -\,x^{-a} + 1 = 1 - x^{-a}$$

Then, the inverse CDF is computed as shown below, where u is sampled from a uniform distribution.

$$\begin{aligned} u &= 1 - x^{-a} \\ x^{-a} &= 1 - u \\ x &= (1-u)^{-\frac{1}{a}} \end{aligned}$$

This inverse CDF is used with 1000 uniformly distributed random variables for 100 times. Monte Carlo (MC) estimate for these evaluations are calculated by taking the running estimate. The resulting running estimates for 100 sequences are shown below in Figure 1.

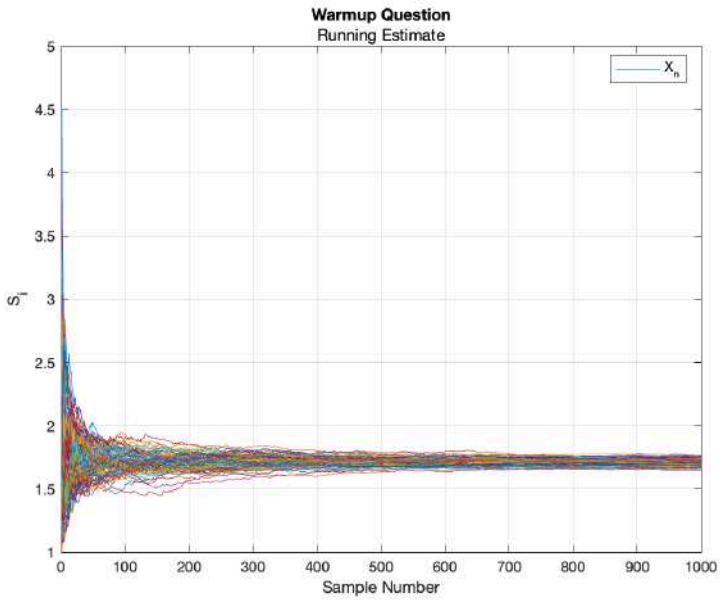

Figure 1

The variance of sequences taken over at each sample converges as shown below in Figure 2.

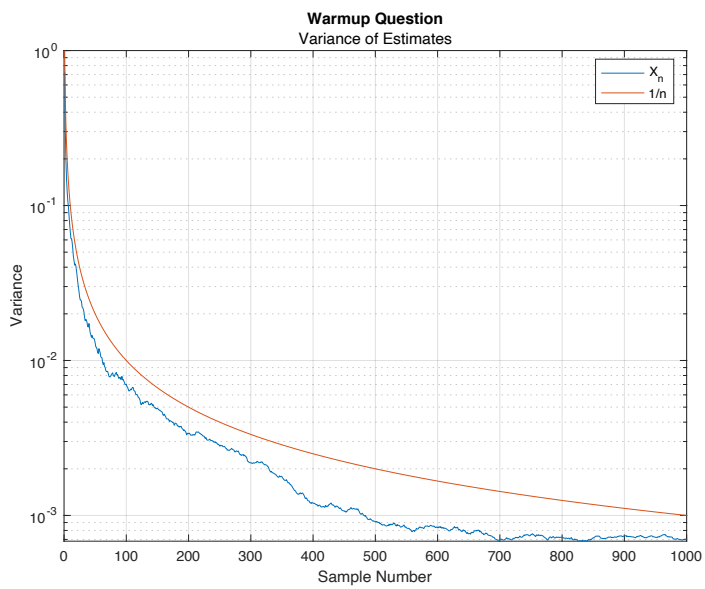

Figure 2

Basically, it follows the 1/n curve since the estimator has 1/n*variance. Also it is observed from the curve in the Figure 2, that the variance drops with increasing sample number. This can also be seen from the histogram of estimates in Figure 3.

The purple distribution which represents the estimates at 200 samples is much more wider than the red distribution which represents the estimates at 1000 samples. This is also intuitively expected.

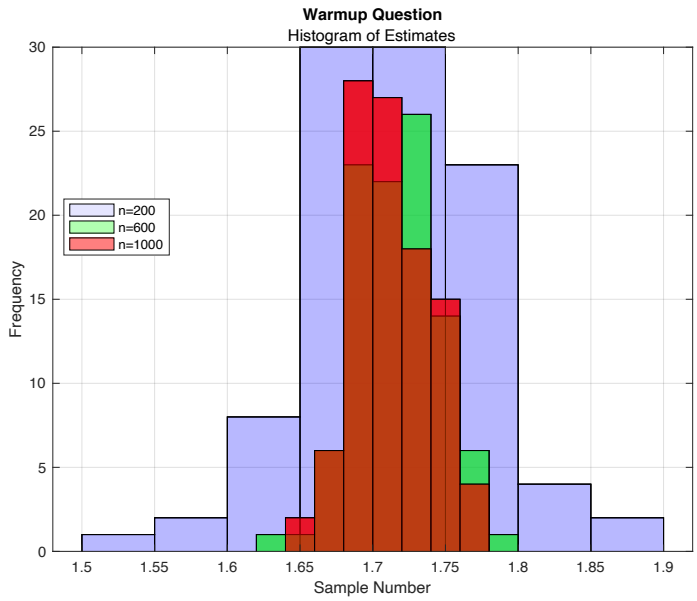

Figure 3

See Appendix 1 for the code for this problem.

2. Importance Sampling for Random Walks

2.1. 1-D Bernoulli Random Walk

Questions 2.1

(a)

In order to simulate the 1-D random walk, a set of steps (either 1 or -1) is created by sampling from a uniform distribution. To assign equal probability to positive and negative steps, a threshold value of 0.5 is used. Random variables above that value are assigned 1, and random variables below that value are assigned -1. Then, the cumulative sum of these steps are taken to find the path travelled. Figure 3 shows an example of a 1D Gaussian walk with 100 steps.

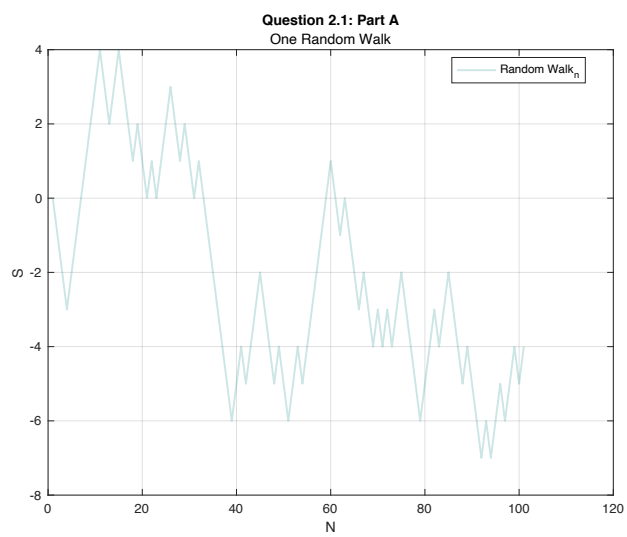

Figure 3

Then, 1000 1D walks are created and the paths are visualized as shown in Figure 4.

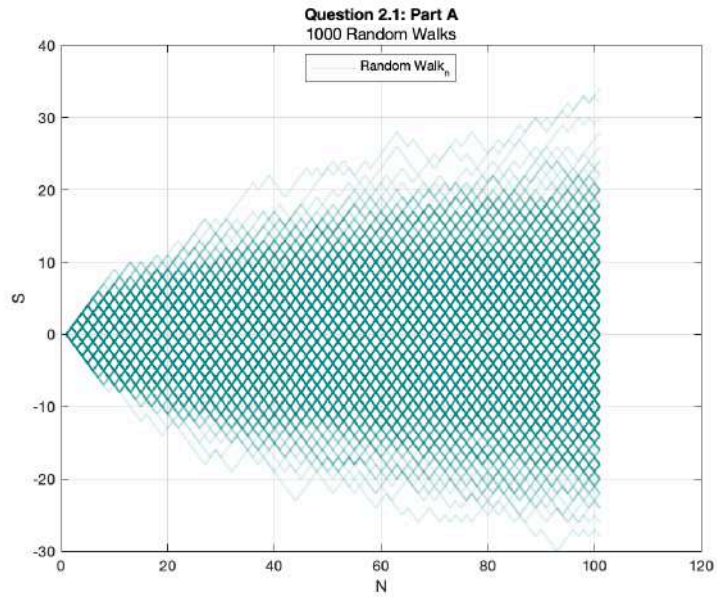

Figure 4

The distance arrived at the end of each path is the S value at N=100 and the distribution of these distances for 1000 steps can be seen in the histogram in Figure 5. This distribution is shaped like a symmetrical binomial distribution as expected with 0.5 p value since there are only two possible outcomes, 1 or -1.

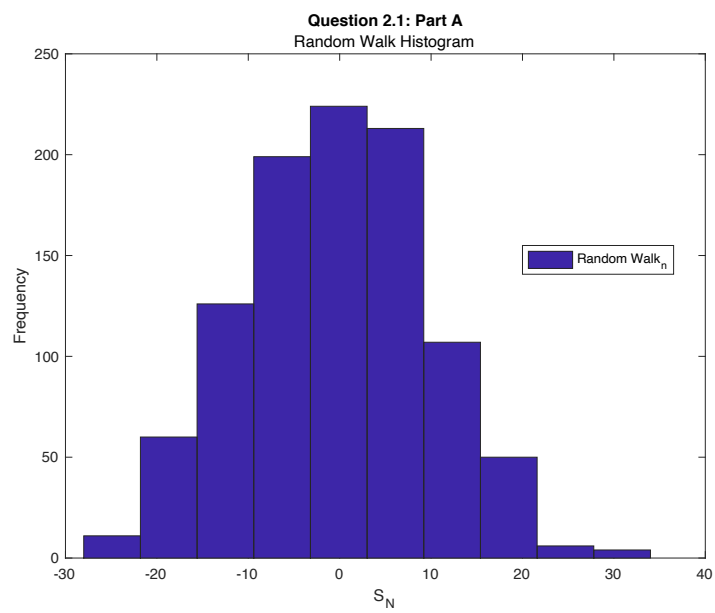

Figure 5

(b)

MC method is used for 100000 trials and the computed probability for P(S>10) is 0.1353. This can also be qualitatively observed from the histogram in Figure 6.

$$P(S > 10) = 0.1353$$

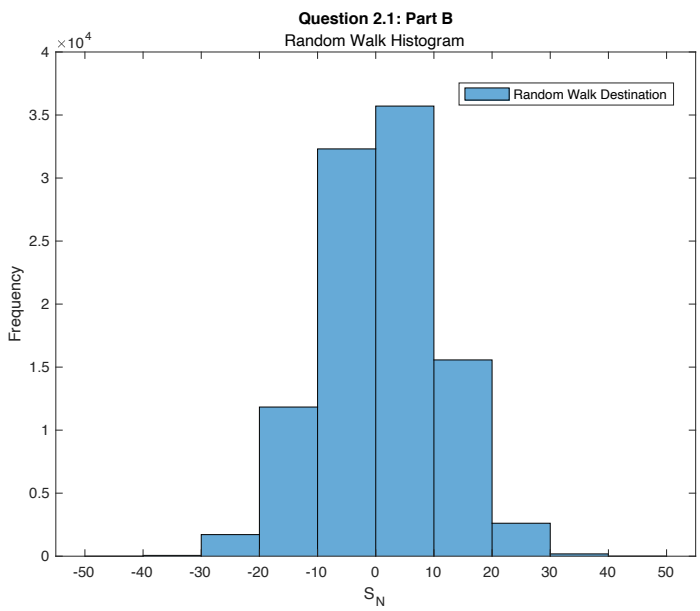

Figure 6

(c)

In order to find this probability, importance sampling is used. Proposed function for this exercise is sampling from the uniform distribution. However, this time the threshold value is selected to be 0.2 so that the walker would take 20% (−1) steps and 80% (+1) steps and the ratio of the original and proposed function (likelihood) is computed with relevant probabilities. The proposed function results in destination values distributed as shown in Figure 7. The proposal function clearly has its mean shifted more towards to the threshold since 55 is not reached frequently with the original distribution. Then, the evaluator function evaluates whether the arrived distance by the proposed distribution is

in the desired range. These are used to compute the importance sample MC estimate and resulted value is:

$$P(S > 55) = 7.9526e - 09$$

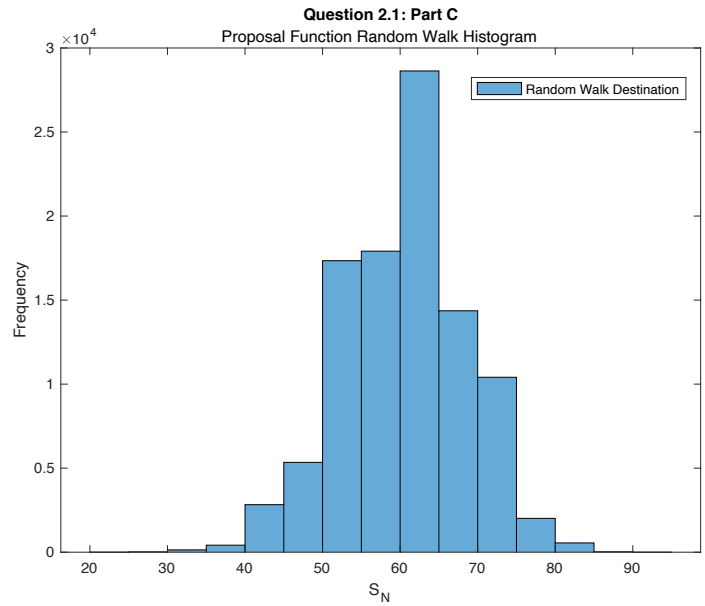

Figure 7

(d)

The analytical expressions are derived as follows:

$$Minimum\ number\ of\ steps\ required\ for\ S > 10 = 10 + \frac{100 - 10}{2} + 1 = 56$$

$$P(S > 10) = \frac{1}{2^{100}} \sum_{k=56}^{100} \binom{100}{k} = 0.1356$$

$$Minimum\ number\ of\ steps\ required\ for\ S > 55 = ceil(55 + \frac{100 - 55}{2}) = 78$$

$$P(S > 55) = \frac{1}{2^{100}} \sum_{k=78}^{100} \binom{100}{k} = 7.9527e - 09$$

These analytical values match with the MC values calculated in (b) and (c).

(e)

(i)

MC standard error can be calculated as

$$std_X[S_n[g]] = \frac{std_x[g]}{\sqrt{n}}$$

The corresponding values are 0.00108 for S>10, and 6.00417e-11 for S>55. Then, the 95% confidence interval is calculated where the upper and lower bounds are set in order to contain 95% of the data. In order to do that, z score which is defined as below is used.

$$H_n = \frac{S_n - \mu}{\sigma/\sqrt{n}}$$

$$P_X(-z \leqslant H_n \leqslant \ z) = P_X(-z \leqslant \frac{S_n - \mu}{\sigma/\sqrt{n}} \leqslant \ z) = 0.95$$

$$S_n = \mu z \pm ste_{MC}$$

where z=1.96 for 95%

$$(S > 10)_n^{upper\ bound} = 0.1332$$

$$(S > 10)_n^{lower\ bound} = 0.1375$$

$$(S > 55)_n^{upper\ bound} = 0.8070e - 08$$

$$(S > 55)_n^{lower\ bound} = 0.7835e - 08$$

(ii)

$$P(0.1332 \leqslant P(S > 10)_{analytical} \leqslant \ 0.1375) = 0.941$$

$$P(0.7835e - 08 \leqslant P(S > 55)_{analytical} \leqslant \ 0.807e - 08) = 0.954$$

So the interval limits hold.

(iii)

Taking the running mean of all steps each time step for 1000 trials results in a distribution as shown in Figure 8 for S>10 and in Figure 9 for S>55.

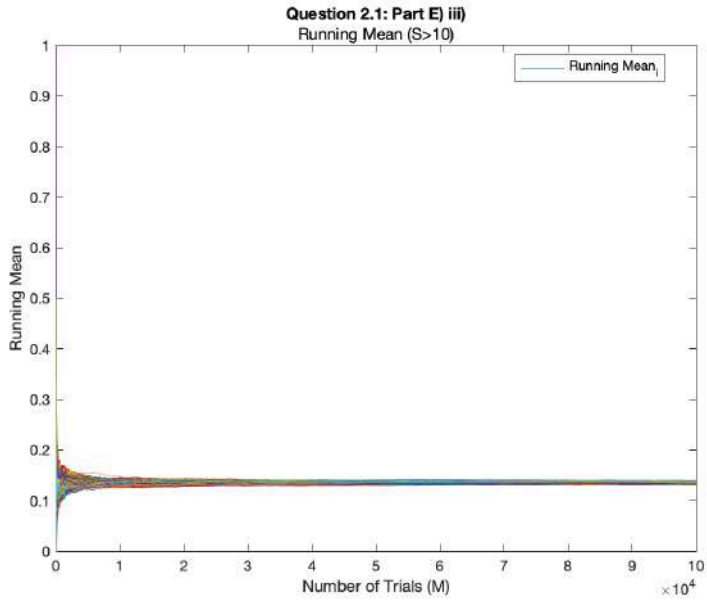

Figure 8

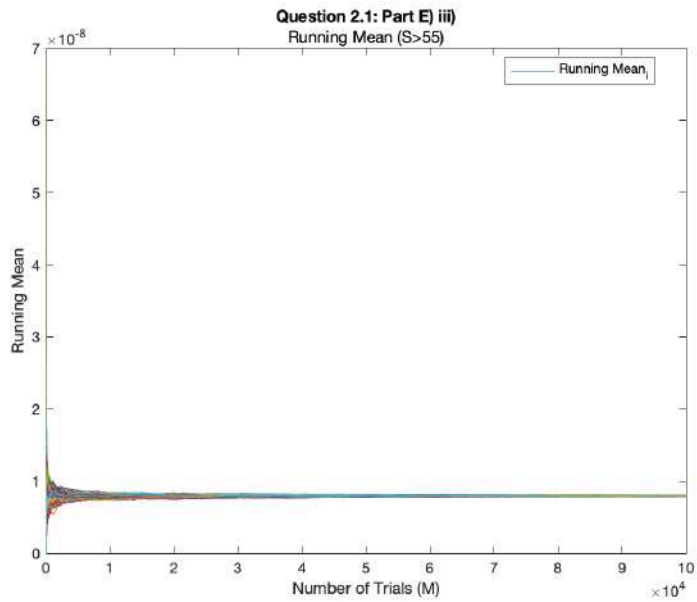

Figure 9

And the envelope created from the minimum and maximum values at each step creates an envelope as below, Figure 10 for S>10 and Figure 11 for S>55.

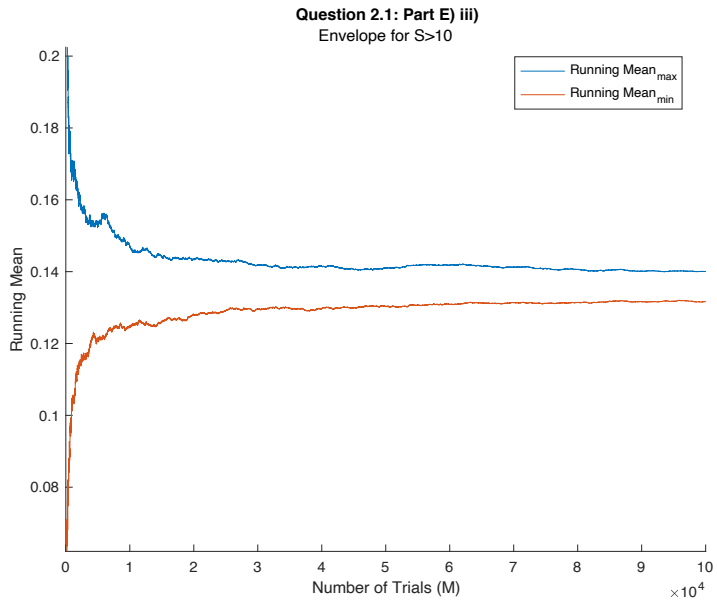

Figure 10

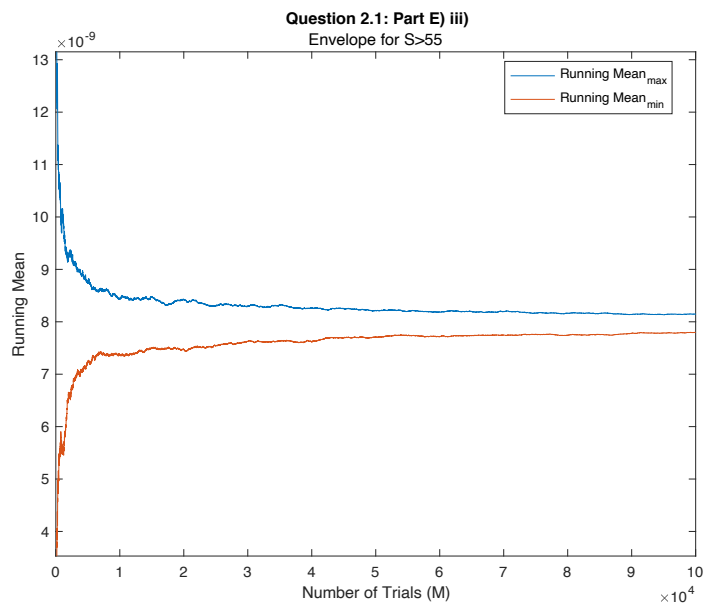

Figure 11

See Appendix 2 for the code for this problem.

2.2. 3-D Gaussian Random Walk

Questions 2.2

(a)

A 3-D Gaussian random walk is generated with 100 steps taken according to a standard normal distribution in all three axis. The histogram of the final distance at the end, step=100, is shown below in Figure 12.

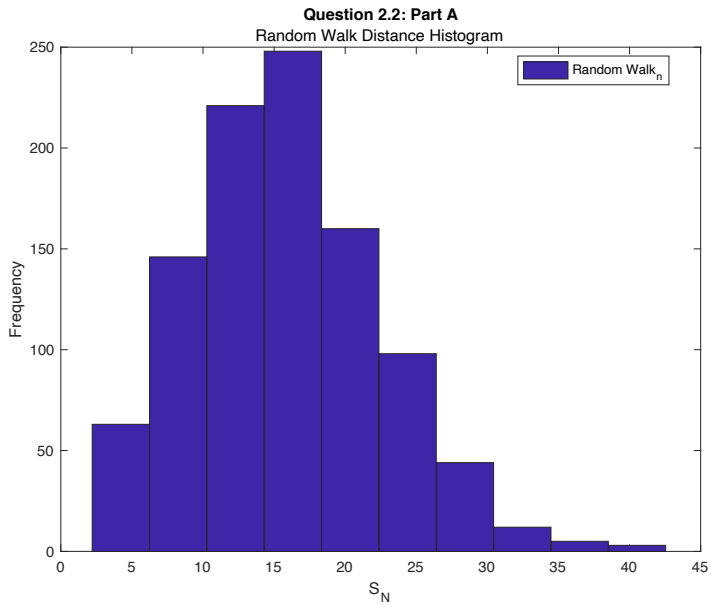

Figure 12

(b)

MC method is used for 100000 trials and the computed probability for P(S>10) is 0.1353. This can also be qualitatively observed from the histogram in Figure 13.

$$P(|S| > 10) = 0.79989$$

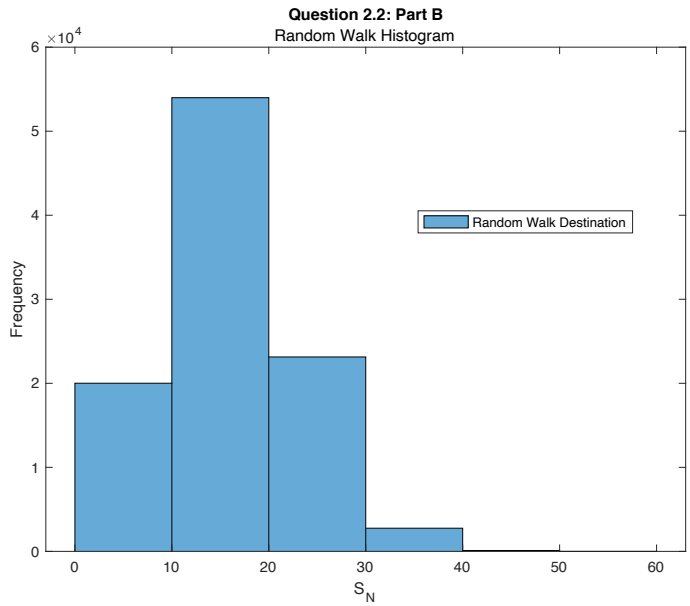

Figure 13

(c)

In order to find this probability, importance sampling is used. Proposed function for this exercise is sampling from the normal distribution. However, this time the mean is selected to be 0.25. The normal curves shifted to the right by this value, so that the walker would take less (−1) steps than (+1) steps and the ratio of the original and proposed function (likelihood) is computed with relevant probabilities. The proposed function results in destination values distributed as shown in Figure 14. The proposal function clearly has its mean shifted more towards to the threshold which is desired. Then, the evaluator function evaluates whether the the arrived distance by the proposed distribution is in the desired range. These are used to compute the importance sample MC estimate and resulted value is:

$$P(|S > 55|) = 2.4148e - 07$$

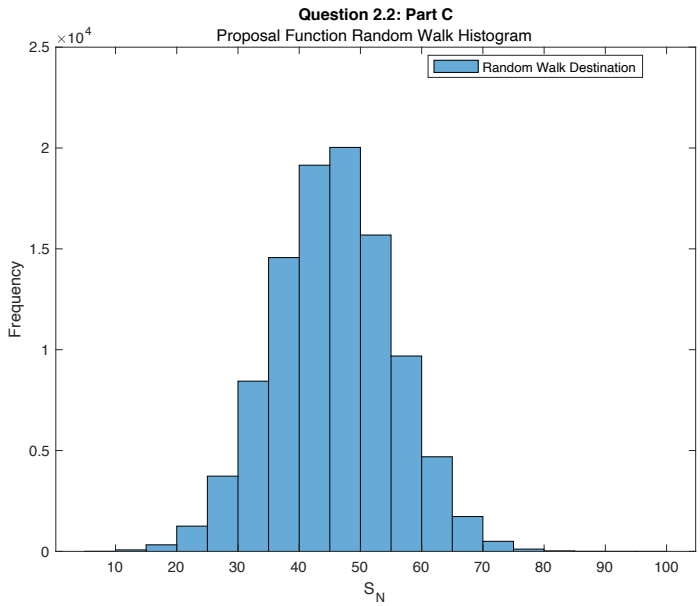

Figure 14

The analytical expression for $P(|S| > 55)$ can be derived as follows:

$$P(|S| > threshold) = P(\sqrt{S_x^2 + S_y^2 + S_x^2} > threshold)$$

$$P(|S|^2 > threshold^2) = P((S_x^2 + S_y^2 + S_x^2) > threshold^2)$$

$$S_X, S_y, S_z \sim N(0,N) \ \ , where \ N = 3$$

$$\sum_{i=1}^{N=3} X_i^2 \ \ , where \ X_i \sim N(0,\sigma^2) \ \ , \sigma^2 = N$$

$$then \ , \ Y \sim \Gamma(\frac{N}{2}, 2\sigma^2)$$

$$P(|S| > threshold) = P(\Gamma(\frac{3}{2}, 2N > threshold^2)) = 1 - P(\Gamma(\frac{3}{2}, 2N \leqslant threshold^2)) \ , \ where \ N = 100$$

$$P(|S| > 10) = 0.8012$$

$$P(|S| > 55) = 1.2226e - 06$$

If the 3-D case is considered, where there a 8 octants in a sphere where the walker can arrive at, the analytical probability can be multiplied by 8. Since the

proposed distribution is shifted to positive walks. This may not be the true approach for the analytic solution. The obtained probability is now 9.7812e-06. These probability values are consistent with the MC outputs.

(d)

As shown above in the 1-D walk case, MC standard error is calculated. The corresponding values are 0.0013 for |S|>10, and 3.0482e-08 for |S|>55. The confidence intervals are also calculated as below:

$$(|S| > 10)_n^{upper\ bound} = 0.8009$$
$$(|S| > 10)_n^{lower\ bound} = 0.7989$$

$$(|S| > 55)_n^{upper\ bound} = 0.241478e - 06$$
$$(|S| > 55)_n^{lower\ bound} = 0.241477e - 06$$

See Appendix 3 for the code for this problem.

3. Multilevel Monte Carlo and Control Variates for Stochastic ODEs

Question 1

The Wiener Process is defined in the problem statement and since $W(t)$ is defined to have Gaussian increments $\Delta W_n = W(t_{n+1}) - W(t_n)$ is also a Gaussian distribution.

$$W(t + \Delta t) = W(t) + \sqrt{\Delta t}\xi \text{ , } where\ \xi \sim N(0,1)$$
$$W(t + \Delta t) \sim N(0,\Delta t)$$

3.1.

In order to create a geometric brownian motion, first a set of 100 Wiener Processes are created sampling from a normal distribution according to the following formula.

$$W^n(t) = \frac{1}{\sqrt{n}} \sum_{1 \leqslant k \leqslant [nt]} X_k \text{ , } where\ n = \frac{t_n}{\Delta t}\ and\ t_n = 1$$

This process is the input for brownian motion's diffusion part. For a time step value of $\Delta t = 0.05$ and $0 \leqslant t \leqslant 1$, and the drift and diffusion parameters as 0.05 and 0.2 respectively, a Wiener distribution is created. Figure 15 shows this process.

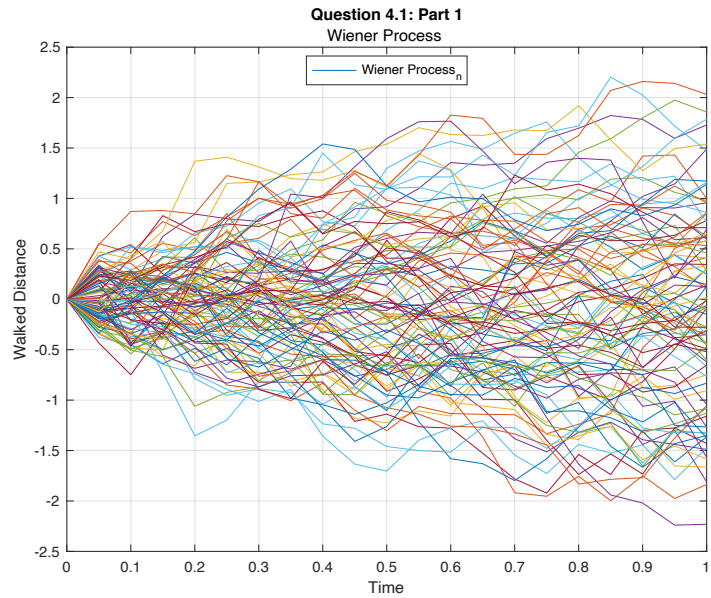

Figure 15

Then, Euler-Maruyama scheme is used to create the geometric Brownian motion as shown in Figure 16.

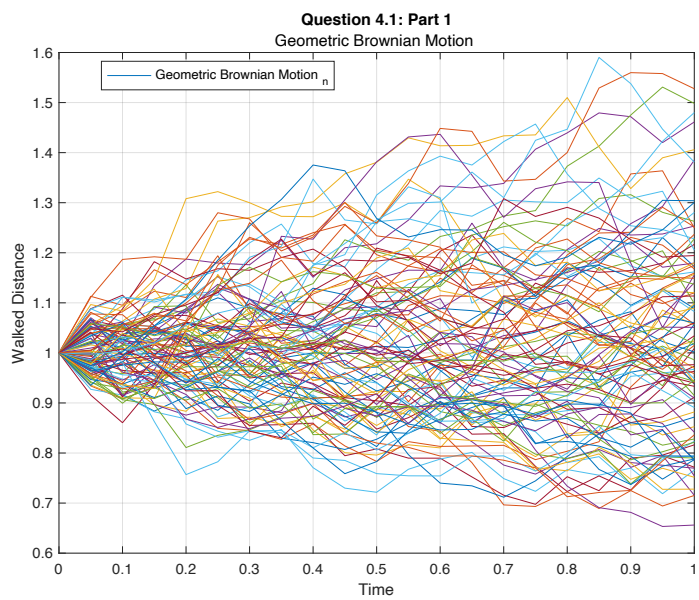

Figure 16

MC method created for this problem resulted in a variance of 0.0385 and an expectation of 1.0326 for Y(1).

The analytical solution is also obtained as follows:

$$E[Y(1)] = E[Y_0 e^{\mu - \frac{\sigma^2}{2}t + \sigma W_t}] \ \ , where \ t = 1$$

$$E[Y(1)] = Y_0 e^{\mu - \frac{\sigma^2}{2}} + E[e^{\sigma W_1}] \ \ , where \ Y_0 = 1$$

From $E[e^x] = e^{\mu_x + \frac{\sigma_x^2}{2}}$,

$$\Delta W \sim N(0, \Delta t)$$

$$W_1 \sim N(0, N\Delta t)$$

$$\sigma^2 W_1 \sim N(0, \sigma N \Delta t)$$

$$E[Y(1)] = e^{\mu - \frac{\sigma^2}{2}} e^{\frac{\sigma^2 N \Delta t}{2}} \ \ , where \ N = \frac{1}{\Delta t}$$

$$E[Y(1)] = e^{\mu - \frac{\sigma^2}{2} + \frac{\sigma^2}{2}} = 1.0513$$

Also,

$$Since \ \ Var(Y) = E[Y^2] - E[Y]^2$$

$$Var(Y(1)) = e^{2\mu - \sigma^2 + 2\sigma^2 N \Delta t} - e^{2\mu - \sigma^2 + \sigma^2 N \Delta t} = 0.0451$$

So the analytical solutions are consistent with the MC outputs.

3.2.

In order to examine Brownian motion with different timescales, Δt and $4\Delta t$, first a function for Wiener Process is defined. Since these fine and coarser motions will have the same statistics, fine scale motion is used to create coarse scale motion. This function takes the Wiener process values form the finer set and creates a ΔW from designated intervals. Therefore, a coarser Wiener process is created as shown below in Figure 17.

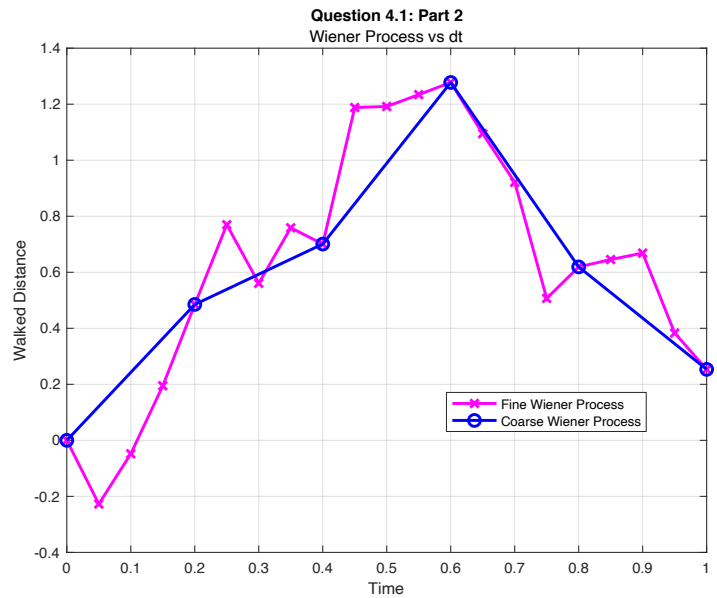

Figure 17

Using these coarser Wiener process values in the Euler-Maruyama scheme, a coarser Brownian walk is created as shown below in Figure 18.

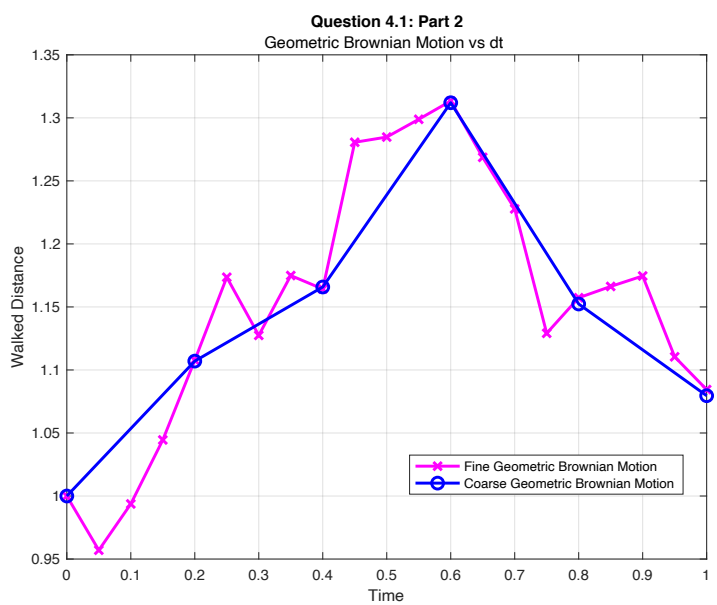

Figure 18

Here, the coarser motion and the finer motion intersect at the multipliers of 4 of Δt but the finer motion follows a smoother path since it takes outputs more samples than the coarser one as expected.

3.3.

For MLMC, the general scheme looks like:

$$E[Y_f] = E[Y_c] + E[Y_f - Y_c]$$

And for the sake of this question where there are 4 levels, the MLMC algorithm is created in accordance with the following expression where Y_4 is the high fidelity model with $\Delta t = 4^{-5}$ and Y_1 the lowest fidelity model with $\Delta t = 4^{-1}$. These levels share the underlying realizations of wiener realization at their respective time steps. This is acquired with using the fine to coarse transformation function as discussed.

$$E[Y_4(1)] = E[Y_1(1)] + \sum_{i=2}^{4} E[Y_i(1) - Y_{i-1}(1)]$$

This results in in the bar graphs shown in the figures below for variance and estimates. It is clear and intuitive that both decrease as the level goes from coarser to finer. Level 1 corresponds to the coarsest level and the fidelity increases as the level number increases. This is expected since a coarser estimator will only take a few samples whereas a finer one takes a lot more samples which reduces the variance as shown in Figure 19 and Figure 20.

The variance value decreasing with increased fidelity can also be derived from the formula

$$Var(Y_l - Y_{l-1}) = Var(Y_l) + Var(Y_{l-1}) - 2Cov(Y_l - Y_{l-1})$$

As the levels get finer, the Y values will be more closer to each other which result in a bigger covariance. So the overall variance of a finer level will be reduced.

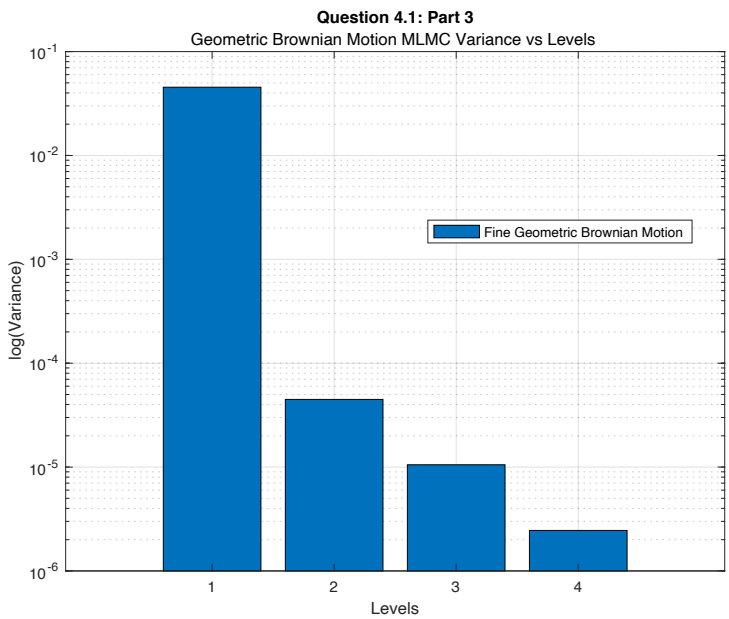

Figure 19

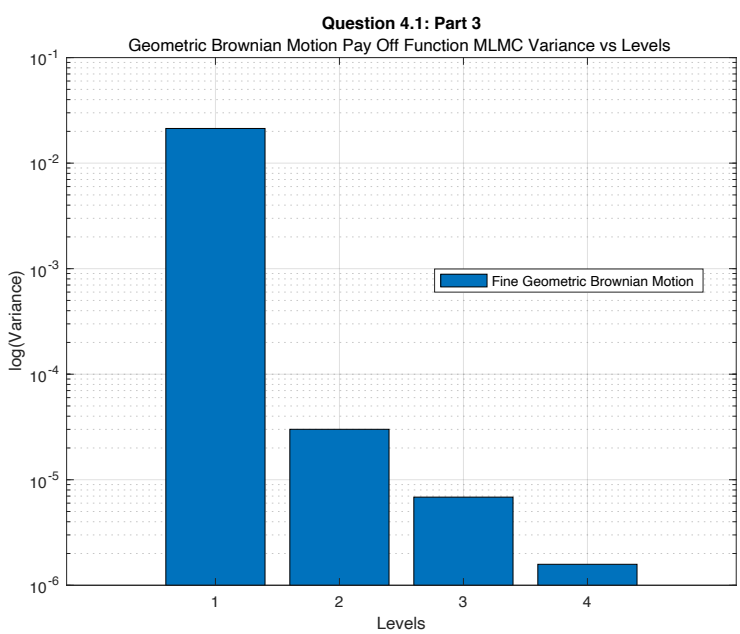

Figure 20

The code ran for MLMC is now modified for the pay-off cost function given. The expectation of this estimator can be seen in the figures below, Figure 21 and Figure 22. The trend from the E[Y(1)] is observed in this case as well, where the expectation increases towards the coarser levels. Intuitively, as the levels become finer, it is expected to converge to the true value better.

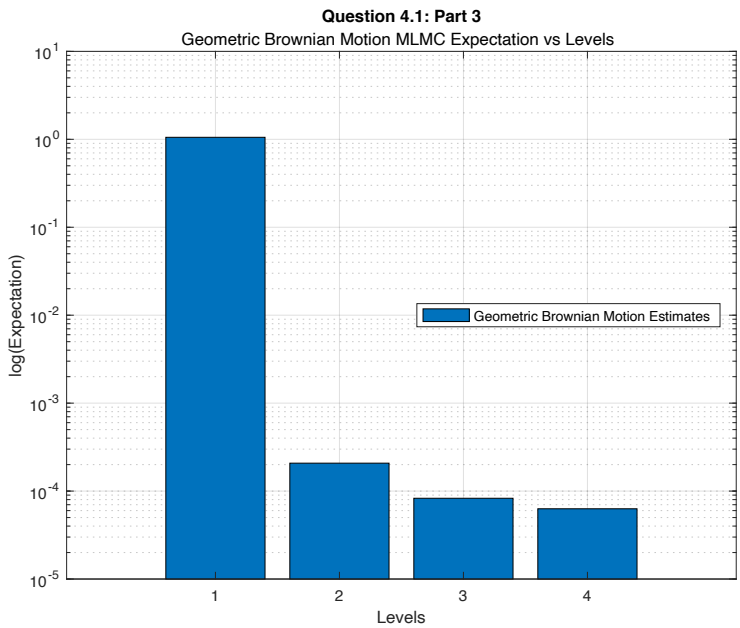

Figure 21

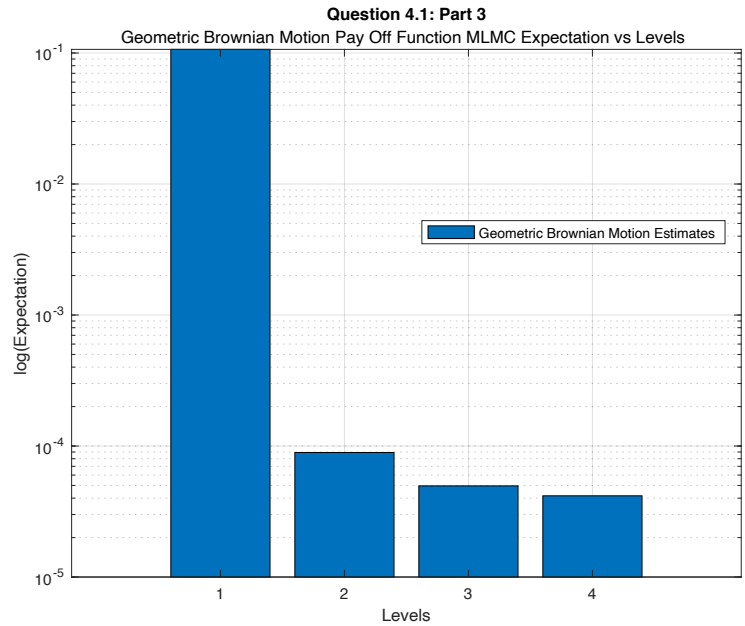

Figure 22

3.4.

In order to compute the theoretical cost of the MLMC estimator, run times obtained from Matlab tic and toc measurements are used. The runtimes for each time is as follows:

$$t_{P_1(Y)} = 0.029989 \ seconds$$

$$t_{P_2(Y)-P_1(Y)} = 0.032751 \ seconds$$

$$t_{P_3(Y)-P_2(Y)} = 0.041187 \ seconds$$

$$t_{P_4(Y)-P_3(Y)} = 0.054871 \ seconds$$

$$\lambda = \epsilon^{-2} \sum_{l=0}^{L} \sqrt{V_l \tilde{C}_l}$$

Where variances V_l and costs $\tilde{C}_l$ are calculated above.

$$N_l = \lambda\sqrt{V_l\tilde{C}_l}$$

$$N_{P_1(Y)} = 0.0027e - 12$$

$$N_{P_2(Y)-P_1(Y)} = 0.0793e - 12$$

$$N_{P_3(Y)-P_2(Y)} = 0.1823e - 12$$

$$N_{P_4(Y)-P_3(Y)} = 0.4014e - 12$$

This is also intuitive since the coarsest level would require the least, and the finest level would require the most allocation.

3.5.

In order to find the required sample size, the sample size where the MLMC and MC both have the same statistics can be found and taken as the result.

See Appendix 4 for the code for this problem.

4. APPENDIX

4.1.Appendix 1

```
% Warmup

clear all
close all
clc

seed = 1333; % rng seed for reproducibility


sumType = 0;
alpha_p = 3/2;
sampleNum = 1000;
n = 100;

MCest = zeros(1,n);

sumType = 1;
[estimate, samples, evaluations] = monteCarlow(n,sampleNum,seed,alpha_p,sumType);

plot(estimate')
title('Warmup Question', 'Running Estimate')
xlabel('Sample Number')
ylabel('S_i')
legend('X_n','Location','best')
grid on

j=1;
for i=1:sampleNum

varn(j)=var(estimate(:,i));
 j=j+1;
end

semilogy(varn)

semilogy(varn)
hold on
semilogy(1./(1:sampleNum))
title('Warmup Question', 'Variance of Estimates')
xlabel('Sample Number')
ylabel('Variance')
legend('X_n','1/n','Location','best')
grid on

figure(4)
i=200:400:sampleNum;
h(1)=histogram(estimate(:,i(1)),'FaceColor','b','FaceAlpha',0.1)
hold on
h(2)=histogram(estimate(:,i(2)),'FaceColor','g','FaceAlpha',0.3)
h(3)=histogram(estimate(:,i(3)),'FaceColor','r','FaceAlpha',0.5)
title('Warmup Question', 'Histogram of Estimates')
```

```
xlabel('Sample Number')
ylabel('Frequency')
legend('n=200','n=600','n=1000','Location','best')
grid on

function [estimate, samples, evaluations] =
monteCarlow(runNumber,numSteps,seed,alpha_p,sumType)
s = rng;
s.Seed = seed;
rng(s.Seed);

u = rand(runNumber,numSteps);
u (u>0.9) = 0.1;
x = (1-u).^(-1/alpha_p);
samples = x;
evaluations = x;

if sumType == 0
    estimate = sum(evaluations') / (numSteps); % indicator func sum columnwise
summation
else
    indMatrix = ones(runNumber,1)*(1:numSteps);
    cumsumMatrix = cumsum(evaluations');
    estimate = cumsumMatrix' ./ (1:numSteps); % columnwise summation
end
end
```

4.2. Appendix 2

```
% 567 RW
clc
clear all
close all

seed = 1333; % rng seed for reproducibility

%% PART A

numSteps = 100;
runNumber = 1000;
pThreshold = 0.5;
steps = generateRandomWalkSteps(runNumber,numSteps,pThreshold,seed);
randomWalk = generateRandomWalk(steps);

figure(1)
plot(randomWalk','LineWidth',1.5,'Color',[0, 0.5, 0.5, 0.10])
title('Question 2.1: Part A','1000 Random Walks')
xlabel('N')
ylabel('S')
legend('Random Walk_n','Location','best')
grid on

figure(2)
hist(randomWalk(:,end))
title('Question 2.1: Part A','Random Walk Histogram')
xlabel('S_N')
```

```
ylabel('Frequency')
legend('Random Walk_n','Location','best')

%% PART B

distLimit = 10;
totDist = [];
runNumber = 10^5;


sumType = 0; % 0: sum, 1: cumsum
[estimate, samples,
evaluations]=monteCarlo2(runNumber,numSteps,seed,pThreshold,distLimit,sumType);

PS10 = estimate;
EvalS10 = evaluations;

%% PART C
sumType = 0; % 0: sum, 1: cumsum
pThresholdc = 0.2;
distLimitc = 55;
[estimate, samples, evaluations] =
ISmonteCarlo2(runNumber,numSteps,seed,pThreshold,pThresholdc,distLimit,distLimitc,sumT
ype);

PS55 = estimate;
EvalS55 = evaluations;

%% PART D

minSteps10 = ceil(distLimit + ((100-distLimit)/2))+1;
syms k x
AnalyticalP10 =
double((1/2^numSteps)*symsum(nchoosek(numSteps,k),minSteps10,numSteps));
% y = 1-binocdf(minSteps10,100,0.5)

minSteps55 = ceil(distLimitc + ((100-distLimitc)/2));
AnalyticalP55 =
double((1/2^numSteps)*symsum(nchoosek(numSteps,k),k,minSteps55,numSteps));

%% PART E

% Part i)

stdError10 = std(EvalS10)/sqrt(runNumber);
stdError55 = std(EvalS55)/sqrt(runNumber);

z = 1.96;
% prob 0.95 ==> z score 1.96

meanEvalS10 = mean(EvalS10);
varEvalS10 = var(EvalS10);
intervalS10_ = [meanEvalS10-z*stdError10 meanEvalS10+z*stdError10];


meanEvalS55 = mean(EvalS55);
varEvalS55 = var(EvalS55);
intervalS55_ = [meanEvalS55-z*stdError55 meanEvalS55+z*stdError55];
```

```
% Part ii)

seed = 1333;
duplicateNum = 1000;

PS10_1000 = zeros(1,duplicateNum);
Eval10_1000 = zeros(runNumber,duplicateNum);
PS55_1000 = zeros(1,duplicateNum);
intervalS10 = zeros(duplicateNum,2);
evalEval = zeros(runNumber,1);


for i=1:duplicateNum
seed = seed+i;

[estimate, samples,
evaluations]=monteCarlo2(runNumber,numSteps,seed,pThreshold,distLimit,sumType);
PS10i = estimate;
EvalS10i = evaluations;
stdError10i = std(EvalS10i)/sqrt(runNumber);
Eval10_1000(:,i)  = EvalS10i;
intervalS10i = [PS10i-z*stdError10i PS10i+z*stdError10i];
intervalS10(i,:) = intervalS10i;


[estimate, samples, evaluations] =
ISmonteCarlo2(runNumber,numSteps,seed,pThreshold,pThresholdc,distLimit,distLimitc,sumT
ype);
PS55i = estimate;
EvalS55i = evaluations;
stdError55i = std(EvalS55i)/sqrt(runNumber);
Eval55_1000(:,i)  = EvalS55i;
intervalS55i = [PS55i-z*stdError55i PS55i+z*stdError55i];
intervalS55(i,:) = intervalS55i;

end

confP10 = sum(intervalS10(:,1) <= AnalyticalP10 & AnalyticalP10 <= intervalS10(:,2))/
duplicateNum;
confP55 = sum(intervalS55(:,1) <= AnalyticalP55 & AnalyticalP55 <= intervalS55(:,2))/
duplicateNum;
save('q21e','confP10','confP55','intervalS10','intervalS55','Eval55_1000','Eval10_1000
')

% Part iii)


runningMean10Eval = zeros(duplicateNum,runNumber);
runningMean55Eval = zeros(duplicateNum,runNumber);

cumsumEval = cumsum(Eval10_1000);
indMatrix = ones(duplicateNum,1)*(1:runNumber);
runningMean10Eval=cumsumEval'./indMatrix;

cumsumEval = cumsum(Eval55_1000);
runningMean55Eval=cumsumEval'./indMatrix;
```

```
runningMean10EvalMax = max(runningMean10Eval);
runningMean10EvalMin = min(runningMean10Eval);
runningMean55EvalMax = max(runningMean55Eval);
runningMean55EvalMin = min(runningMean55Eval);

figure(3)
plot(1:runNumber,runningMean10EvalMax
hold on
plot(1:runNumber,runningMean10EvalMin)
title('Question 2.1: Part E) iii)','Envelope for S>10')
xlabel('Number of Trials (M)')
ylabel('Running Mean')
legend('Running Mean_{max}','Running Mean_{min}','Location','best')


figure(4)
plot(1:runNumber,runningMean55EvalMax)
hold on
plot(1:runNumber,runningMean55EvalMin)
title('Question 2.1: Part E) iii)','Envelope for S>55')
xlabel('Number of Trials (M)')
ylabel('Running Mean')
legend('Running Mean_{max}','Running Mean_{min}','Location','best')
%
save('q21e','confP10','confP55','intervalS10','intervalS55','Eval55_1000','Eval10_1000
','runningMean10Eval','runningMean55Eval')

% generate a set of steps for a random walk

function X = generateRandomWalkSteps(runNumber,numSteps,threshold,seed)
% rng seed fix
s = rng;
s.Seed = seed;
rng(s.Seed)
X = rand(runNumber,numSteps); % sample from a unifrom
X (X < threshold) = -1;
X (X > threshold) =  1;
end

% generate a rv that is shrunk in space and sped up in time from a sequence of steps

function X = generateRandomWalk(steps)
%     steps=steps';
    stepSize = size(steps);
    X = zeros(1,stepSize(1));
    X = [X; cumsum(steps')];
    X = X';
end
function [estimate, samples, evaluations] =
monteCarlo2(runNumber,numSteps,seed,pThreshold,distLimit,sumType)

steps = generateRandomWalkSteps(runNumber,numSteps,pThreshold,seed);
randomWalk=generateRandomWalk(steps);
samples = randomWalk;
totDist = randomWalk(:,end);
evaluations =evaluatorFunc(totDist,distLimit);
if sumType == 0
    estimate = sum(evaluations) / (runNumber); % indicator func sum columnwise
summation
else
```

```
    estimate = cumsum(evaluations) / (1:runNumber); % columnwise summation
end
end
function [estimate, samples, evaluations] =
ISmonteCarlo2(runNumber,numSteps,seed,pThresholdf,pThresholdq,distLimitf,distLimitq,su
mType)


fsteps = generateRandomWalkSteps(runNumber,numSteps,pThresholdf,seed);
frandomWalk=generateRandomWalk(fsteps);
qsteps = generateRandomWalkSteps(runNumber,numSteps,pThresholdq,seed);
qrandomWalk=generateRandomWalk(qsteps);
q_x = prob_x(qsteps,pThresholdq);
f_x = prob_x(qsteps,pThresholdf);

g_x =evaluatorFunc(qrandomWalk(:,end),distLimitq);

likelihoodfq   = f_x./q_x;
plikelihoodfq  = prod(likelihoodfq');
evaluations    = g_x.*plikelihoodfq';

samples = qrandomWalk;
if sumType == 0
    estimate = sum(evaluations) / (runNumber); % indicator func sum columnwise
summation
else
    estimate = cumsum(evaluations) / (1:runNumber+1); % columnwise summation
end
end

function evaluatorFuncOut = evaluatorFunc(totDist, threshold)
g_xTotDist = totDist;
g_xTotDist (totDist <= threshold) = 0;
g_xTotDist (totDist > threshold) = 1;
probDistLimit = sum(g_xTotDist)/length(g_xTotDist);
evaluatorFuncOut = g_xTotDist;
end

function probWalk = prob_x(inputArray,pValue)
inputArray (inputArray ==  -1) = pValue ;
inputArray (inputArray ==  1) = 1-pValue ;
probWalk = (inputArray);
end
```

4.3. Appendix 3

```
% 567 3D RW
clc
clear all
close all

seedX = 1333; % rng seed for reproducibility
seedY = 4806; % rng seed for reproducibility
seedZ = 1881; % rng seed for reproducibility
seed  = [seedX seedY seedZ];

%% PART A

numSteps  = 100;
```

```
runNumber = 1000;
stepsX = generateRandomWalkStepsNormal(runNumber,numSteps,seed(1));
stepsY = generateRandomWalkStepsNormal(runNumber,numSteps,seed(2));
stepsZ = generateRandomWalkStepsNormal(runNumber,numSteps,seed(3));
randomWalk = cat(3,stepsX,stepsY,stepsZ);

randomWalkT = cat(3,(randomWalk(:,:,1))',(randomWalk(:,:,2))',(randomWalk(:,:,3))');
posEnd = sum(randomWalkT);

posEnd = squeeze(posEnd);
distEnd = vecnorm(posEnd'); % If A is a matrix, then vecnorm returns the norm of each column.

figure(2)
hist(distEnd)
title('Question 2.2: Part A','Random Walk Distance Histogram')
xlabel('S_N')
ylabel('Frequency')
legend('Random Walk_n','Location','best')

%% PART B
distLimit = 10;
runNumber = 100000;
sumType = 0;

[estimate, samples, evaluations] = monteCarloNormal(runNumber,numSteps,seed,distLimit,sumType);
EvalS10=evaluations;
estimate

%% PART C
% IS Sampling Rules
distLimitc = 55;
runNumber = 100000;
sumType = 0;

mu1 = 0;
sigma1 = 1;

mu2 = 0.25;
sigma2 = 1;

[estimate, samples, evaluations] =
ISmonteCarloNormal(runNumber,numSteps,seed,mu1,sigma1,mu2,sigma2,distLimitc,sumType);
EvalS55 = evaluations;
estimate

%% PART D

stdError10 = std(EvalS10)/sqrt(runNumber);

stdError55 = std(EvalS55)/sqrt(runNumber);

z = 1.96;
% prob 0.95 ==> z score 1.96

meanEvalS10 = mean(EvalS10);
varEvalS10 = var(EvalS10);
intervalS10 = [meanEvalS10-z*varEvalS10/(sqrt(runNumber)) meanEvalS10+z*varEvalS10/(sqrt(runNumber))];


meanEvalS55 = mean(EvalS55);
varEvalS55 = var(EvalS55);
intervalS55 = [meanEvalS55-z*varEvalS55/(sqrt(runNumber)) meanEvalS55+z*varEvalS55/(sqrt(runNumber))];
```

```
%% extra
AnalyticalP10 = 1 - gamcdf(distLimit,3/2,200)
AnalyticalP55 = 1 - gamcdf(distLimitc,3/2,200)

% generate a set of steps for a random walk

function X = generateRandomWalkStepsNormal(runNumber,numSteps,seed)
% rng seed fix
s = rng;
s.Seed = seed;
rng(s.Seed)

X = randn(runNumber,numSteps); % sample from a unifrom

end

function [estimate, samples, evaluations] = monteCarloNormal(runNumber,numSteps,seed,distLimit,sumType)

stepsX = generateRandomWalkStepsNormal(runNumber,numSteps,seed(1));
stepsY = generateRandomWalkStepsNormal(runNumber,numSteps,seed(2));
stepsZ = generateRandomWalkStepsNormal(runNumber,numSteps,seed(3));
randomWalk = cat(3,stepsX,stepsY,stepsZ);
samples = randomWalk;

randomWalkT = cat(3,(randomWalk(:,:,1))',(randomWalk(:,:,2))',(randomWalk(:,:,3))');
posEnd = sum(randomWalkT);
posEnd = squeeze(posEnd);
distEnd = vecnorm(posEnd'); % If A is a matrix, then vecnorm returns the norm of each column.
evaluations=evaluatorFunc(distEnd,distLimit);


if sumType == 0
    estimate = sum(evaluations) / (runNumber); % indicator func sum columnwise summation
else
    estimate = cumsum(evaluations) / (1:runNumber+1); % columnwise summation
end
end

% generate a set of steps for a random walk

function X = generateRandomWalkStepsNormalIS(runNumber,numSteps,mu,sigma,seed)
% rng seed fix
s = rng;
s.Seed = seed;
rng(s.Seed)

X = sigma*randn(runNumber,numSteps) + mu; % sample from a unifrom

end

function [estimate, samples, evaluations] =
ISmonteCarloNormal(runNumber,numSteps,seed,mu1,sigma1,mu2,sigma2,distLimit,sumType)

fstepsX = generateRandomWalkStepsNormal(runNumber,numSteps,seed(1));
fstepsY = generateRandomWalkStepsNormal(runNumber,numSteps,seed(2));
fstepsZ = generateRandomWalkStepsNormal(runNumber,numSteps,seed(3));
frandomWalk = cat(3,fstepsX,fstepsY,fstepsZ);
fsamples = frandomWalk;

frandomWalkT = cat(3,(frandomWalk(:,:,1))',(frandomWalk(:,:,2))',(frandomWalk(:,:,3))');
fposEnd = sum(frandomWalkT);
fposEnd = squeeze(fposEnd);
fdistEnd = vecnorm(fposEnd'); % If A is a matrix, then vecnorm returns the norm of each column.
```

```
qstepsX = generateRandomWalkStepsNormalIS(runNumber,numSteps,mu2,sigma2,seed(1));
qstepsY = generateRandomWalkStepsNormalIS(runNumber,numSteps,mu2,sigma2,seed(2));
qstepsZ = generateRandomWalkStepsNormalIS(runNumber,numSteps,mu2,sigma2,seed(3));
qrandomWalk = cat(3,qstepsX,qstepsY,qstepsZ);
qsamples = qrandomWalk;

qrandomWalkT = cat(3,(qrandomWalk(:,:,1))',(qrandomWalk(:,:,2))',(qrandomWalk(:,:,3))');
qposEnd = sum(qrandomWalkT);
qposEnd = squeeze(qposEnd);
qdistEnd = vecnorm(qposEnd'); % If A is a matrix, then vecnorm returns the norm of each column.

q_x=normalDistPDF(mu2,sigma2,qsamples);
f_x=normalDistPDF(mu1,sigma1,qsamples);
g_x  = evaluatorFunc(qdistEnd,distLimit);
plikelihoodfq = zeros(runNumber,1);
likelihoodfq   = f_x./q_x;

for i=1:runNumber
likelihoodStep = squeeze(likelihoodfq(i,:,:));
pEachStep = prod(likelihoodStep');
pEachRun = prod(pEachStep);
plikelihoodfq(i,1) = pEachRun;
end

evaluations    = g_x.*plikelihoodfq';
samples = qrandomWalk;

if sumType == 0
    estimate = sum(evaluations) / (runNumber); % indicator func sum columnwise summation
else
    estimate = cumsum(evaluations) / (1:runNumber+1); % columnwise summation
end
end

function evaluatorFuncOut = evaluatorFunc(totDist, threshold)


g_xTotDist = totDist;
g_xTotDist (totDist <= threshold) = 0;
g_xTotDist (totDist > threshold) = 1;
probDistLimit = sum(g_xTotDist)/length(g_xTotDist);
evaluatorFuncOut = g_xTotDist;
end

function p = normalDistPDF(mu,sigma,x)
p = (1/(sigma*sqrt(2*pi)))*exp(-0.5*((x-mu)/sigma).^2);
end
```

4.4. Appendix 3

```
% 567
% 4.1

clear all
close all
clc

seed = 1333; % rng seed for reproducibility
rng; %seed fix
s = rng;
s.Seed = seed;
rng(s.Seed);
```

```
mu = 0.05;
sigma = 0.2;
Y_0 = 1;
% 0<t<1
tFinal = 1;
% dt =1/N;
dt = 0.05;
% N = 1/dt;
runNumber = 100;


%% PART 1

W = bms(tFinal, dt, runNumber);
Y = gbms(W,mu,sigma,Y_0,dt);

% plots

timeVec = 0:dt:tFinal;

figure(1)
plot(timeVec,W')
title('Question 4.1: Part 1','Wiener Process ')
xlabel('Time')
ylabel('Walked Distance')
legend('Wiener Process_n','Location','best')
grid on

figure(2)
plot(timeVec,Y')
title('Question 4.1: Part 1','Geometric Brownian Motion')
xlabel('Time')
ylabel('Walked Distance')
legend('Geometric Brownian Motion_n','Location','best')
grid on


var_Y=var(Y(:,end));
exp_Y=mean(Y(:,end));


AnalyticExp_Y = exp(mu - (sigma^2)/2 + (sigma^2)/2 );
AnalyticVar_Y = exp(2*mu - sigma^2 + 2*sigma^2)- exp(2*mu - sigma^2 + sigma^2);

%% PART 2

runNumber = 1;
m=4;
dtm = dt*m;    % coarse time step


Wf = bms(tFinal, dt, runNumber,seed);
Wc = bmf2c(tFinal, dtm, runNumber, Wf, m);

Yf = gbms(Wf,mu,sigma,Y_0,dt);
Yc = gbms(Wc,mu,sigma,Y_0,dtm);

% plots

timeVecC = 0:dtm:tFinal;
timeVecF = 0:dt:tFinal;

figure(3)
plot(timeVecF,Wf,'m-x','LineWidth',2,'MarkerSize',8)
```

```
hold on
plot(timeVecC,Wc,'b-o','LineWidth',2,'MarkerSize',8)
title('Question 4.1: Part 2','Wiener Process vs dt')
xlabel('Time')
ylabel('Walked Distance')
legend('Fine Wiener Process','Coarse Wiener Process','Location','best')
grid on

figure(4)
plot(timeVecF,Yf,'m-x','LineWidth',2,'MarkerSize',8)
hold on
plot(timeVecC,Yc,'b-o','LineWidth',2,'MarkerSize',8)
title('Question 4.1: Part 2','Geometric Brownian Motion vs dt')
xlabel('Time')
ylabel('Walked Distance')
legend('Fine Geometric Brownian Motion','Coarse Geometric Brownian Motion','Location','best')
grid on

%% PART 3

runNumber = 1000;
i = [2 3 4 5];

dt = 4^-i(4);
Wf = bms(tFinal, dt, runNumber);
Yf = gbms(Wf,mu,sigma,Y_0,dt);

m1=4;
dtc1 = dt*m1;    % coarse time step 1
Wc1 = bmf2c(tFinal, dtc1, runNumber, Wf, m1);
Yc1 = gbms(Wc1,mu,sigma,Y_0,dtc1);

m2=4^2;
dtc2 = dt*m2;
Wc2 = bmf2c(tFinal, dtc2, runNumber, Wf, m2);
Yc2 = gbms(Wc2,mu,sigma,Y_0,dtc2);

m3=4^3;
dtc3 = dt*m3;
Wc3 = bmf2c(tFinal, dtc3, runNumber, Wf, m3);
Yc3 = gbms(Wc3,mu,sigma,Y_0,dtc3);

% plots

timeVecF = 0:dt:tFinal;
timeVecC1 = 0:dtc1:tFinal;
timeVecC2 = 0:dtc2:tFinal;
timeVecC3 = 0:dtc3:tFinal;

Yf_end = Yf(:,end);
Yc1_end = Yc1(:,end);
Yc2_end = Yc2(:,end);
Yc3_end = Yc3(:,end);

V_Yc3 = var(Yc3_end);
V_Yc2Yc3 = var(Yc2_end-Yc3_end);
V_Yc1Yc2 = var(Yc1_end-Yc2_end);
V_YfYc1 = var(Yf_end-Yc1_end);

VarY = [V_Yc3 V_Yc2Yc3 V_Yc1Yc2 V_YfYc1];

V_Yf = V_Yc3 + V_Yc2Yc3 + V_Yc1Yc2 + V_YfYc1;

E_Yc3    = sum(Yc3_end)/length(Yc3_end);
```

```
E_Yc2Yc3 = sum(Yc2_end-Yc3_end)/length(Yc2_end-Yc3_end);
E_Yc1Yc2 = sum(Yc1_end-Yc2_end)/length(Yc1_end-Yc2_end);
E_YfYc1  = sum(Yf_end-Yc1_end)/length(Yf_end-Yc1_end);

E_Yf = E_Yc3 + E_Yc2Yc3 + E_Yc1Yc2 + E_YfYc1;
EY   = [E_Yc3  E_Yc2Yc3  E_Yc1Yc2  E_YfYc1];

% P

P_Yc3 = exp(-0.05)*max(0,(Yc3_end)-1);
P_Yc2 = exp(-0.05)*max(0,(Yc2_end)-1);
P_Yc1 = exp(-0.05)*max(0,(Yc1_end)-1);
P_Yf = exp(-0.05)*max(0,(Yf_end)-1);

P_Yc2Yc3 = P_Yc2-P_Yc3;
P_Yc1Yc2 = P_Yc1-P_Yc2;
P_YfYc1  = P_Yf-P_Yc1;

% P Expectation

EP_Yc3    = sum(P_Yc3)/length(P_Yc3);
EP_Yc2Yc3 = sum(P_Yc2Yc3)/length(P_Yc2Yc3);
EP_Yc1Yc2 = sum(P_Yc1Yc2)/length(P_Yc1Yc2);
EP_YfYc1  = sum(P_YfYc1)/length(P_YfYc1);

EP_Yf = EP_Yc3 + EP_Yc2Yc3 + EP_Yc1Yc2 + EP_YfYc1;
EYP   = [EP_Yc3 EP_Yc2Yc3 EP_Yc1Yc2 EP_YfYc1];

% P Variance

VP_Yc3 = var(P_Yc3);
VP_Yc2Yc3 = var(P_Yc2Yc3);
VP_Yc1Yc2 = var(P_Yc1Yc2);
VP_YfYc1  = var(P_YfYc1);

VP_Yf = VP_Yc3 + VP_Yc2Yc3 + VP_Yc1Yc2 + VP_YfYc1;
VarYP = [VP_Yc3 VP_Yc2Yc3 VP_Yc1Yc2 VP_YfYc1];

figure(7)
bar((VarY))
set(gca,'YScale','log')
title('Question 4.1: Part 3','Geometric Brownian Motion MLMC Variance vs Levels')
xlabel('Levels')
ylabel('log(Variance)')
legend('Geometric Brownian Motion Variances','Coarse Geometric Brownian Motion #1','Coarse Geometric
Brownian Motion #2','Coarse Geometric Brownian Motion #3','Location','best')
grid on


figure(8)
bar((VarYP))
set(gca,'YScale','log')
title('Question 4.1: Part 3','Geometric Brownian Motion Pay Off Function MLMC Variance vs Levels')
xlabel('Levels')
ylabel('log(Variance)')
legend('Geometric Brownian Motion Variances','Coarse Geometric Brownian Motion #1','Coarse Geometric
Brownian Motion #2','Coarse Geometric Brownian Motion #3','Location','best')
grid on



figure(9)
bar((EY))
set(gca,'YScale','log')
```

```
title('Question 4.1: Part 3','Geometric Brownian Motion MLMC Expectation vs Levels')
xlabel('Levels')
ylabel('log(Expectation)')
legend('Geometric Brownian Motion Estimates','Coarse Geometric Brownian Motion #1','Coarse Geometric
Brownian Motion #2','Coarse Geometric Brownian Motion #3','Location','best')
grid on


figure(10)
bar(((EYP)))
set(gca,'YScale','log')
title('Question 4.1: Part 3','Geometric Brownian Motion Pay Off Function MLMC Expectation vs Levels')
xlabel('Levels')
ylabel('log(Expectation)')
% ylim([10^-6 10^-1])
legend('Geometric Brownian Motion Estimates','Coarse Geometric Brownian Motion #1','Coarse Geometric
Brownian Motion #2','Coarse Geometric Brownian Motion #3','Location','best')
grid on

%% PART 4

t4=0.029989;
t3= 0.032751;
t2=0.041187;
t1=0.054871;

t= t1+t2+t3+t4;

normalized_t4 = t4/t1;
normalized_t3 = t3/t1;
normalized_t2 = t2/t1;
normalized_t1 = t1/t1;
tarVar=0.005;
lambda =
tarVar^(-2)*(sqrt(normalized_t4*VP_Yc3)*sqrt(normalized_t3*VP_Yc2Yc3)*sqrt(normalized_t2*VP_Yc1Yc2)*sqr
t(normalized_t1*VP_YfYc1))^2;
N_L = lambda.*sqrt([VP_Yc3 VP_Yc2Yc3 VP_Yc1Yc2 VP_YfYc1].\[normalized_t4 normalized_t3 normalized_t2
normalized_t1]);

function bmsOut = bms(tFinal, dt, runNumber)
sampleNum = ceil(tFinal/dt);
samples = [zeros(runNumber,1) randn(runNumber,sampleNum)*sqrt(dt)];
bmsOut=cumsum(samples'); %If A is a matrix, then cumsum(A) returns a matrix containing the cumulative
sums for each column of A.
end

function gbmsOut = gbms(W,mu,sigma,Y_0,dt)
W=W';
sizeW = size(W);
Y(1:sizeW(1),1) = Y_0;
for i=1:sizeW(2)-1
    b = mu * Y(:,i);
    h = sigma * Y(:,i);
    dW = W(:,i+1)-W(:,i);
    Y(:,i+1) = Y(:,i) + b.*dt + h.*dW;
end
gbmsOut=Y;
end

function bmf2cOut = bmf2c(tFinal, dt, runNumber, bf, m)
sampleNum = ceil(tFinal/dt)+1;
bc = zeros(sampleNum,runNumber);
bc(1,:) = 0;
for i = 2:sampleNum
```

```
    delta = bf((i-1)*m+1,:) - bf((i-2)*m+1,:);
    bc(i,:) = bc(i-1,:) + delta;
end
bmf2cOut = bc;
end
```

AE567

Statistical Inference, Estimation, and Learning

Project 2:

Part 1: Bayesian Inference and Decisions

Deniz Dost

UM ID: 82337583

Oct. 26, 2021
Deniz Dost, 82337583

1. Introduction

The problem statement is given with a set of initial data from a sensor on a flying robot as shown below in Table 1. The sensor outputs are given according to the x and y coordinates. The output basically shows the strength of the scent to localize a buried object. The field in consideration is two-dimensional $(X, Y) : [-1,1] \times [-1,1]$.

X	Y	Sensor
0.1	0.05	3.39382006
-0.9	0.3	3.2073034
0.2	0.4	3.39965035
0.8	-0.3	3.68810201
-0.6	0.3	2.96941623
0.3	-0.2	2.99495501
0.5	-0.84	3.94274928
-0.5	0.85	2.7968011
-0.01	-0.76	3.34929734
-0.9	-0.9	3.91296165

Table 1: Initial Sensor Data

The initial set of data points can be seen in three dimension in Figure 1.

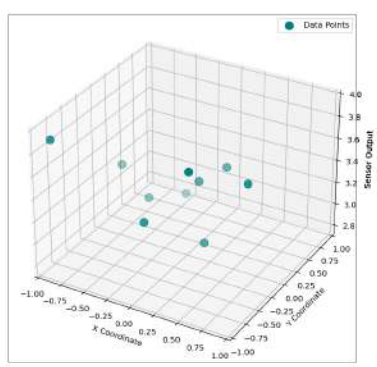

Figure 1: Initial Set of Data Points

2. GPR Setting

In order to solve this problem, an algorithm for non-parametric Gaussian Process Regression (GPR) is used. This algorithm basically uses the training data (training inputs and training outputs), such as the data provided in Table 1, and then computes the array of mean and the covariance matrix at prediction points. The computation for the mean is done according to the following formula:

$$E[f(x)\,|\,d] = \mu(x,y) + k(x,\bar{x},y,\bar{y})(k(x,\bar{x},y,\bar{y}) + \sigma^2 I)^{-1}(\bar{z} - \mu(\bar{x},\bar{y}))$$

And the computation for covariance is done according to the following formula:

$$Cov(f(x), f(x)\,|\,d) = k(x,x,y,y) - k(x,\bar{x},y,\bar{y})(k(\bar{x},\bar{x},\bar{y},\bar{y}) + \sigma^2 I)^{-1}(k(\bar{x},x,\bar{y},y))$$

where;

- d is the data, $(\bar{x},\bar{y})$ as the x and y coordinates of the sensor data and $\bar{z}$ as the sensor output value,
- (x,y) is the prediction point coordinates
- σ^2 is the noise variance,
- μ is the prior mean function, which is calculated from the data set for this problem setting by simply averaging the given sensor data, and
- $k(x,y)$ is the covariance kernel function which outputs a covariance matrix.

The covariance kernel is a function between two arbitrary points which computes the covariance. Here, the squared exponential kernel is used which is a simple algorithm to work with two dimensional problem like in this case with only two parameters to be tuned. The squared exponential kernel is formulated as below for the one dimensional case:

$$k(x,x';\tau,l) = \tau\ e^{(-0.5\frac{(x-x')^2}{l^2})}$$

This can be converted into the two dimensional case for this problem statement as below:

$$k(x,x',y,y';\tau,l) = k(x,x')k(y,y')$$

where

$$k(x, x'; \tau_1, l) = \tau_1 \; e^{(-0.5\frac{(x-x')^2}{l^2})}$$

$$k(y, y'; \tau_2, l) = \tau_2 \; e^{(-0.5\frac{(y-y')^2}{l^2})}, \quad where \; \tau_2 = 1$$

Inference is performed directly for the predictions. Nonparametric GPR does not require a basis function. Nonparametric GPR is used in this problem since the data provided is only a small amount and the computational cost associated with it, offline cost of the number of data points and online cost of inversion which is its third power, would not be much.

For this problem, the field is discretized by $n = 50$, where $X = linspace([-1 : n : 1])$ and $Y = linspace([-1 : n : 1])$, to create prediction points. τ, l, and σ^2 are chosen to be 0.3, 0.2, 10^{-8} respectively starting with the numbers used in examples and modifying them by hand to a point.

3. Initial GPR Results

When the GPR function is run with the data provided in the previous section, the result is computed as shown in Figure 2 with the sensor data shown in black dots. A surface graph as shown in Figure 3 can be obtained to qualitatively observe the trends in the mean (upper surface) and variance (lower surface) in a better way.

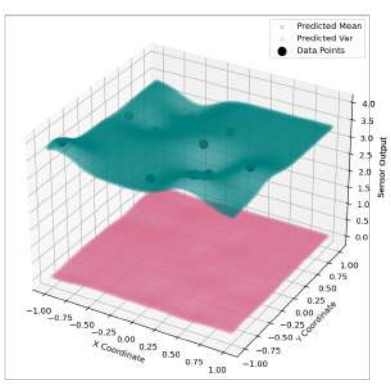

Figure 2: Mean and Variance at Prediction Points with Initial Data Set

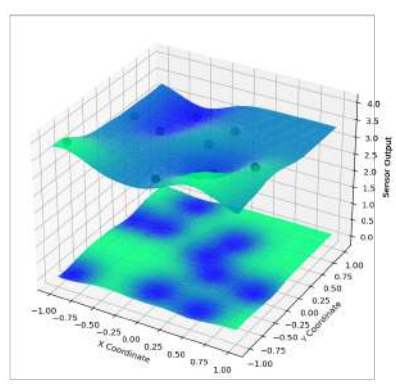

Figure 3: Mean and Variance Surfaces at Prediction Points with Initial Data Set

There are clearly some ups and downs in both mean and variance at the prediction points. It would be preferred to examine further sensor data at some of these. However, only looking at the mean and variance may not be enough on this decision. A further analysis can be done by combining them. In this case, computing the standard deviation from the variation and a look at the mean ± 2(standard deviation) value as shown in Figure 4 could be useful. This way, the information both from the mean, on how strong the scent would be, and the variance, how certain/spread the output is, can be used simultaneously. Again, a surface plot would be useful to understand the trend as shown in Figure 5. Here, it is observed that some peaks in the mean are between a smaller space encapsulated by the lower and upper surfaces, and some are in larger spaces.

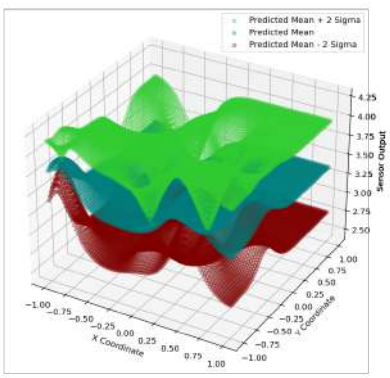

Figure 4: Mean ± 2 Standard Deviation at Prediction Points with Initial Data Set

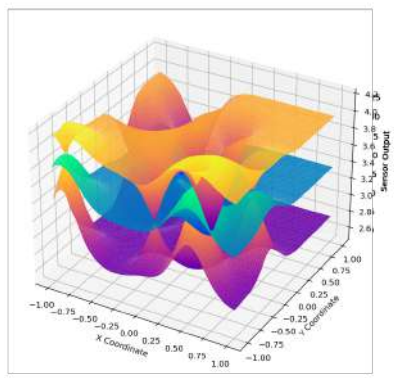

Figure 5: Mean ± 2 Standard Deviation Surfaces at Prediction Points with Initial Data Set

With the mean values at prediction points, there is the expectation of scent strength from the sensor. A look at the standard deviation alongside with mean and mean ± 2(standard deviation) can be seen in Figure 6 and a clearer surface graph with standard deviation at the very bottom in Figure 7. This way, the peaks of mean can be compared with its standard deviation to make an educated guess on the further sensor readings to be acquired. Low standard deviation areas are the certain areas. High standard deviation areas are the spread areas, where if a peak exists around would mean that there is uncertainty on that area and the scent could also be coming from there.

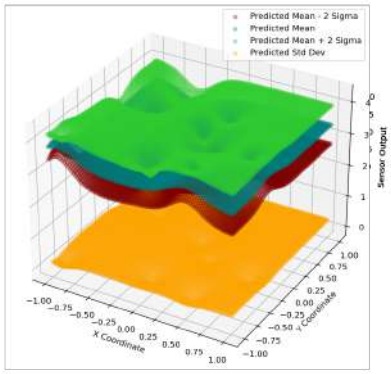

Figure 6: Mean ± 2 Standard Deviation and Standard Deviation at Prediction Points with Initial Data Set

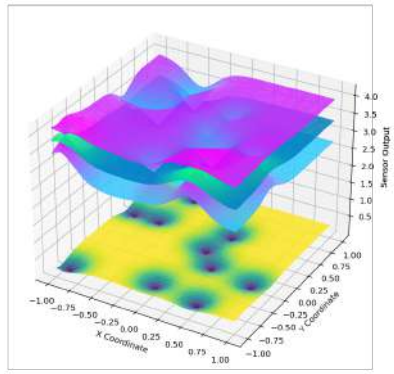

Figure 7: Mean ± 2 Standard Deviation and Standard Deviation Surfaces at Prediction Points with Initial Data Set

4. Hyperparameter Selection

The presented results above are obtained through initial τ, l, and σ^2 values. These are the hyper parameters for this problem statement since they are the inputs of the function for inference algorithms used, namely the kernel and GPR. These parameters can be tuned, both intuitively by hand and algorithmically.

One of the ways to tune the hyper parameters algorithmically is by maximizing the marginal likelihood. This method can be used in low data regime, which is also present in this problem statement since only 10 sensor readings are provided. The expression to be maximized is as shown below:

$$log(\bar{y}\,|\,\bar{x},\theta) = -0.5(\bar{y}-\mu(\bar{x}))^T(k(\bar{x},\bar{x})+\sigma^2 I)^{-1}(\bar{y}-\mu(\bar{x})) - 0.5log\,|\,k(\bar{x},\bar{x})+\sigma^2 I\,| - 0.5nlog(2\pi)$$

where $\bar{x}$ is the sensor rating coordinates (for this problem setting $(\bar{x},\bar{y})$) and $\bar{y}$ is the sensor outputs at these corresponding coordinates.

So, a likelihood function is created to compute the likelihood to be maximized. Then, the python built in fiction for minimization is used to minimize the negative of this likelihood. That way, an optimization for the hyperparameters are made. The output for the hyperparameters is 3.78974722e-01 2.64125695e-01 5.54443024e-09 respectively for τ, l, and σ^2 for the initial set of data.

The results obtained when the GPR is done with the optimized hyper parameters are shown in Figure 8, and as surfaces in Figure 9. The trends are similar since the optimized values do not differ drastically from the initial values.

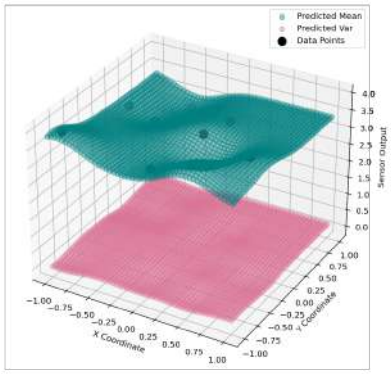

Figure 8: Mean and Variance at Prediction Points with Initial Data Set with Optimized Hyperparameters

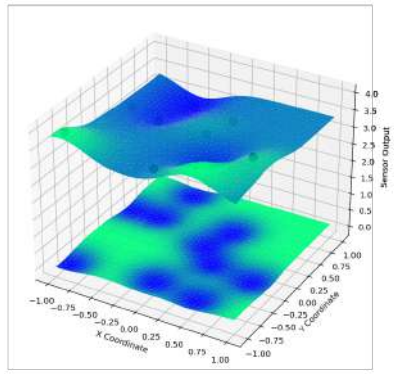

Figure 9: Mean and Variance Surfaces at Prediction Points with Initial Data Set with Optimized Hyperparameters

In order to take both the mean and variance into account, Figure 10 and Figure 11 are constructed as below. The figures presented can be used to decide on points to further get readings from at each level of this project.

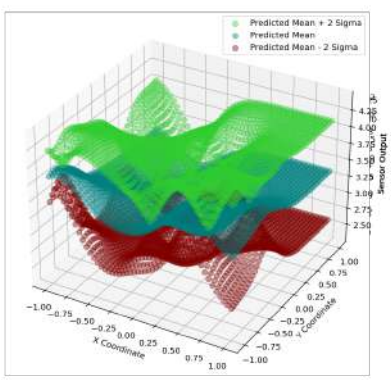

Figure 10: Mean ± 2 Standard Deviation and Mean at Prediction Points with Initial Data Set with Optimized Hyperparameters

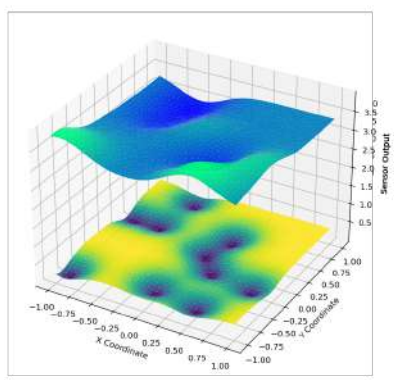

Figure 11: Mean and Standard Deviation Surfaces at Prediction Points with Initial Data Set with Optimized Hyperparameters

The points to get more data points can be selected from the graph by eye too. However, it is also useful to use an algorithm to find these points.

5. Data Point Selection

In order to find the data points to further investigate, three different things can be considered, such as the mean, mean ± 2(standard deviation), and variance at predicted points. The peaks for each data set can be found. Mean peaks exhibit the expected high sensor data which is how strong the scent is. The variance peaks are where the model is having high variance which would mean that the expected scent at that prediction point is more vague. So, these areas would require more data points to inspect. And the mean ± 2(standard deviation) would represent a combination of these two. An algorithm which follows the following method is run to find the coordinates of the peaks at each of the aforementioned as shown below:

- Create a pad of zeros at the perimeter of the matrix to avoid undefined evaluations at the corners and sides
- For all y coordinates check every x coordinate
- Evaluate the value (mean, mean ± 2(standard deviation), and variation)
- See if its bigger than all of its eight neighbours

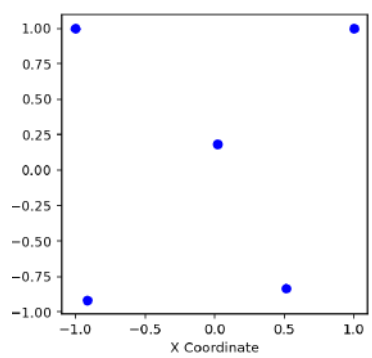

Figure 12: Peak Point Coordinates for Mean

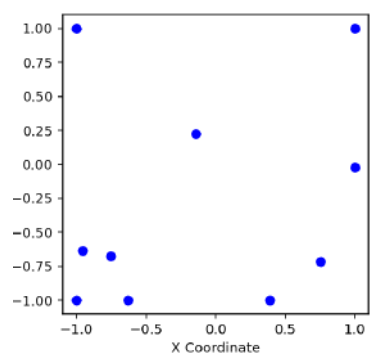

Figure 13: Peak Point Coordinates for Mean ± 2(standard deviation)

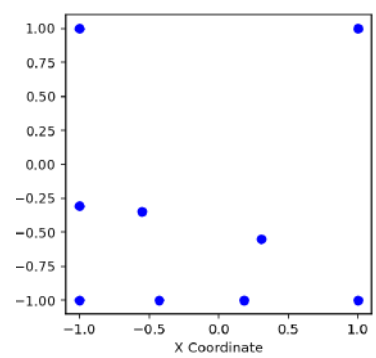

Figure 14: Peak Point Coordinates for Variance

The coordinates obtained for the three set can be seen in Figures 12-14. The values are examined at these points. The observation can also be done visually from the Figures 9-11. Points shown in Figure 13 were of interest since they show the behavior

of both mean and spread. Two points were selected from that, and the other two were selected form the mean and variance considering the visual evaluation of these two at the same time. This way, a set of coordinates to require sensor readings are obtained.

See Appendix 1 for the code of this part for the analysis with the initial data set.

6. Analysis with the First Data Set

The initial data set is now expanded with the additional data requested according to the previous section. So, the first data set is as shown below in Table 2 with the additional data shown at the bottom:

X	Y	Sensor
0.1	0.05	3.39382006
-0.9	0.3	3.2073034
0.2	0.4	3.39965035
0.8	-0.3	3.68810201
-0.6	0.3	2.96941623
0.3	-0.2	2.99495501
0.5	-0.84	3.94274928
-0.5	0.85	2.7968011
-0.01	-0.76	3.34929734
-0.9	-0.9	3.91296165
0.51	-0.84	3.96070842
0.75	-0.7	3.91571481
-0.75	-0.7	3.57317869
0.18	-1	3.80625401

Table 2: First Set of Sensor Data

The first set of data points can be seen in three dimension in Figure 15:

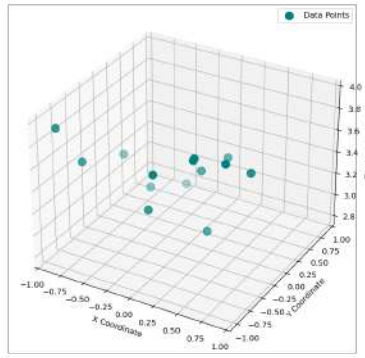

Figure 15: First Set of Sensor Data

In this part onward, only the analysis with the optimized hyper parameters are included. When the marginal likelihood is minimized, the output for the hyperparameters follows as 3.91039125e-01 3.58599428e-01 7.27023913e-05 respectively for τ, l, and σ^2. When these parameters are used to obtain results from the GPR algorithm, the output is as shown in Figure 16, and in Figure 17 as mean surface at the top and variance surface at the bottom. Mean surface looks to have sharper peaks in a sense while the main trend seems similar. Also, variance is flatter in more area.

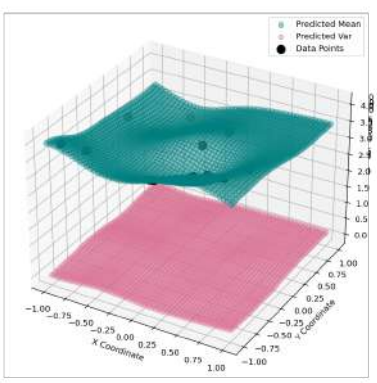

Figure 16: Mean and Variance at Prediction Points with the First Data Set with Optimized Hyperparameters

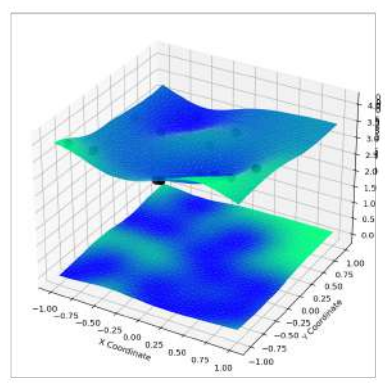

Figure 17: Mean and Variance at Prediction Points with the First Data Set with Optimized Hyperparameters

This makes sense since the data points have increased, which decrease the variance at these coordinates as shown in Figure 19. It is also observed from Figure 18, in a closer look at the mean, that the data points fit on the predicted mean surface. There seems to be another peak emerging behind the most obvious peak as a difference from the initial data set results.

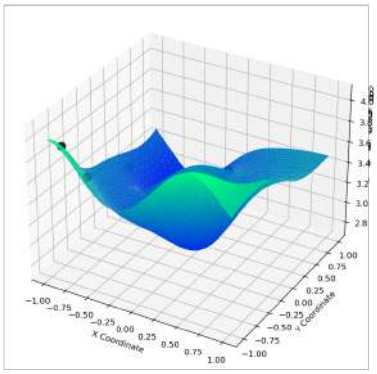

Figure 18: Mean at Prediction Points with the First Data Set with Optimized Hyperparameters

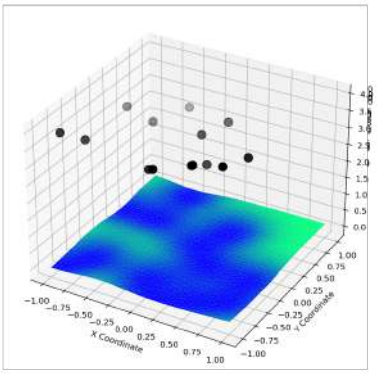

Figure 19: Variance at Prediction Points with the First Data Set with Optimized Hyperparameters

As before, mean ± 2(standard deviation) evaluations can be seen in Figure 20 as well as the standard deviation (at the bottom) and the mean (on the top) surfaces in Figure 21. The standard deviation trend is intuitively same with the variance trend as before. These plots are used to decide on sampling points as described in the previous sections.

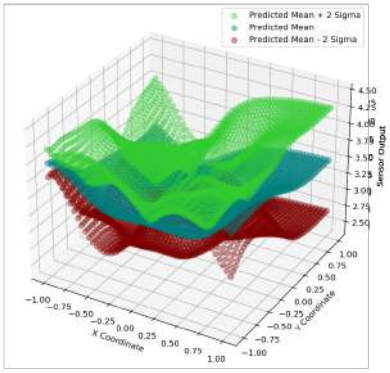

Figure 20: Mean ± 2 Standard Deviation and Mean at Prediction Points with First Data Set with Optimized Hyperparameters

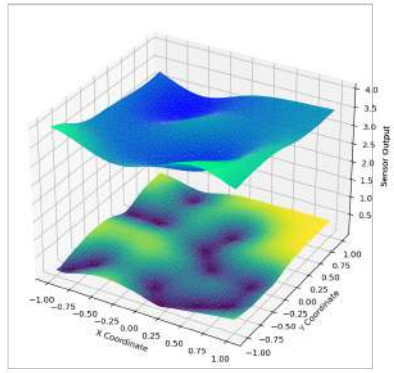

Figure 21: Mean and Standard Deviation Surfaces at Prediction Points with the First Data Set with Optimized Hyperparameters

When the peaks at each of these sets are evaluated, the following figures, Figure 22-24 are obtained.

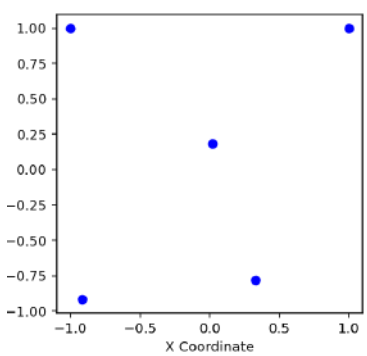

Figure 22: Peak Point Coordinates for Mean

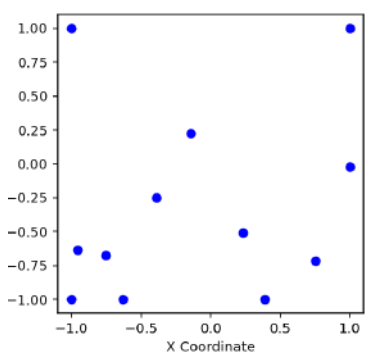

Figure 23: Peak Point Coordinates for Mean ± 2(standard deviation)

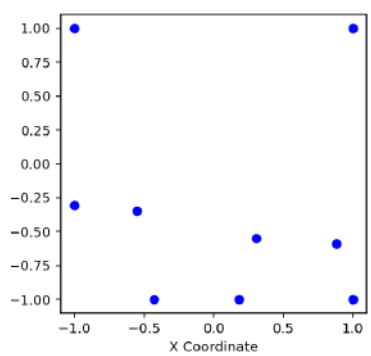

Figure 24: Peak Point Coordinates for Variance

These points are evaluated and their combination is also taken into account when requiring new data points as described previously.

See Appendix 2 for the code of this part for the analysis with the first data set.

7. Analysis with the Second Data Set

The first data set is now expanded with the additional data requested according to the previous section. So, the second data set is as shown below in Table 3 with the additional data shown at the bottom:

X	Y	Sensor
0.1	0.05	3.39382006
-0.9	0.3	3.2073034
0.2	0.4	3.39965035
0.8	-0.3	3.68810201
-0.6	0.3	2.96941623
0.3	-0.2	2.99495501
0.5	-0.84	3.94274928
-0.5	0.85	2.7968011
-0.01	-0.76	3.34929734
-0.9	-0.9	3.91296165
0.51	-0.84	3.96070842
0.75	-0.7	3.91571481
-0.75	-0.7	3.57317869
0.18	-1	3.80625401
0.33	-0.78	3.60667046
0.88	-0.59	3.71839899
0.23	-0.51	4.36210715
-0.39	-0.25	3.45358599

Table 3: Second Set of Sensor Data

The second set of data points can be seen in three dimension in Figure 25:

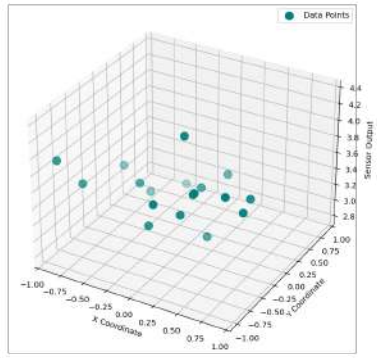

Figure 25: Second Set of Sensor Data

The output for the hyperparameters is 3.94739573e-01 1.57105366e-01 1.07775013e-08 respectively for τ, l, and σ^2. When these parameters are used to obtain results from the GPR algorithm, the output is as shown in Figure 26, and in Figure 27 as mean surface at the top and variance surface at the bottom. There is an obvious different peak in the top plot of Figure 27. The variance is also low on sensor data points.

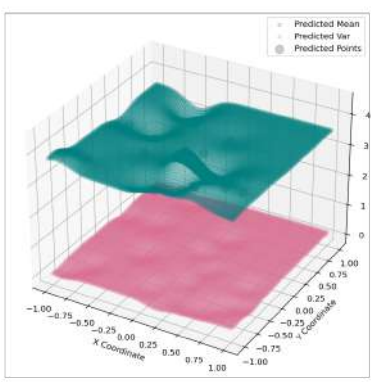

Figure 26: Mean and Variance at Prediction Points with the Second Data Set with Optimized Hyperparameters

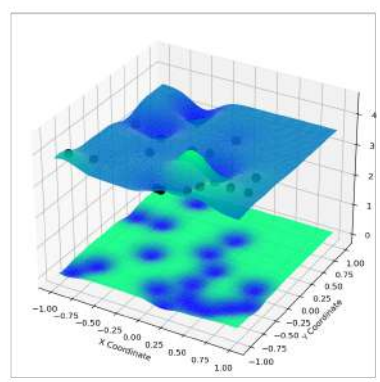

Figure 27: Mean and Variance Surfaces at Prediction Points with the Second Data Set with Optimized Hyperparameters

This makes sense since the data points have increased in that area, which decrease the variance at these coordinates as shown in Figure 29. It is also observed from Figure 28,

in a closer look at the mean, that the data points fit on the predicted mean surface. The peak emerging behind the previous peak as a difference from the first data set results is observed qualitatively.

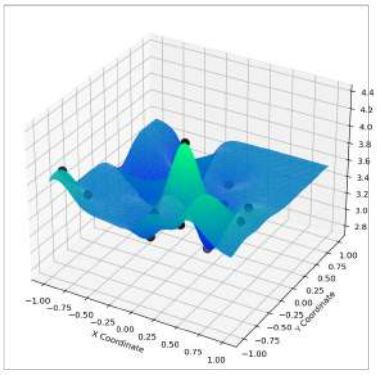

Figure 28: Mean at Prediction Points with the Second Data Set with Optimized Hyperparameters

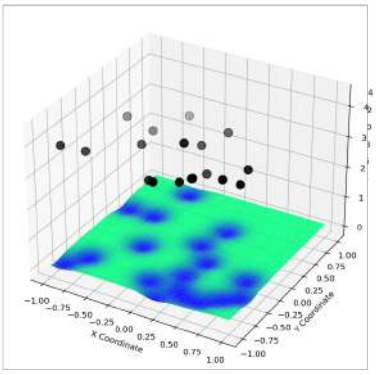

Figure 29: Variance at Prediction Points with the Second Data Set with Optimized Hyperparameters

As before, mean ± 2(standard deviation) evaluations can be seen in Figure 30 as well as the standard deviation (at the bottom) and the mean (on the top) surfaces in Figure 31. The standard deviation trend is intuitively same with the variance trend as before. And the standard deviation at the new peak is low. These plots are used to decide on sampling points as described in the previous sections.

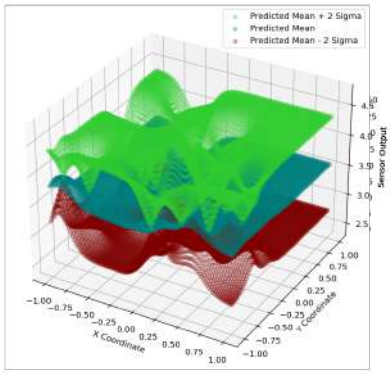

Figure 30: Mean ± 2 Standard Deviation and Mean at Prediction Points with the Second Data Set with Optimized Hyperparameters

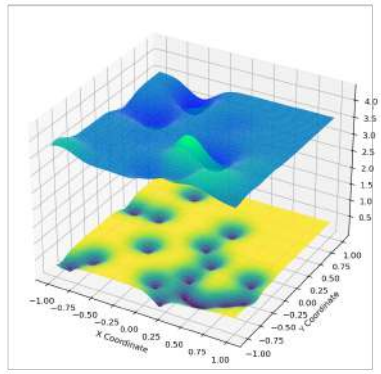

Figure 31: Mean and Standard Deviation Surfaces at Prediction Points with the Second Data Set with Optimized Hyperparameters

When the peaks at each of these sets are evaluated, the following figures, Figure 32-34 are obtained.

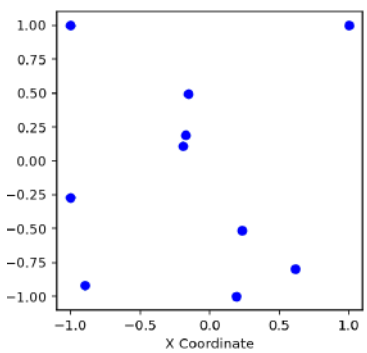

Figure 32: Peak Point Coordinates for Mean

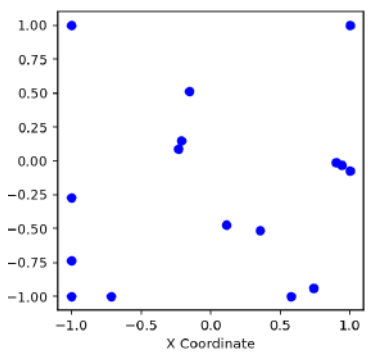

Figure 33: Peak Point Coordinates for Mean ± 2(standard deviation)

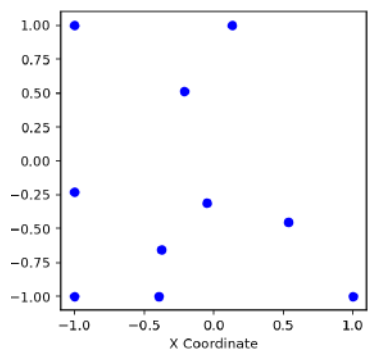

Figure 34: Peak Point Coordinates for Variance

The first data point is obtained by examining the mean surface where the last one was obtained by examining the variance. The other two is from the mean ± 2(standard deviation) examination. Also, their relationships, when is one high/low where the other is high/low is also considered when making this decision.

See Appendix 3 for the code of this part for the analysis with the second data set.

8. Analysis with the Third Data Set

The third data set is as shown below in Table 4 with the additional data shown at the bottom:

X	Y	Sensor
0.1	0.05	3.39382006
-0.9	0.3	3.2073034
0.2	0.4	3.39965035
0.8	-0.3	3.68810201
-0.6	0.3	2.96941623
0.3	-0.2	2.99495501
0.5	-0.84	3.94274928
-0.5	0.85	2.7968011
-0.01	-0.76	3.34929734
-0.9	-0.9	3.91296165
0.51	-0.84	3.96070842
0.75	-0.7	3.91571481
-0.75	-0.7	3.57317869
0.18	-1	3.80625401
0.33	-0.78	3.60667046
0.88	-0.59	3.71839899
0.23	-0.51	4.36210715
-0.39	-0.25	3.45358599
0.62	-0.8	4.0757880
0.35	-0.5	4.18858322
-0.15	0.5	4.82464266
-0.05	-0.45	4.05570494

Table 4: Third Set of Sensor Data

The third set of data points can be seen in three dimension in Figure 35:

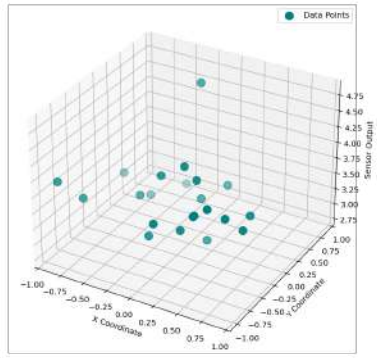

Figure 35: Third Set of Sensor Data

The output for the hyperparameters is 5.03440238e-01 1.97190426e-01 6.65356895e-09 respectively for τ, l, and σ^2. When these parameters are used to obtain results from the GPR algorithm, the output is as shown in Figure 36, and in Figure 37 as mean surface at the top and variance surface at the bottom. There is another peak in the top plot of Figure 37, which reaches the highest value so far. The variance is also flatter on on general compared to the previous.

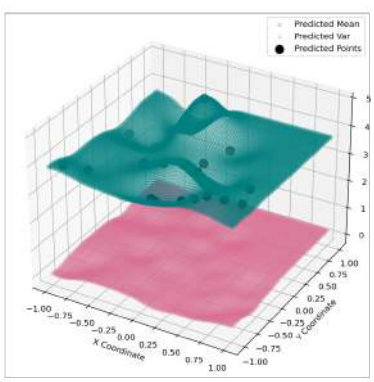

Figure 36: Mean and Variance at Prediction Points with the Third Data Set with Optimized Hyperparameters

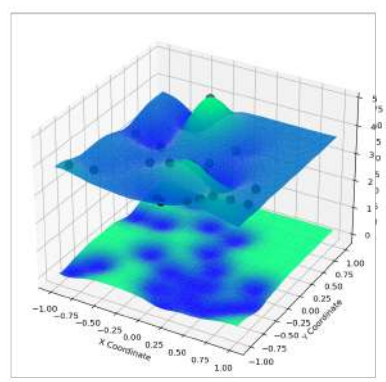

Figure 37: Mean and Variance Surfaces at Prediction Points with the Third Data Set with Optimized Hyperparameters

This makes sense since the data points have increased, which decrease the variance around these coordinates as shown in Figure 39. It is also observed from Figure 38, in

a closer look at the mean, that the data points fit on the predicted mean surface. The new peak emerging is obvious by qualitative examination since it has the highest value so far. Variance also seems to be low in that area in Figure 39.

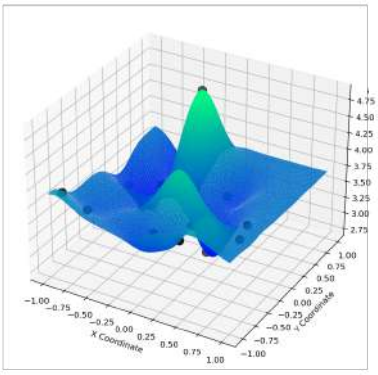

Figure 38: Mean at Prediction Points with the Third Data Set with Optimized Hyperparameters

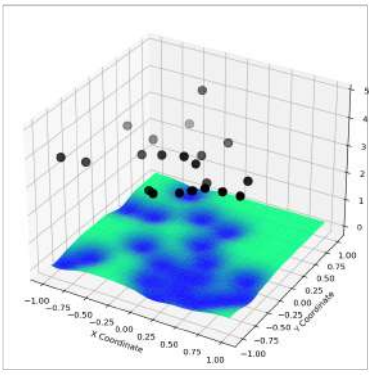

Figure 39: Variance at Prediction Points with the Third Data Set with Optimized Hyperparameters

As before, mean ± 2(standard deviation) evaluations can be seen in Figure 40 as well as the standard deviation (at the bottom) and the mean (on the top) surfaces in Figure 41. The standard deviation trend is intuitively same with the variance trend as before. And the standard deviation at the new peak is low.

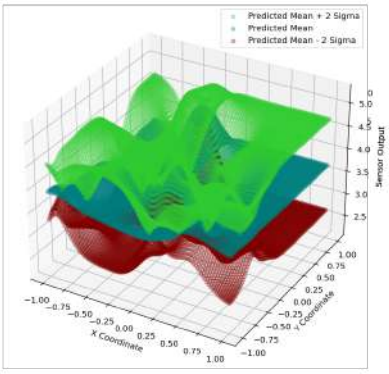

Figure 40: Mean ± 2 Standard Deviation and Mean at Prediction Points with the Third Data Set with Optimized Hyperparameters

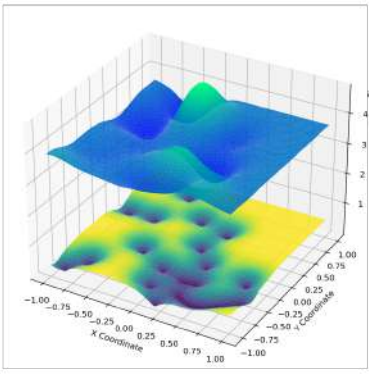

Figure 41: Mean and Standard Deviation Surfaces at Prediction Points with the Third Data Set with Optimized Hyperparameters

When the peaks at each of these sets are evaluated, the following figures, Figure 42-44 are obtained. This time, the low point of variance is also examined, and it is observed that it is actually low at the highest peak, from Figure 45. This makes sense since there is sensor data from that part of the area.

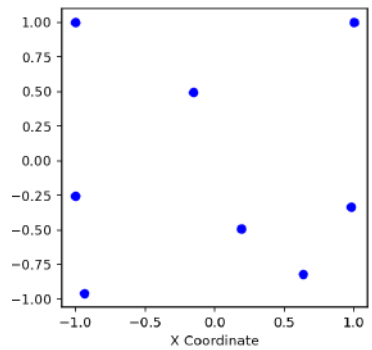

Figure 42: Peak Point Coordinates for Mean

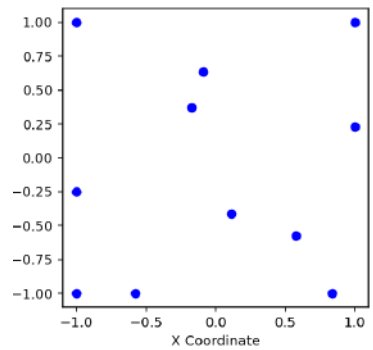

Figure 43: Peak Point Coordinates for Mean ± 2(standard deviation)

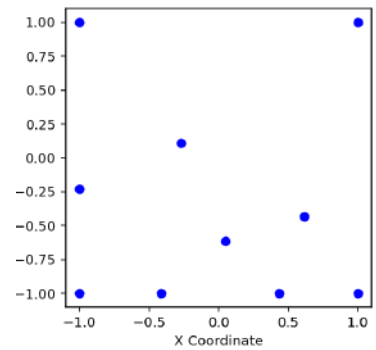

Figure 44: Peak Point Coordinates for Variance

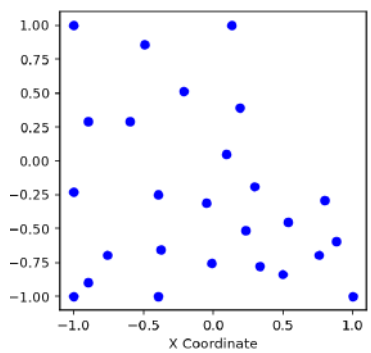

Figure 44: Low Point Coordinates for Variance

The possible location can be interpreted as somewhere around (-0.151515,0.494949) with the expected value of 4.824232 as the scent strength reading. This is commented on by examining the graphs above. Figure 42 depicts the peaks for the mean at the prediction point. Therefore, a high value would indicate high scent. Figure 45 depicts the lowest values of variances for the prediction points. The point with the high mean corresponds to a point with low variance which would indicate that the model is more

certain in that area. The variance at that point is 1.78808797e-04, which is one of the lowest in the set.

See Appendix 4 for the code of this part for the analysis with the third data set.

9. Appendices

Appendix 1:

```
#### APPENDIX 1 #####

%matplotlib inline
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import math
from scipy.optimize import minimize

# Input Values

xySensor  = np.array([[0.1,-0.9,0.2,0.8,-0.6,0.3,0.5,-0.5,-0.01,-0.9],
                  [0.05,0.3,0.4,-0.3,0.3,-0.2,-0.84,0.85,-0.76,-0.9]])
#xSensor   = xySensor[0,:]
#ySensor   = xySensor[1,:]
outSensor =
np.array([3.39382006,3.2073034,3.39965035,3.68810201,2.96941623,2.99495501,3.94274928,2.7968011,3.34929
734,3.91296165])

discretization = 50
xPred =  np.linspace(-1, 1, discretization)
yPred =  np.linspace(-1, 1, discretization)
xPredMesh, yPredMesh = np.meshgrid(xPred,yPred)
xyPred = (np.array([xPredMesh.flatten(),yPredMesh.flatten()]))

MEAN = np.mean(outSensor)

noise = 1e-8
tau = 0.3
l = 0.2

# Plot Data Points

plt.figure(num='Data Points',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='teal',label="Data Points",s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: Kernel

def sqexp(x, xp, tauin, lin):
    """Squared exponential kernel (1 dimensional)

    Inputs
    ------
    x : (N), array of multiple inputs
    xp: float

    Returns
```

```
    -------
    cov (N,) -- Covariance between each input at *x* and the function values at *x*
    """
    cov = tauin**2 * np.exp(-1/2 * (x - xp)**2 / lin**2)
    return cov

def sqexp2d(x, xp, tauin, lin):
    """Squared exponential kernel (2 dimensional) """
    xX   = x[0,:]
    xY   = x[1,:]
    xpX  = xp[0]
    xpY  = xp[1]
    tauX = tauin
    tauY = 1 # tauY=1 bc I only want one tau to tune
    sqexpX = sqexp(xX, xpX, tauX, lin)
    sqexpY = sqexp(xY, xpY, tauY, lin)
    cov = sqexpX*sqexpY
    return cov

# Function: Covariance

def build_covariance(x, xp,tauin, lin, kern):
    """Build a covariance matrix

    Inputs
    -------
    x: (N) array of inputs
    xp: (M) array of inptus
    kern: a function mapping inputs to covariance

    Outputs
    -------
    cov: (N, M) covariance matrix
    """
    out = np.zeros((x.shape[1], xp.shape[1]))
    for jj in range(xp.shape[1]):
        out[:, jj] = kern(x, xp[:,jj], tauin, lin)
    return out

# Function: GPR

def gpr(xtrain, ytrain, xpred, noise_var, mean_func, kernel):
    """Gaussian process regression Algorithm

    Inputs
    -------
    xtrain: (N, ) training inputs
    ytrain: (N, ) training outputs
    xpred:  (M, ) locations at which to make predictions
    noise_var: (N, ) noise at every training output
    mean_func: function to compute the prior mean
    kernel: covariance kernel

    Returns
    -------
    pred_mean : (M, ) predicted mean at prediction points
    pred_cov : (M, M) predicted covariance at the prediction points
    --
    """
    #xtrain  = np.transpose(xtrain)
    cov = build_covariance(xtrain, xtrain, tau, l, kernel)
    u, s, v = np.linalg.svd(cov)
    sqrtcov = np.dot(u, np.sqrt(np.diag(s)))
```

```
    noiseArray=(np.ones(np.size(xtrain[1,:])))*noise_var

    # pseudoinverse is better conditioned
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))

    vec_pred = build_covariance(xpred, xtrain, tau, l, kernel)

    pred_mean = MEAN + np.dot(vec_pred, np.dot(invcov, ytrain - MEAN))
    cov_predict_pre = build_covariance(xpred, xpred, tau, l, kernel)
    cov_predict_up = np.dot(vec_pred, np.dot(invcov, vec_pred.T))
    pred_cov = cov_predict_pre - cov_predict_up
    #print(cov_predict_pre)

    return pred_mean, pred_cov

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.show()

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.2)
fig1.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label='Predicted Var', alpha=0.2)
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: ± 2 Sigma Pred Computation

def twoSigma(inputMean, inputCov):
    varPred    = np.array(np.diag(inputCov))
    stdDevPred = np.sqrt(varPred)
    P2SigmaPred = inputMean + (2*stdDevPred)
    N2SigmaPred = inputMean - (2*stdDevPred)
    plt.figure(num='Pred Mean  ± 2 Sigma Pred Computation',figsize=(10,8))
    ax = plt.axes(projection='3d')
       ax.scatter3D(xPredMeshF,yPredMeshF,P2SigmaPred,c='limegreen',label='Predicted  Mean  +  2  Sigma',  alpha=0.4,
s=40)
    ax.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.4, s=40)
```

```
    ax.scatter3D(xPredMeshF,yPredMeshF,N2SigmaPred,c='maroon',label='Predicted Mean - 2 Sigma', alpha=0.4, s=40)
    locs, labels = plt.xticks()  # Get the current locations and labels.
    plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    ax.set_zlabel("Sensor Output")
    plt.legend()
    plt.show(ax)
    return(stdDevPred,P2SigmaPred,N2SigmaPred)

stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

# Function: ± 2 Sigma Pred Computation

def twoSigma2(inputMean, inputCov):
    varPred    = np.array(np.diag(inputCov))
    stdDevPred = np.sqrt(varPred)
    P2SigmaPred = inputMean + (2*stdDevPred)
    N2SigmaPred = inputMean - (2*stdDevPred)
    plt.figure(num='Pred Mean  ± 2 Sigma Pred Computation',figsize=(10,8))
    ax = plt.axes(projection='3d')
    ax.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
    ax.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaPred,cmap='plasma',label='Predicted Mean + 2 Sigma')
    ax.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaPred,cmap='plasma',label='Predicted Mean - 2 Sigma')
    locs, labels = plt.xticks()  # Get the current locations and labels.
    plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    ax.set_zlabel("Sensor Output")
    plt.show(ax)
    return(stdDevPred,P2SigmaPred,N2SigmaPred)



# ± 2 Sigma Pred Computation
varPred,stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma2(meanPred, covPred)

# STD Pred Plot
plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig3.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='viridis',label="Predicted Points")
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig3.set_zlabel("Sensor Output")
plt.show()

# STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.scatter3D(xPredMeshF,yPredMeshF,N2SigmaMeanPred,c='maroon',label='Predicted Mean - 2 Sigma', alpha=0.3)
fig3.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.3)
fig3.scatter3D(xPredMeshF,yPredMeshF,P2SigmaMeanPred,c='limegreen',label='Predicted Mean + 2 Sigma', alpha=0.3)
fig3.scatter3D(xPredMeshF, yPredMeshF, stdDevMeanPred,c='orange',label='Predicted Std Dev', alpha=0.3)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
```

```
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig3.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: HyperParameters

def likelihood(x):
    tau_l, l_l, noise_l = x
    noiseArray=(np.ones(np.size(xySensor[0,:])))*noise_l
    cov = build_covariance(xySensor, xySensor, tau_l, l_l, sqexp2d)
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))
    likelihoodOut = 0.5 * np.dot((outSensor.T-MEAN), np.dot(invcov, (outSensor-MEAN)))\
          + 0.5 * np.log(np.linalg.det(cov + np.diag(noiseArray)))\
          + 0.5 * np.size(xySensor[1,:]) * np.log(2*np.pi)

    return likelihoodOut

# HyperParameters Results

noise = 1e-8
tau = 0.3
l = 0.2

xInit=np.array([tau, l, noise])
bounds = [(1e-15,None),(1e-15,None),(1e-15,None)]
minOut = minimize(likelihood, xInit, bounds=bounds, method='nelder-mead')
minOutX = np.array(minOut.x)
tau = minOutX[0]
l = minOutX[1]
noise = minOutX[2]
meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)
fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label="Predicted Mean")
fig4.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label="Predicted Cov")
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig4.set_zlabel("Sensor Output")
plt.show()

plt.figure(num='Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.4, s=40)
fig4.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label='Predicted Var', alpha=0.4, s=40)
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig3.set_zlabel("Sensor Output")
plt.legend()
plt.show()

print(minOutX)

# Hyperparameter ± 2 Sigma Pred Computation
```

```
stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

varPred = np.array(np.diag(covPred))

# Hyperparameter STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig5 = plt.axes(projection='3d')
#fig5.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig5.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='viridis',label="Predicted Points")
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig5.set_zlabel("Sensor Output")
plt.show()

# Function : Finding indices of the peaks of a matrix

def findPeaks(inputMatrix):
    paddedMatrix = np.pad(inputMatrix,(1),'constant',constant_values=0)
    length1      = len(inputMatrix[0])
    length2      = len(inputMatrix[1])
    peakInd      = np.zeros((length1,length2))
    for xi in range(0,length1):
        for yi in range(0,length2):
            if paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+2 ]:
                peakInd[xi,yi] = 1

    return peakInd

# PredMatrix Peak Indices

meanPredMatrix       = meanPred.reshape(discretization,discretization)
meanPredPeakIndCoeff = findPeaks(meanPredMatrix)
meanPredPeak         = meanPredMatrix[meanPredPeakIndCoeff!=0]
print(meanPredPeak)

meanPredPeakIndX = xPredMesh * meanPredPeakIndCoeff
meanPredPeakIndX = meanPredPeakIndX[meanPredPeakIndX!=0]

meanPredPeakIndY = yPredMesh * meanPredPeakIndCoeff
meanPredPeakIndY = meanPredPeakIndY[meanPredPeakIndY!=0]

meanPredPeakInd  = np.array([meanPredPeakIndX,meanPredPeakIndY])
print(meanPredPeakInd)

fig6= plt.figure('PredMatrix Indices',figsize=(4,4))
plt.plot(meanPredPeakIndX,meanPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
```

```
# PredMatrix + 2 Sigma Peak Indices

meanPredP2Sigma = P2SigmaMeanPred
meanPredP2SigmaMatrix       = meanPredP2Sigma.reshape(discretization,discretization)
meanPredP2SigmaPeakIndCoeff = findPeaks(meanPredP2SigmaMatrix)
meanPredP2SigmaPeak         = meanPredP2SigmaMatrix[meanPredP2SigmaPeakIndCoeff!=0]
print(meanPredP2SigmaPeak)

meanPredP2SigmaPeakIndX = xPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndX = meanPredP2SigmaPeakIndX[meanPredP2SigmaPeakIndX!=0]

meanPredP2SigmaPeakIndY = yPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndY = meanPredP2SigmaPeakIndY[meanPredP2SigmaPeakIndY!=0]

meanPredP2SigmaPeakInd  = np.array([meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY])
print(meanPredP2SigmaPeakInd)

fig7= plt.figure('PredMatrix + 2 Sigma Peak Indices',figsize=(4,4))
plt.plot(meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# VarMatrix Peak Indices

#varPred = varPred
varPredMatrix       = varPred.reshape(discretization,discretization)
varPredPeakIndCoeff = findPeaks(varPredMatrix)
varPredPeak         = varPredMatrix[varPredPeakIndCoeff!=0]
print(varPredPeak)

varPredPeakIndX = xPredMesh * varPredPeakIndCoeff
varPredPeakIndX = varPredPeakIndX[varPredPeakIndX!=0]

varPredPeakIndY = yPredMesh * varPredPeakIndCoeff
varPredPeakIndY = varPredPeakIndY[varPredPeakIndY!=0]

varPredPeakInd  = np.array([varPredPeakIndX,varPredPeakIndY])
print(varPredPeakInd)


fig8= plt.figure('VarMatrix Peak Indices',figsize=(4,4))
plt.plot(varPredPeakIndX,varPredPeakIndY,'o',c='b')
plt.ylabel("Y Coordinate")
plt.xlabel("X Coordinate")

plt.show()
```

Appendix 2:

```
#### APPENDIX 2 #####

%matplotlib inline
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import math
from scipy.optimize import minimize
```

```
# Input Values

xySensor  = np.array([[ 0.1,-0.9,0.2, 0.8,-0.6, 0.3,  0.5,-0.5,-0.01,-0.9,
                        5.100000000000000089e-01,  7.500000000000000000e-01,  -7.500000000000000000e-01,
1.799999999999999933e-01],
               [0.05, 0.3,0.4,-0.3, 0.3,-0.2,-0.84,0.85,-0.76,-0.9,
                      -8.399999999999999689e-01, -6.999999999999999556e-01,  -6.999999999999999556e-01,
-1.000000000000000000e+00]])
#xSensor   = xySensor[0,:]
#ySensor   = xySensor[1,:]
#xySensor  = np.transpose(xySensor)
outSensor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
= np.array([3.39382006,3.2073034,3.39965035,3.68810201,2.96941623,2.99495501,3.94274928,2.7968011,3.34929
734,3.91296165,
                        3.960708425707508251e+00, 3.915714809492213622e+00, 3.573178696741371230e+00,
3.806254007495223135e+00])

discretization = 50
xPred =  np.linspace(-1, 1, discretization)
yPred =  np.linspace(-1, 1, discretization)
xPredMesh, yPredMesh = np.meshgrid(xPred,yPred)
xyPred = (np.array([xPredMesh.flatten(),yPredMesh.flatten()]))


MEAN = np.mean(outSensor)
noise = 1e-8
tau = 0.3
l = 0.2

plt.figure(num='Data Points',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='teal',label="Data Points",s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: Kernel

def sqexp(x, xp, tauin, lin):
    """Squared exponential kernel (1 dimensional)

    Inputs
    ------
    x : (N), array of multiple inputs
    xp: float

    Returns
    -------
    cov (N,) -- Covariance between each input at *x* and the function values at *x*
    """
    cov = tauin**2 * np.exp(-1/2 * (x - xp)**2 / lin**2)
    return cov

def sqexp2d(x, xp, tauin, lin):
    """Squared exponential kernel (2 dimensional) """
    xX   = x[0,:]
    xY   = x[1,:]
    xpX  = xp[0]
    xpY  = xp[1]
```

```
    tauX = tauin
    tauY = 1 # tauY=1 bc I only want one tau to tune
    sqexpX = sqexp(xX, xpX, tauX, lin)
    sqexpY = sqexp(xY, xpY, tauY, lin)
    cov = sqexpX*sqexpY
    return cov

# Function: Covariance

def build_covariance(x, xp,tauin, lin, kern):
    """Build a covariance matrix

    Inputs
    -------
    x: (N) array of inputs
    xp: (M) array of inptus
    kern: a function mapping inputs to covariance

    Outputs
    -------
    cov: (N, M) covariance matrix
    """
    out = np.zeros((x.shape[1], xp.shape[1]))
    for jj in range(xp.shape[1]):
        out[:, jj] = kern(x, xp[:,jj], tauin, lin)
    return out

# Function: GPR

def gpr(xtrain, ytrain, xpred, noise_var, mean_func, kernel):
    """Gaussian process regression Algorithm

    Inputs
    -------
    xtrain: (N, ) training inputs
    ytrain: (N, ) training outputs
    xpred:  (M, ) locations at which to make predictions
    noise_var: (N, ) noise at every training output
    mean_func: function to compute the prior mean
    kernel: covariance kernel

    Returns
    -------
    pred_mean : (M, ) predicted mean at prediction points
    pred_cov : (M, M) predicted covariance at the prediction points
    --
    """
    #xtrain  = np.transpose(xtrain)
    cov = build_covariance(xtrain, xtrain, tau, l, kernel)
    u, s, v = np.linalg.svd(cov)
    sqrtcov = np.dot(u, np.sqrt(np.diag(s)))

    noiseArray=(np.ones(np.size(xtrain[1,:])))*noise_var

    # pseudoinverse is better conditioned
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))

    vec_pred = build_covariance(xpred, xtrain, tau, l, kernel)

    pred_mean = MEAN + np.dot(vec_pred, np.dot(invcov, ytrain - MEAN))
    cov_predict_pre = build_covariance(xpred, xpred, tau, l, kernel)
    cov_predict_up = np.dot(vec_pred, np.dot(invcov, vec_pred.T))
    pred_cov = cov_predict_pre - cov_predict_up
    #print(cov_predict_pre)
```

```
    return pred_mean, pred_cov

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.2)
#fig1.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label='Predicted Var', alpha=0.2)
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.show()

# Function: ± 2 Sigma Pred Computation

def twoSigma(inputMean, inputCov):
    varPred    = np.array(np.diag(inputCov))
    stdDevPred = np.sqrt(varPred)
    P2SigmaPred = inputMean + (2*stdDevPred)
    N2SigmaPred = inputMean - (2*stdDevPred)

    plt.figure(num='Pred Mean  ± 2 Sigma Pred Computation',figsize=(10,8))
    ax = plt.axes(projection='3d')
       ax.scatter3D(xPredMeshF,yPredMeshF,P2SigmaPred,c='limegreen',label='Predicted  Mean  +  2  Sigma',  alpha=0.4,
s=40)
    ax.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.4, s=40)
    ax.scatter3D(xPredMeshF,yPredMeshF,N2SigmaPred,c='maroon',label='Predicted Mean - 2 Sigma', alpha=0.4, s=40)
    #ax.plot_trisurf(xPredMeshF, yPredMeshF, stdDevPred,cmap='jet',label="Predicted Points")
    #ax.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
    locs, labels = plt.xticks()  # Get the current locations and labels.
    plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    ax.set_zlabel("Sensor Output")
    plt.legend()
    plt.show(ax)
```

```
    return(varPred,stdDevPred,P2SigmaPred,N2SigmaPred)


# ± 2 Sigma Pred Computation
varMeanPred,stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

# STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig3.scatter3D(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
fig3.scatter3D(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig3.scatter3D(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='magma',label="Predicted Points")
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

# Function: HyperParameters

def likelihood(x):
    tau_l, l_l, noise_l = x
    noiseArray=(np.ones(np.size(xySensor[0,:])))*noise_l
    cov = build_covariance(xySensor, xySensor, tau_l, l_l, sqexp2d)
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))
    likelihoodOut = 0.5 * np.dot((outSensor.T-MEAN), np.dot(invcov, (outSensor-MEAN)))\
          + 0.5 * np.log(np.linalg.det(cov + np.diag(noiseArray)))\
          + 0.5 * np.size(xySensor[1,:]) * np.log(2*np.pi)

    return likelihoodOut

# HyperParameters Results

noise = 1e-8
tau = 0.3
l = 0.2

xInit=np.array([tau, l, noise])
bounds = [(1e-15,None),(1e-15,None),(1e-15,None)]
minOut = minimize(likelihood, xInit, bounds=bounds, method='nelder-mead')
minOutX = np.array(minOut.x)
tau = minOutX[0]
l = minOutX[1]
noise = minOutX[2]
meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label="Predicted Mean")
fig4.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label="Predicted Cov")
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
#plt.legend()
plt.show()
```

```
plt.figure(num='Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=0.4, s=40)
fig4.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label='Predicted Var', alpha=0.4, s=40)
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Data Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig3.set_zlabel("Sensor Output")
plt.legend()
plt.show()

print(minOutX)

# Hyperparameter ± 2 Sigma Pred Computation
varPred, stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

# Hyperparameter STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig5 = plt.axes(projection='3d')
fig5.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig5.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='viridis',label="Predicted Points")
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
#plt.legend()
plt.show()

# Function : Finding indices of the peaks of a matrix

def findPeaks(inputMatrix):
    paddedMatrix = np.pad(inputMatrix,(1),'constant',constant_values=0)
    length1      = len(inputMatrix[0])
    length2      = len(inputMatrix[1])
    peakInd      = np.zeros((length1,length2))
    for xi in range(0,length1):
        for yi in range(0,length2):
            if paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+2 ]:

                peakInd[xi,yi] = 1

    return peakInd

# PredMatrix Peak Indices

meanPredMatrix       = meanPred.reshape(discretization,discretization)
meanPredPeakIndCoeff = findPeaks(meanPredMatrix)
meanPredPeak         = meanPredMatrix[meanPredPeakIndCoeff!=0]
```

```
print(meanPredPeak)

meanPredPeakIndX = xPredMesh * meanPredPeakIndCoeff
meanPredPeakIndX = meanPredPeakIndX[meanPredPeakIndX!=0]

meanPredPeakIndY = yPredMesh * meanPredPeakIndCoeff
meanPredPeakIndY = meanPredPeakIndY[meanPredPeakIndY!=0]

meanPredPeakInd  = np.array([meanPredPeakIndX,meanPredPeakIndY])
print(meanPredPeakInd)

fig6= plt.figure('PredMatrix Indices',figsize=(4,4))
plt.plot(meanPredPeakIndX,meanPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# PredMatrix + 2 Sigma Peak Indices

meanPredP2Sigma = P2SigmaMeanPred
meanPredP2SigmaMatrix       = meanPredP2Sigma.reshape(discretization,discretization)
meanPredP2SigmaPeakIndCoeff = findPeaks(meanPredP2SigmaMatrix)
meanPredP2SigmaPeak         = meanPredP2SigmaMatrix[meanPredP2SigmaPeakIndCoeff!=0]
print(meanPredP2SigmaPeak)

meanPredP2SigmaPeakIndX = xPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndX = meanPredP2SigmaPeakIndX[meanPredP2SigmaPeakIndX!=0]

meanPredP2SigmaPeakIndY = yPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndY = meanPredP2SigmaPeakIndY[meanPredP2SigmaPeakIndY!=0]

meanPredP2SigmaPeakInd  = np.array([meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY])
print(meanPredP2SigmaPeakInd)

fig7= plt.figure('PredMatrix + 2 Sigma Peak Indices',figsize=(4,4))
plt.plot(meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# VarMatrix Peak Indices

varPred = varPred
varPredMatrix       = varPred.reshape(discretization,discretization)
varPredPeakIndCoeff = findPeaks(varPredMatrix)
varPredPeak         = varPredMatrix[varPredPeakIndCoeff!=0]
print(varPredPeak)

varPredPeakIndX = xPredMesh * varPredPeakIndCoeff
varPredPeakIndX = varPredPeakIndX[varPredPeakIndX!=0]

varPredPeakIndY = yPredMesh * varPredPeakIndCoeff
varPredPeakIndY = varPredPeakIndY[varPredPeakIndY!=0]

varPredPeakInd  = np.array([varPredPeakIndX,varPredPeakIndY])
print(varPredPeakInd)


fig8= plt.figure('VarMatrix Peak Indices',figsize=(4,4))
plt.plot(varPredPeakIndX,varPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
```

Appendix 3:

```
#### APPENDIX 3 #####

%matplotlib inline
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
#from matplotlib import cm
import math
from scipy.optimize import minimize

# Input Values

xySensor  = np.array([[ 0.1,-0.9,0.2, 0.8,-0.6, 0.3,  0.5,-0.5,-0.01,-0.9,
                        5.100000000000000089e-01,  7.500000000000000000e-01,  -7.500000000000000000e-01,
1.799999999999999933e-01,
                         3.333333299999999833e-01, 8.787878799999999657e-01, 2.323232299999999917e-01,
-3.939393900000000004e-01],
               [0.05, 0.3,0.4,-0.3, 0.3,-0.2,-0.84,0.85,-0.76,-0.9,
                      -8.399999999999999689e-01, -6.999999999999999556e-01,  -6.999999999999999556e-01,
-1.000000000000000000e+00,
                       -7.777777799999999742e-01, -5.959596000000000338e-01, -5.151515199999999739e-01,
-2.525252499999999789e-01]])
outSensor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
=
np.array([3.39382006,3.2073034,3.39965035,3.68810201,2.96941623,2.99495501,3.94274928,2.7968011,3.34929
734,3.91296165,
                         3.960708425707508251e+00, 3.915714809492213622e+00, 3.573178696741371230e+00,
3.806254007495223135e+00,
                         3.606670456681302372e+00, 3.718398992818654936e+00, 4.362107148258433043e+00,
3.453585989860598726e+00])

discretization = 100
xPred =  np.linspace(-1, 1, discretization)
yPred =  np.linspace(-1, 1, discretization)
xPredMesh, yPredMesh = np.meshgrid(xPred,yPred)
xyPred = (np.array([xPredMesh.flatten(),yPredMesh.flatten()]))


MEAN = np.mean(outSensor)
noise = 1e-8
tau = 0.3
l = 0.2

plt.figure(num='Data Points',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='teal',label="Data Points",s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: Kernel
```

```
def sqexp(x, xp, tauin, lin):
    """Squared exponential kernel (1 dimensional)

    Inputs
    ------
    x : (N), array of multiple inputs
    xp: float

    Returns
    -------
    cov (N,) -- Covariance between each input at *x* and the function values at *x*
    """
    cov = tauin**2 * np.exp(-1/2 * (x - xp)**2 / lin**2)
    return cov

def sqexp2d(x, xp, tauin, lin):
    """Squared exponential kernel (2 dimensional) """
    xX   = x[0,:]
    xY   = x[1,:]
    xpX  = xp[0]
    xpY  = xp[1]
    tauX = tauin
    tauY = 1 # tauY=1 bc I only want one tau to tune
    sqexpX = sqexp(xX, xpX, tauX, lin)
    sqexpY = sqexp(xY, xpY, tauY, lin)
    cov = sqexpX*sqexpY
    return cov

# Function: Covariance

def build_covariance(x, xp,tauin, lin, kern):
    """Build a covariance matrix

    Inputs
    -------
    x: (N) array of inputs
    xp: (M) array of inptus
    kern: a function mapping inputs to covariance

    Outputs
    -------
    cov: (N, M) covariance matrix
    """
    out = np.zeros((x.shape[1], xp.shape[1]))
    for jj in range(xp.shape[1]):
        out[:, jj] = kern(x, xp[:,jj], tauin, lin)
    return out

# Function: GPR

def gpr(xtrain, ytrain, xpred, noise_var, mean_func, kernel):
    """Gaussian process regression Algorithm

    Inputs
    -------
    xtrain: (N, ) training inputs
    ytrain: (N, ) training outputs
    xpred:  (M, ) locations at which to make predictions
    noise_var: (N, ) noise at every training output
    mean_func: function to compute the prior mean
    kernel: covariance kernel

    Returns
    -------
```

```
    pred_mean : (M, ) predicted mean at prediction points
    pred_cov : (M, M) predicted covariance at the prediction points
    --
    """
    #xtrain  = np.transpose(xtrain)
    cov = build_covariance(xtrain, xtrain, tau, l, kernel)
    u, s, v = np.linalg.svd(cov)
    sqrtcov = np.dot(u, np.sqrt(np.diag(s)))

    noiseArray=(np.ones(np.size(xtrain[1,:])))*noise_var

    # pseudoinverse is better conditioned
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))

    vec_pred = build_covariance(xpred, xtrain, tau, l, kernel)

    pred_mean = MEAN + np.dot(vec_pred, np.dot(invcov, ytrain - MEAN))
    cov_predict_pre = build_covariance(xpred, xpred, tau, l, kernel)
    cov_predict_up = np.dot(vec_pred, np.dot(invcov, vec_pred.T))
    pred_cov = cov_predict_pre - cov_predict_up
    #print(cov_predict_pre)

    return pred_mean, pred_cov

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig1.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# GPR Results

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.show()

# Function: ± 2 Sigma Pred Computation

def twoSigma(inputMean, inputCov):
    varPred    = np.array(np.diag(inputCov))
    stdDevPred = np.sqrt(varPred)
```

```
    P2SigmaPred = inputMean + (2*stdDevPred)
    N2SigmaPred = inputMean - (2*stdDevPred)

    plt.figure(num='Pred Mean  ± 2 Sigma Pred Computation',figsize=(10,8))
    ax = plt.axes(projection='3d')
    ax.scatter3D(xPredMeshF,yPredMeshF,P2SigmaPred,c='limegreen',label='Predicted Mean + 2 Sigma', alpha=.3)
    ax.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=.3)
    ax.scatter3D(xPredMeshF,yPredMeshF,N2SigmaPred,c='maroon',label='Predicted Mean - 2 Sigma', alpha=.3)
    locs, labels = plt.xticks()  # Get the current locations and labels.
    plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    ax.set_zlabel("Sensor Output")
    plt.legend()
    plt.show(ax)

    return(varPred,stdDevPred,P2SigmaPred,N2SigmaPred)


# ± 2 Sigma Pred Computation
varMeanPred,stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

# STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean', alpha=1)
fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma', alpha=1)
fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma', alpha=1)
fig3.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='magma',label='Predicted Points', alpha=1)
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# Function: HyperParameters

def likelihood(x):
    tau_l, l_l, noise_l = x
    noiseArray=(np.ones(np.size(xySensor[0,:])))*noise_l
    cov = build_covariance(xySensor, xySensor, tau_l, l_l, sqexp2d)
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))
    likelihoodOut = 0.5 * np.dot((outSensor.T-MEAN), np.dot(invcov, (outSensor-MEAN)))\
                + 0.5 * np.log(np.linalg.det(cov + np.diag(noiseArray)))\
                + 0.5 * np.size(xySensor[1,:]) * np.log(2*np.pi)

    return likelihoodOut

# HyperParameters Results

noise = 1e-8
tau = 0.3
l = 0.2

xInit=np.array([tau, l, noise])
bounds = [(1e-15,None),(1e-15,None),(1e-15,None)]
minOut = minimize(likelihood, xInit, bounds=bounds, method='nelder-mead')
minOutX = np.array(minOut.x)
tau = minOutX[0]
l = minOutX[1]
```

```
noise = minOutX[2]
meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)
fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label="Predicted Mean", alpha=0.2)
#fig4.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label="Predicted Var", alpha=0.2)
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100, alpha=0.2)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

print(minOutX)

# HyperParameters Results

fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label="Predicted Mean")
#fig4.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label="Predicted Cov")
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
#plt.legend()
plt.show()

# Hyperparameter ± 2 Sigma Pred Computation
varPred, stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)


# Hyperparameter STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig5 = plt.axes(projection='3d')
fig5.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig5.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='viridis',label='Std Dev')
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# Hyperparameter STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig5 = plt.axes(projection='3d')
fig5.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig5.scatter3D(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='magma',label='Std Dev')
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
```

```
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

# Function : Finding indices of the peaks of a matrix

def findPeaks(inputMatrix):
    paddedMatrix = np.pad(inputMatrix,(1),'constant',constant_values=0)
    length1      = len(inputMatrix[0])
    length2      = len(inputMatrix[1])
    peakInd      = np.zeros((length1,length2))
    for xi in range(0,length1):
        for yi in range(0,length2):
            if paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+2 ]:

                peakInd[xi,yi] = 1

    return peakInd

# PredMatrix Peak Indices

meanPredMatrix       = meanPred.reshape(discretization,discretization)
meanPredPeakIndCoeff = findPeaks(meanPredMatrix)
meanPredPeak         = meanPredMatrix[meanPredPeakIndCoeff!=0]
print(meanPredPeak)

meanPredPeakIndX = xPredMesh * meanPredPeakIndCoeff
meanPredPeakIndX = meanPredPeakIndX[meanPredPeakIndX!=0]

meanPredPeakIndY = yPredMesh * meanPredPeakIndCoeff
meanPredPeakIndY = meanPredPeakIndY[meanPredPeakIndY!=0]

meanPredPeakInd  = np.array([meanPredPeakIndX,meanPredPeakIndY])
print(meanPredPeakInd)

fig6= plt.figure('PredMatrix Indices',figsize=(4,4))
plt.plot(meanPredPeakIndX,meanPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

meanPredMaxInd = meanPredMatrix*meanPredPeakIndCoeff
MeanPredMax = meanPredMatrix[meanPredMaxInd!=0]
print(meanPredMatrix[meanPredPeakIndCoeff!=0])

print(meanPredPeakIndCoeff)

# PredMatrix + 2 Sigma Peak Indices

meanPredP2Sigma = P2SigmaMeanPred
meanPredP2SigmaMatrix       = meanPredP2Sigma.reshape(discretization,discretization)
meanPredP2SigmaPeakIndCoeff = findPeaks(meanPredP2SigmaMatrix)
meanPredP2SigmaPeak         = meanPredP2SigmaMatrix[meanPredP2SigmaPeakIndCoeff!=0]
print(meanPredP2SigmaPeak)

meanPredP2SigmaPeakIndX = xPredMesh * meanPredP2SigmaPeakIndCoeff
```

```
meanPredP2SigmaPeakIndX = meanPredP2SigmaPeakIndX[meanPredP2SigmaPeakIndX!=0]

meanPredP2SigmaPeakIndY = yPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndY = meanPredP2SigmaPeakIndY[meanPredP2SigmaPeakIndY!=0]

meanPredP2SigmaPeakInd  = np.array([meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY])
print(meanPredP2SigmaPeakInd)

fig7= plt.figure('PredMatrix + 2 Sigma Peak Indices',figsize=(4,4))
plt.plot(meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# VarMatrix Peak Indices

#varPred = varPred
varPredMatrix       = varPred.reshape(discretization,discretization)
varPredPeakIndCoeff = findPeaks(varPredMatrix)
varPredPeak         = varPredMatrix[varPredPeakIndCoeff!=0]
print(varPredPeak)

varPredPeakIndX = xPredMesh * varPredPeakIndCoeff
varPredPeakIndX = varPredPeakIndX[varPredPeakIndX!=0]

varPredPeakIndY = yPredMesh * varPredPeakIndCoeff
varPredPeakIndY = varPredPeakIndY[varPredPeakIndY!=0]

varPredPeakInd  = np.array([varPredPeakIndX,varPredPeakIndY])
print(varPredPeakInd)

fig8= plt.figure('VarMatrix Peak Indices',figsize=(4,4))
plt.plot(varPredPeakIndX,varPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
```

Appendix 4:

```
#### APPENDIX 4 #####

%matplotlib inline
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import math
from scipy.optimize import minimize

# Input Values

xySensor  = np.array([[ 0.1,-0.9,0.2, 0.8,-0.6, 0.3,  0.5,-0.5,-0.01,-0.9,
                        5.100000000000000089e-01,  7.500000000000000000e-01,  -7.500000000000000000e-01,
1.799999999999999933e-01,
                        3.333333299999999833e-01,  8.787878799999999657e-01,   2.323232299999999917e-01,
-3.939393900000000004e-01,
```

```
                    6.199999999999999956e-01,  3.499999999999999778e-01,  -1.499999999999999944e-01,
-5.000000000000000278e-02],
              [0.05, 0.3,0.4,-0.3, 0.3,-0.2,-0.84,0.85,-0.76,-0.9,
                   -8.399999999999999689e-01, -6.999999999999999556e-01,  -6.999999999999999556e-01,
-1.000000000000000000e+00,
                  -7.777777799999999742e-01, -5.959596000000000338e-01,  -5.151515199999999739e-01,
-2.525252499999999789e-01,
                   -8.000000000000000444e-01, -5.000000000000000000e-01,   5.000000000000000000e-01,
-4.500000000000000111e-01]])

outSensor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
= np.array([3.39382006,3.2073034,3.39965035,3.68810201,2.96941623,2.99495501,3.94274928,2.7968011,3.34929
734,3.91296165,
                    3.960708425707508251e+00, 3.915714809492213622e+00, 3.573178696741371230e+00,
3.806254007495223135e+00,
                    3.606670456681302372e+00, 3.718398992818654936e+00, 4.362107148258433043e+00,
3.453585989860598726e+00,
                    4.075788006170817823e+00, 4.188583218050431434e+00, 4.824642660411089246e+00,
4.055704943752908243e+00])

discretization = 100
xPred =  np.linspace(-1, 1, discretization)
yPred =  np.linspace(-1, 1, discretization)
xPredMesh, yPredMesh = np.meshgrid(xPred,yPred)
xyPred = (np.array([xPredMesh.flatten(),yPredMesh.flatten()]))


MEAN = np.mean(outSensor)
noise = 1e-8
tau = 0.3
l = 0.2

plt.figure(num='Data Points',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='teal',label="Data Points",s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# Function: Kernel

def sqexp(x, xp, tauin, lin):
    """Squared exponential kernel (1 dimensional)

    Inputs
    ------
    x : (N), array of multiple inputs
    xp: float

    Returns
    -------
    cov (N,) -- Covariance between each input at *x* and the function values at *x*
    """
    cov = tauin**2 * np.exp(-1/2 * (x - xp)**2 / lin**2)
    return cov

def sqexp2d(x, xp, tauin, lin):
    """Squared exponential kernel (2 dimensional) """
    xX   = x[0,:]
```

```
    xY   = x[1,:]
    xpX  = xp[0]
    xpY  = xp[1]
    tauX = tauin
    tauY = 1 # tauY=1 bc I only want one tau to tune
    sqexpX = sqexp(xX, xpX, tauX, lin)
    sqexpY = sqexp(xY, xpY, tauY, lin)
    cov = sqexpX*sqexpY
    return cov

# Function: Covariance

def build_covariance(x, xp,tauin, lin, kern):
    """Build a covariance matrix

    Inputs
    -------
    x: (N) array of inputs
    xp: (M) array of inptus
    kern: a function mapping inputs to covariance

    Outputs
    -------
    cov: (N, M) covariance matrix
    """
    out = np.zeros((x.shape[1], xp.shape[1]))
    for jj in range(xp.shape[1]):
        out[:, jj] = kern(x, xp[:,jj], tauin, lin)
    return out

# Function: GPR

def gpr(xtrain, ytrain, xpred, noise_var, mean_func, kernel):
    """Gaussian process regression Algorithm

    Inputs
    -------
    xtrain: (N, ) training inputs
    ytrain: (N, ) training outputs
    xpred:  (M, ) locations at which to make predictions
    noise_var: (N, ) noise at every training output
    mean_func: function to compute the prior mean
    kernel: covariance kernel

    Returns
    -------
    pred_mean : (M, ) predicted mean at prediction points
    pred_cov : (M, M) predicted covariance at the prediction points
    --
    """
    #xtrain  = np.transpose(xtrain)
    cov = build_covariance(xtrain, xtrain, tau, l, kernel)
    u, s, v = np.linalg.svd(cov)
    sqrtcov = np.dot(u, np.sqrt(np.diag(s)))

    noiseArray=(np.ones(np.size(xtrain[1,:])))*noise_var

    # pseudoinverse is better conditioned
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))

    vec_pred = build_covariance(xpred, xtrain, tau, l, kernel)

    pred_mean = MEAN + np.dot(vec_pred, np.dot(invcov, ytrain - MEAN))
    cov_predict_pre = build_covariance(xpred, xpred, tau, l, kernel)
```

```
    cov_predict_up = np.dot(vec_pred, np.dot(invcov, vec_pred.T))
    pred_cov = cov_predict_pre - cov_predict_up
    #print(cov_predict_pre)

    return pred_mean, pred_cov

# GPR Results

meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig1.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.legend()
plt.show()

# GPR Results

xPredMeshF, yPredMeshF = xPredMesh.flatten(),yPredMesh.flatten()
plt.figure(num='mean and cov',figsize=(10,8))
fig1 = plt.axes(projection='3d')
fig1.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig1.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label='Predicted Cov')
fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
fig1.set_zlabel("Sensor Output")
plt.show()

# Function: ± 2 Sigma Pred Computation

def twoSigma(inputMean, inputCov):
    varPred    = np.array(np.diag(inputCov))
    stdDevPred = np.sqrt(varPred)
    P2SigmaPred = inputMean + (2*stdDevPred)
    N2SigmaPred = inputMean - (2*stdDevPred)
    plt.figure(num='Pred Mean  ± 2 Sigma Pred Computation',figsize=(10,8))
    ax = plt.axes(projection='3d')
    ax.scatter3D(xPredMeshF,yPredMeshF,P2SigmaPred,c='limegreen',label='Predicted Mean + 2 Sigma', alpha=.3)
    ax.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label='Predicted Mean', alpha=.3)
    ax.scatter3D(xPredMeshF,yPredMeshF,N2SigmaPred,c='maroon',label='Predicted Mean - 2 Sigma', alpha=.3)
    locs, labels = plt.xticks()  # Get the current locations and labels.
    plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
    plt.xlabel("X Coordinate")
    plt.ylabel("Y Coordinate")
    ax.set_zlabel("Sensor Output")
    plt.legend()
    plt.show(ax)

    return(varPred,stdDevPred,P2SigmaPred,N2SigmaPred)
```

```
# ± 2 Sigma Pred Computation
varMeanPred,stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)


# STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
fig3.scatter3D(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
fig3.scatter3D(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig3.scatter3D(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='magma',label="Predicted Points")
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

# Function: HyperParameters

def likelihood(x):
    tau_l, l_l, noise_l = x
    noiseArray=(np.ones(np.size(xySensor[0,:])))*noise_l
    cov = build_covariance(xySensor, xySensor, tau_l, l_l, sqexp2d)
    invcov = np.linalg.pinv(cov + np.diag(noiseArray))
    likelihoodOut = 0.5 * np.dot((outSensor.T-MEAN), np.dot(invcov, (outSensor-MEAN)))\
          + 0.5 * np.log(np.linalg.det(cov + np.diag(noiseArray)))\
          + 0.5 * np.size(xySensor[1,:]) * np.log(2*np.pi)
    return likelihoodOut

# HyperParameters Results

noise = 1e-8
tau = 0.3
l = 0.2

xInit=np.array([tau, l, noise])
bounds = [(1e-15,None),(1e-15,None),(1e-15,None)]
minOut = minimize(likelihood, xInit, bounds=bounds, method='nelder-mead')
minOutX = np.array(minOut.x)
tau = minOutX[0]
l = minOutX[1]
noise = minOutX[2]
meanPred, covPred = gpr(xySensor, outSensor, xyPred, noise, MEAN, sqexp2d)

fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.scatter3D(xPredMeshF,yPredMeshF,meanPred,c='teal',label="Predicted Mean", alpha=0.2)
fig4.scatter3D(xPredMeshF,yPredMeshF,np.diag(covPred),c='palevioletred',label="Predicted Var", alpha=0.2)
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

print(minOutX)

# HyperParameters Results
```

```
fig = plt.figure('Hyperparameter Pred Mean',figsize=(10,8))
fig4 = plt.axes(projection='3d')
fig4.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label="Predicted Mean")
fig4.plot_trisurf(xPredMeshF,yPredMeshF,np.diag(covPred),cmap='winter',label="Predicted Cov")
fig4.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# Hyperparameter ± 2 Sigma Pred Computation
varPred, stdDevMeanPred, P2SigmaMeanPred, N2SigmaMeanPred = twoSigma(meanPred, covPred)

# STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig3 = plt.axes(projection='3d')
fig3.plot_trisurf(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean', alpha=1)
fig3.plot_trisurf(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='viridis',label='Predicted Points', alpha=1)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# Hyperparameter STD Pred Plot

plt.figure(num='Pred Mean STD',figsize=(10,8))
fig5 = plt.axes(projection='3d')
fig5.scatter3D(xPredMeshF,yPredMeshF,meanPred,cmap='winter',label='Predicted Mean')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,P2SigmaMeanPred,cmap='cool',label='Predicted Mean + 2 Sigma')
#fig3.plot_trisurf(xPredMeshF,yPredMeshF,N2SigmaMeanPred,cmap='cool',label='Predicted Mean - 2 Sigma')
fig5.scatter3D(xPredMeshF, yPredMeshF, stdDevMeanPred,cmap='magma',label='Std Dev')
#fig1.scatter3D(xySensor[0,:], xySensor[1,:], outSensor, c='k',label="Predicted Points", s=100)
locs, labels = plt.xticks()  # Get the current locations and labels.
plt.xticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.yticks(np.arange(-1, 1.25, step=0.25))  # Set label locations.
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

# Function : Finding indices of the peaks of a matrix

def findPeaks(inputMatrix):
    paddedMatrix = np.pad(inputMatrix,(1),'constant',constant_values=0)
    length1      = len(inputMatrix[0])
    length2      = len(inputMatrix[1])
    peakInd      = np.zeros((length1,length2))
    for xi in range(0,length1):
        for yi in range(0,length2):
            if paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[   xi , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+1 , yi+2 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi   ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+1 ] and \
               paddedMatrix[xi+1,yi+1] > paddedMatrix[ xi+2 , yi+2 ]:
                peakInd[xi,yi] = 1
```

```
    return peakInd

# PredMatrix Peak Indices

meanPredMatrix       = meanPred.reshape(discretization,discretization)
meanPredPeakIndCoeff = findPeaks(meanPredMatrix)
meanPredPeak         = meanPredMatrix[meanPredPeakIndCoeff!=0]
print(meanPredPeak)

meanPredPeakIndX = xPredMesh * meanPredPeakIndCoeff
meanPredPeakIndX = meanPredPeakIndX[meanPredPeakIndX!=0]

meanPredPeakIndY = yPredMesh * meanPredPeakIndCoeff
meanPredPeakIndY = meanPredPeakIndY[meanPredPeakIndY!=0]

meanPredPeakInd  = np.array([meanPredPeakIndX,meanPredPeakIndY])
print(meanPredPeakInd)

fig6= plt.figure('PredMatrix Indices',figsize=(4,4))
plt.plot(meanPredPeakIndX,meanPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

meanPredMaxInd = meanPredMatrix*meanPredPeakIndCoeff
MeanPredMax = meanPredMatrix[meanPredMaxInd!=0]
print(meanPredMatrix[meanPredPeakIndCoeff!=0])
print(meanPredPeakIndX)
print(meanPredPeakIndY)

# PredMatrix + 2 Sigma Peak Indices

meanPredP2Sigma = P2SigmaMeanPred
meanPredP2SigmaMatrix       = meanPredP2Sigma.reshape(discretization,discretization)
meanPredP2SigmaPeakIndCoeff = findPeaks(meanPredP2SigmaMatrix)
meanPredP2SigmaPeak         = meanPredP2SigmaMatrix[meanPredP2SigmaPeakIndCoeff!=0]
print(meanPredP2SigmaPeak)

meanPredP2SigmaPeakIndX = xPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndX = meanPredP2SigmaPeakIndX[meanPredP2SigmaPeakIndX!=0]

meanPredP2SigmaPeakIndY = yPredMesh * meanPredP2SigmaPeakIndCoeff
meanPredP2SigmaPeakIndY = meanPredP2SigmaPeakIndY[meanPredP2SigmaPeakIndY!=0]

meanPredP2SigmaPeakInd  = np.array([meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY])
print(meanPredP2SigmaPeakInd)

fig7= plt.figure('PredMatrix + 2 Sigma Peak Indices',figsize=(4,4))
plt.plot(meanPredP2SigmaPeakIndX,meanPredP2SigmaPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# VarMatrix Peak Indices

#varPred = varPred
varPredMatrix       = varPred.reshape(discretization,discretization)
varPredPeakIndCoeff = findPeaks(varPredMatrix)
varPredPeak         = varPredMatrix[varPredPeakIndCoeff!=0]
print(varPredPeak)

varPredPeakIndX = xPredMesh * varPredPeakIndCoeff
varPredPeakIndX = varPredPeakIndX[varPredPeakIndX!=0]
```

```
varPredPeakIndY = yPredMesh * varPredPeakIndCoeff
varPredPeakIndY = varPredPeakIndY[varPredPeakIndY!=0]

varPredPeakInd  = np.array([varPredPeakIndX,varPredPeakIndY])
print(varPredPeakInd)


fig8= plt.figure('VarMatrix Peak Indices',figsize=(4,4))
plt.plot(varPredPeakIndX,varPredPeakIndY,'o',c='b')
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
```

UNIVERSITY OF MICHIGAN

AE567

Statistical Inference, Estimation, and Learning

Project 2:

Part 2: Bayesian Inference for Nonlinear Models

Deniz Dost

UM ID: 82337583

Nov. 9, 2021
Deniz Dost, 82337583

1. Delayed Rejection Adaptive Metropolis

In this section of the project, four algorithms are implemented, Standard Metropolis Hastings (MH), Adaptive Metropolis Hastings (AM), Delayed Rejection Metropolis Hastings (DR), and Delayed Rejection Adaptive Metropolis Hastings (DRAM). These are used on the simple banana shaped distribution with two dimensional posterior $\pi(x_1, x_2)$ as:

$$(x_1, x_2 + (x_1^2 + 1)) \sim N\left(\mu, \Sigma\right) = N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

This banana function is created by

$$logpdf = log\left(\frac{1}{(2\pi)^{\frac{n}{2}}|\Sigma|^{\frac{1}{2}}} exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)\right)$$

Then, the posterior function for this part looks like as shown in Figure 1.

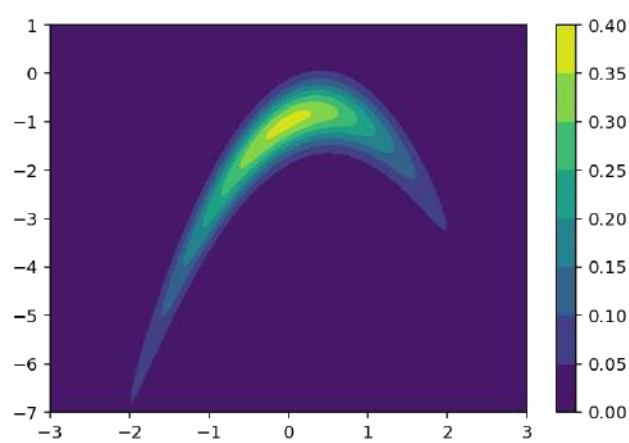

Figure 1: Banana logpdf function

Then, Laplace Approximation is used to optimize the posterior and compute the approximate hessian. So, a MAP point and a covariance matrix is obtained from this posterior to be used in the MCMC algorithms as the initialization point and covariance. These values are obtained as follows:

$$MAP = [-7.67x10^{-6}, -1]$$

$$\Sigma_{Laplace} = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$$

And MAP can be seen on the banana posterior function as shown below in Figure 2.

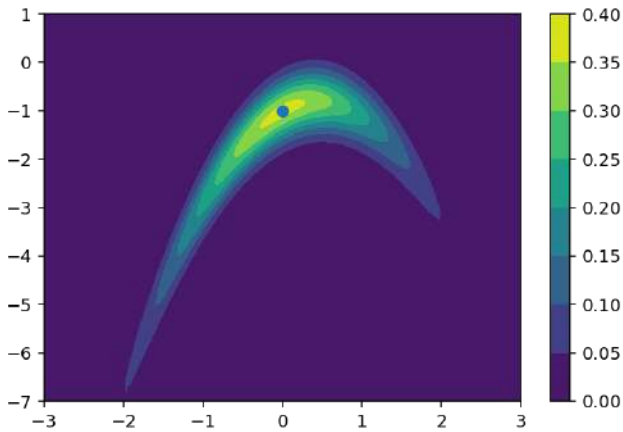

Figure 2: MAP on Banana Logpdf

1.1. Metropolis Hastings MCMC (MH)

MARKOV CHAIN MONTE CARLONUN BES SARTI

In the MH algorithm, first, an initial value $x^{(0)}$ is set to the state. Then, a proposed state is sampled from a distribution $y \sim (q(y|x^{(k)}))$. An acceptance probability for that proposal is computed as shown below.

$$a_1(x,y) = min\left(1, \frac{f_X(y)q_1(x|y)}{f_X(x)q_1(y|x)}\right)$$

Considering the Gaussian distribution, q_1's are equal to each other, so they cancel.

Finally, the proposed state is accepted or rejected according to that probability

$$x^{(k+1)} = \begin{cases} y\ , & w/\ probability \quad a(x^{(k)}, y) \\ x^{(k)}\ , & otherwise \end{cases}$$

The 1D and 2D marginal plots for MH is shown below in Figure 3.

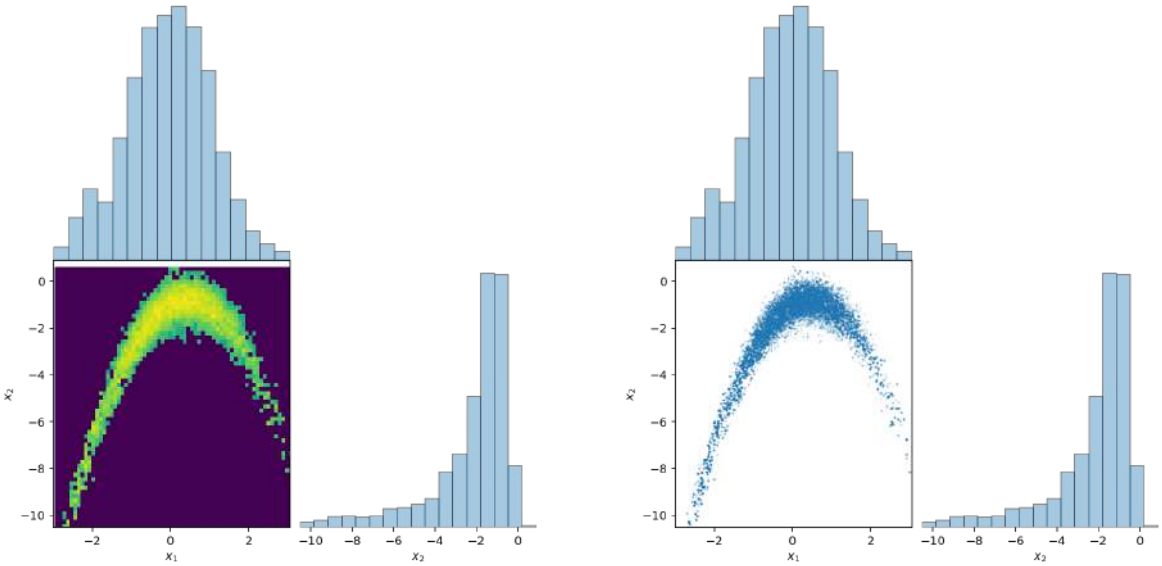

Figure 3: 1D and 2D Marginal Plots for MH

The autocorrelation plot is shown below in Figure 4.

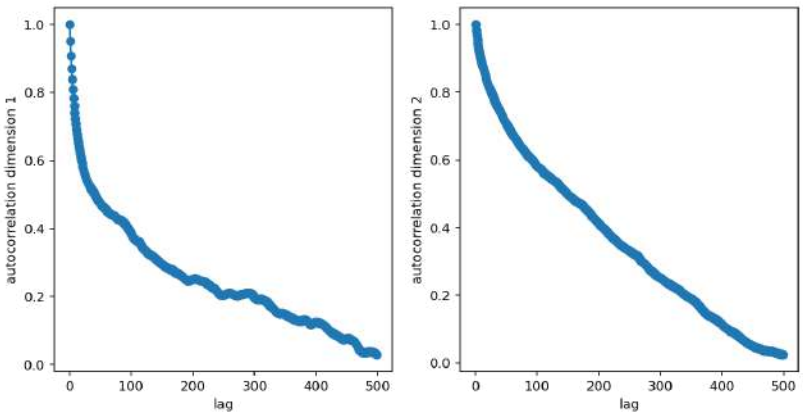

Figure 4: Autocorrelation Plots for MH

The integrated autocorrelation can be calculated from the autocorrelation values as shown below:

$$IAC = 1 + 2\sum_{L=1}^{\infty} corr(f(X_0), f(X_l))$$

The integrated autocorrelation value is 254.85 for the first dimension and 361.88 for the second dimension.

The acceptance ratio is 0.26.

Visual inspection of mixing is shown below in Figure 5.

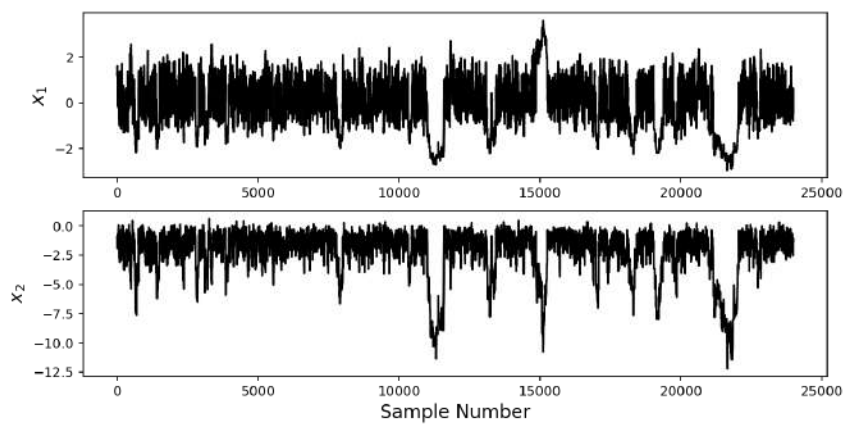

Figure 5: Visual Inspection of Mixing for MH

1.2. Adaptive Metropolis Hastings MCMC (AM)

For the AM, the proposal function is updated from the previous time step knowledge by updating the covariance as governed by the following formula

$$S_{k+1} = \frac{k-1}{k} S_k + \frac{s_d}{k}\left[\xi I + k\,\bar{x}^{k-1}\bar{x^{k-1}}^T - (k+1)\bar{x}^k\bar{x}^{k^T} + x^{(k)}x^{(k)^T}\right], \quad where \quad s_d = \frac{2.4^2}{d}$$

where the empirical mean is calculated as

$$\bar{x} = \frac{1}{k+1}x^{(k)} + \frac{k}{k+1}\bar{x}^{k-1}$$

This is done after a certain threshold value, k_0.

The 1D and 2D marginal plots for AM is shown below in Figure 6.

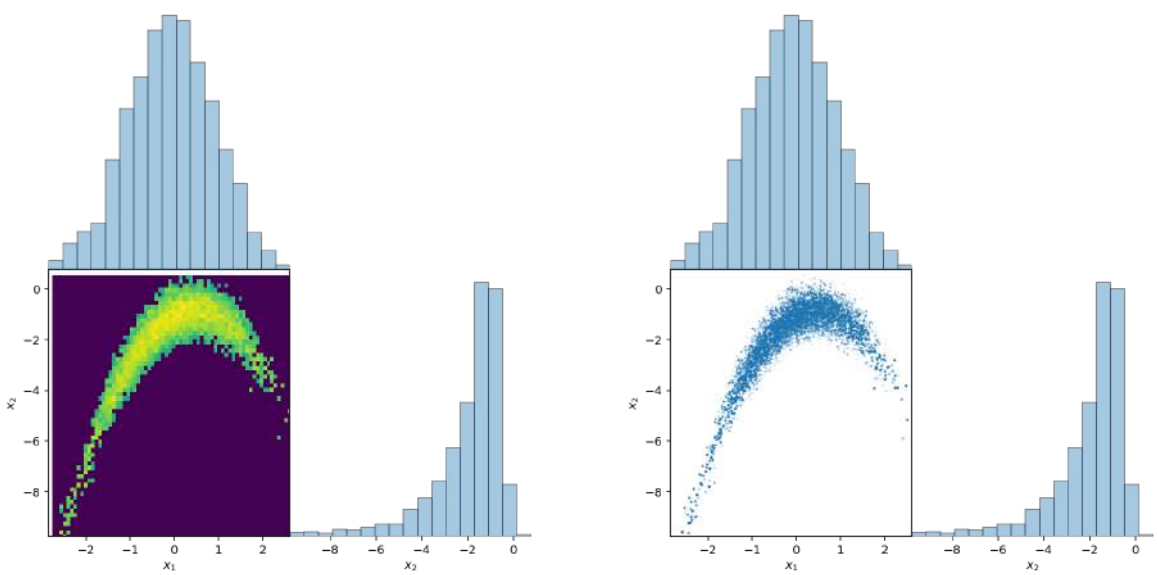

Figure 6: 1D and 2D Marginal Plots for AM

The autocorrelation plot is shown below in Figure 7.

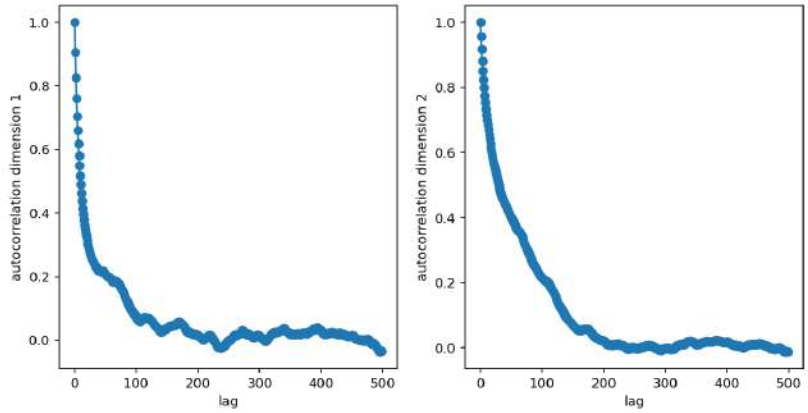

Figure 7: Autocorrelation Plots for AM

The integrated autocorrelation value is 69.02 for the first dimension and 114.36 for the second dimension.

The acceptance ratio is 0.21.

Visual inspection of mixing is shown below in Figure 8.

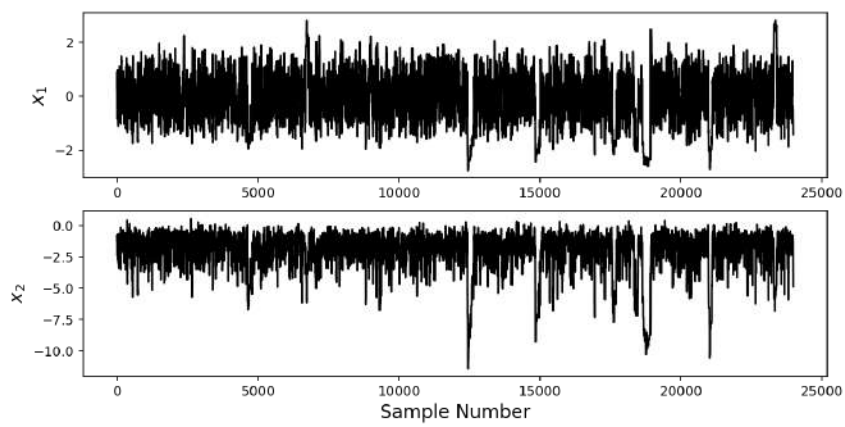

Figure 8: Visual Inspection of Mixing for AM

1.3. Delayed Rejection Metropolis Hastings MCMC (DR)

DR algorithm, in the scope of this project, basically proposes a second stage move instead of staying in the same position in the case of rejection of the first proposal. In addition to the first candidate move y_1 in MH, another proposal $y_2 \sim (q_2(y_2|x, y_1))$ is made with the acceptance probability

$$a_2(x, y_1, y_2) = min\left(1, \frac{f_X(y_2)q_1(y_1|y_2)q_2(x|y_2, y_1)(1 - a_1(y_2, y_1))}{f_X(x)q_1(y_1|x)q_2(y_2|y_1, x)(1 - a_1(x, y_1))}\right)$$

Here, q_2's cancel whereas q_1's don't.

The 1D and 2D marginal plots for DR is shown below in Figure 9.

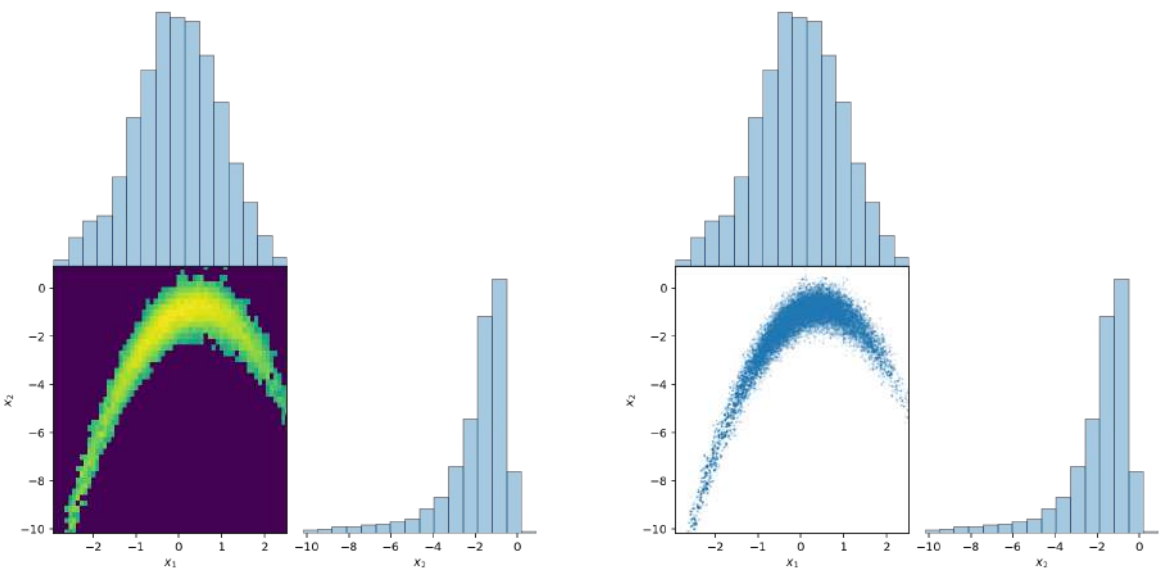

Figure 9: 1D and 2D Marginal Plots for DR

The autocorrelation plot is shown below in Figure 10.

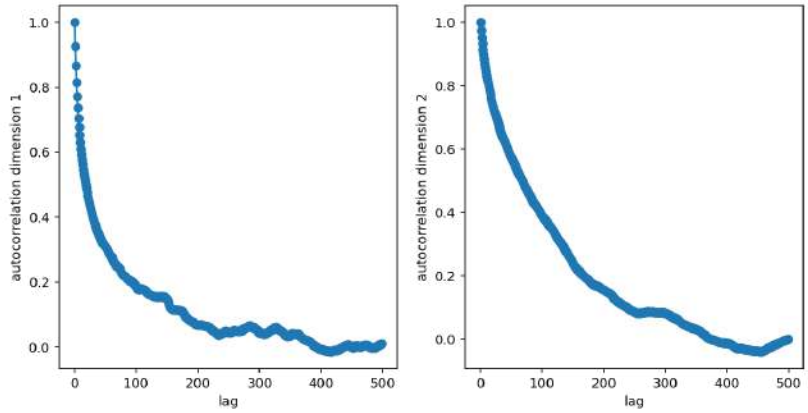

Figure 10: Autocorrelation Plots for DR

The integrated autocorrelation value is 117.95 for the first dimension and 194.39 for the second dimension.

The acceptance ratio is 0.27.

Visual inspection of mixing is shown below in Figure 11.

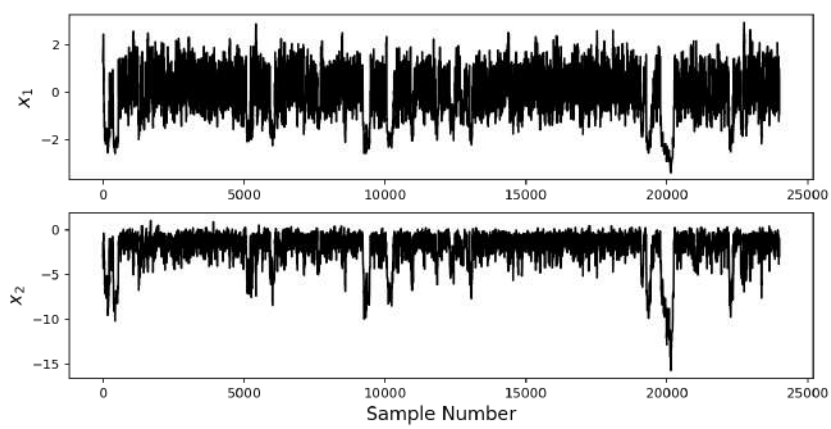

Figure 11: Visual Inspection of Mixing for DR

1.4. Delayed Rejection Adaptive Metropolis Hastings MCMC (DRAM)

In this method, AM and DR methods which are explained above are combined. The 1D and 2D marginal plots for DRAM is shown below in Figure 12.

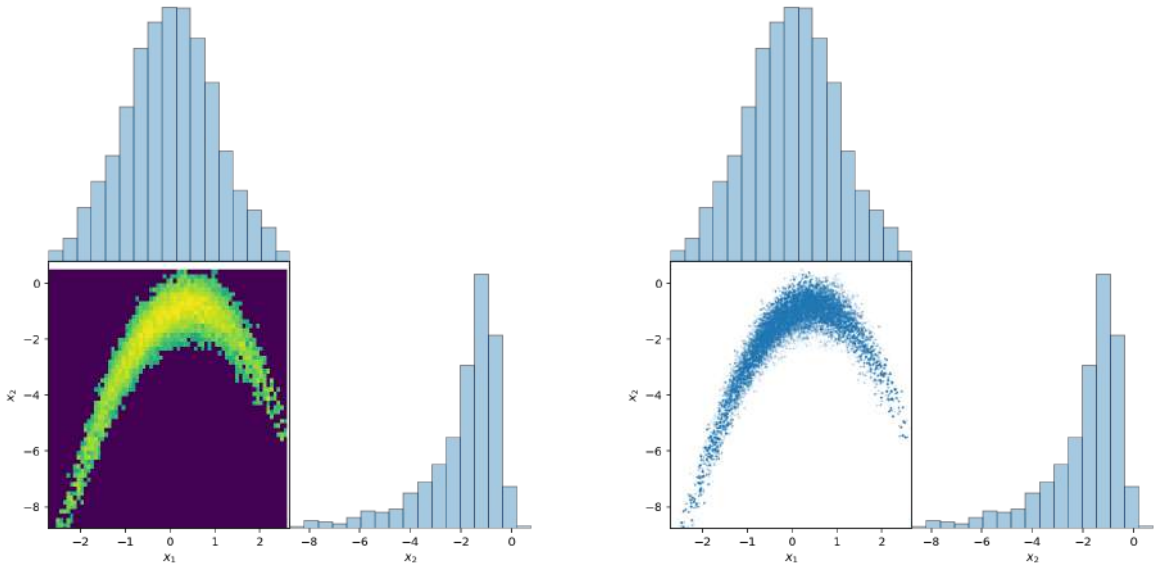

Figure 12: 1D and 2D Marginal Plots for DRAM

The autocorrelation plot is shown below in Figure 13.

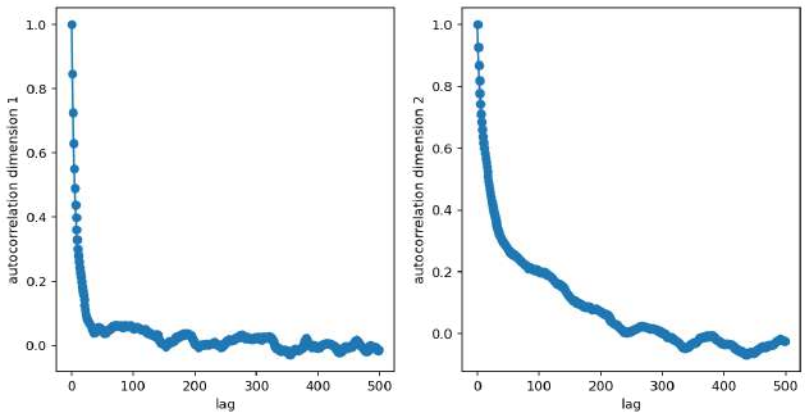

Figure 13: Autocorrelation Plots for DRAM

The integrated autocorrelation value is 34.42 for the first dimension and 88.58 for the second dimension.

The acceptance ratio is 0.32.

Visual inspection of mixing is shown below in Figure 14.

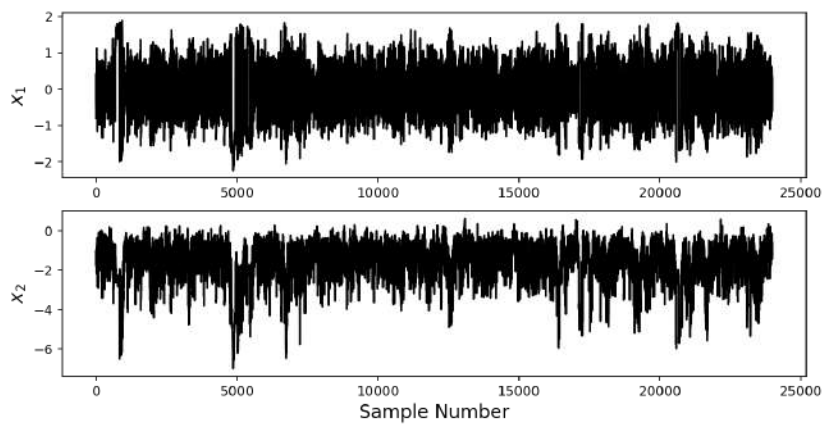

Figure 14: Visual Inspection of Mixing for DRAM

1.5. Comparison

The comparison for these algorithms can be done by examining the visual outputs as well as the acceptance ratios. The acceptance ratios are not immensely different, however DRAM has the highest acceptance ratio.

Table 1: Acceptance Ratios

	MH	AM	DR	DRAM
Acceptance Ratio	0.26	0.21	0.27	0.32

Autocorrelation graphs show the correlation between each sample set evolving in time. So, the faster/rapid this autocorrelation value goes to zero, the better job a sampling algorithm is doing in order to sample with less dependance/ relation on the previous samples. So, faster decay means that the algorithm is better.

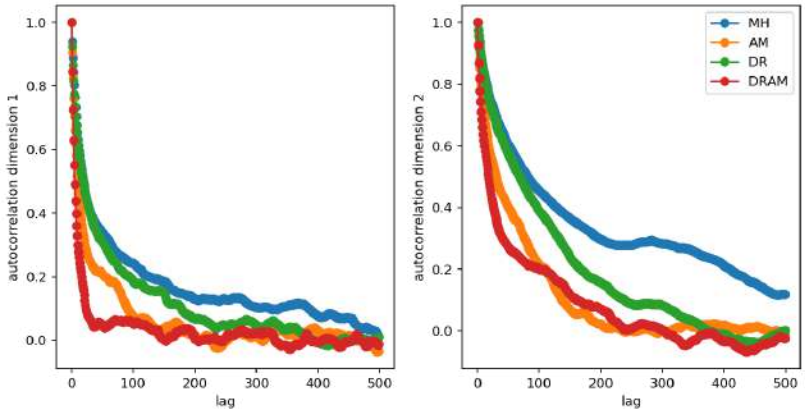

Figure 15: Autocorrelation Plots for the Algorithms

It can be seen from Figure 15 that DRAM has the fastest decay, whereas MH has the slowest decay. This would suggest that they are the best and the worst algorithms respectively.

Integrated autocorrelation (IAC) can also be used to comment on the efficiency of the algorithm. The higher the value is, the worse the algorithm is since it is basically the autocorrelation values summed up. The IAC values shown on Table 2 also supports that DRAM is the best algorithm whereas MH is the worst. DRAM lowered IAC values drastically, and MH had distinctively high IAC values.

Table 2: IAC Values

	MH		AM		DR		DRAM	
	Dim 1	Dim 2	Dim 1	Dim 2	Dim 1	Dim 2	Dim 1	Dim 2
IAC	254.85	361.88	69.02	114.36	117.95	194.39	34.42	88.58

The visual inspections of these algorithms show the samples acquired with each algorithm. So, it can be interpreted that sampling is done better when the samples are more random, varying in a random manner rather than having spikes and trends, like in the white noise. As it can be seen in Figure 14, DRAM has its samples looking the most like white noise, so this also supports that DRAM is a better algorithm.

2. Truth Model

SIR (Susceptible, Infected and Recovered) model of a disease in a population is governed by the differential equations given below for each section. The total population is 1000 and the initial value for each state is as shown below for both of the cases.

$$N = 1000$$

$$S(0) = 900$$

$$I(0) = 100$$

$$R(0) = 0$$

This problem considers the evolution of SIR portions of the population in the timescale:

$$t \in [0,6]$$

$$t = linspace(0,6,61)$$

The differential equations are solved with the Runge-Kutta 45 method for each section to compute the evolution. The state of interest here is I (Infected). Data from that is created by adding noise to it.

$$D = I + \xi \qquad , where \quad \xi \sim N(0,1)$$

2.1. Identifiable Version

SIR (Susceptible, Infected and Recovered) model of a disease in a population in the identifiable case is governed by the differential equations given below.

$$\frac{dS}{dt} = \delta N - \delta S - \beta IS$$

$$\frac{dI}{dt} = \beta IS = (r + \delta)I$$

$$\frac{dR}{dt} = rI - \delta R$$

In the identifiable version of this model, the true values of the parameters

$$\theta = (\beta, r, \delta) = (0.02, 0.6, 0.15)$$

The states are evolving as shown in Figure 15 In this case.

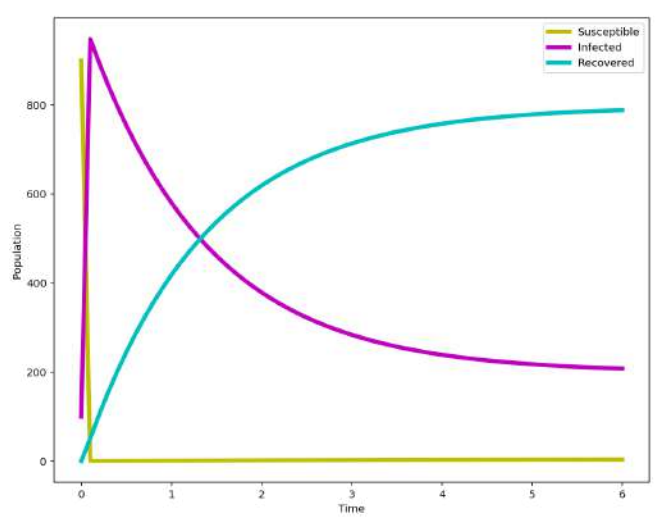

Figure 15

The data created from the I state, I_{data}, computed is as shown below.

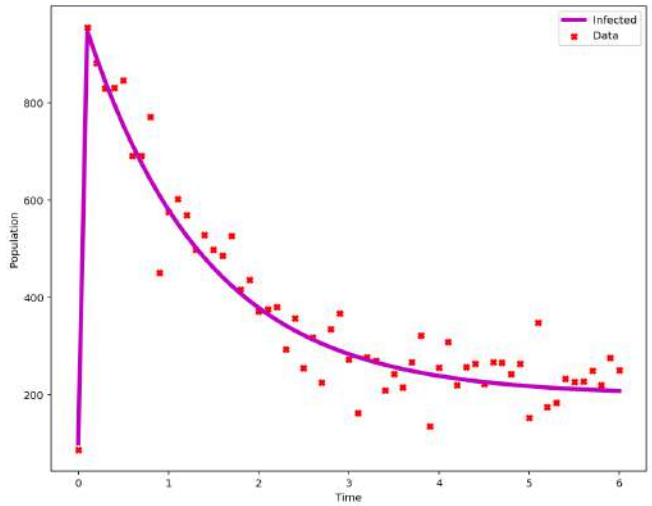

Figure 16

2.2. Non-Identifiable Version

SIR (Susceptible, Infected and Recovered) model of a disease in a population in the identifiable case is governed by the differential equations given below.

$$\frac{dS}{dt} = \delta N - \delta S - \gamma \kappa IS$$

$$\frac{dI}{dt} = \gamma \kappa IS = (r + \delta)I$$

$$\frac{dR}{dt} = rI - \delta R$$

In the non-identifiable version of this model, the true values of the parameters

$$\theta = (\gamma, \kappa, r, \delta) = (0.2, 0.1, 0.6, 0.15)$$

The states are evolving as shown in Figure 17 In this case.

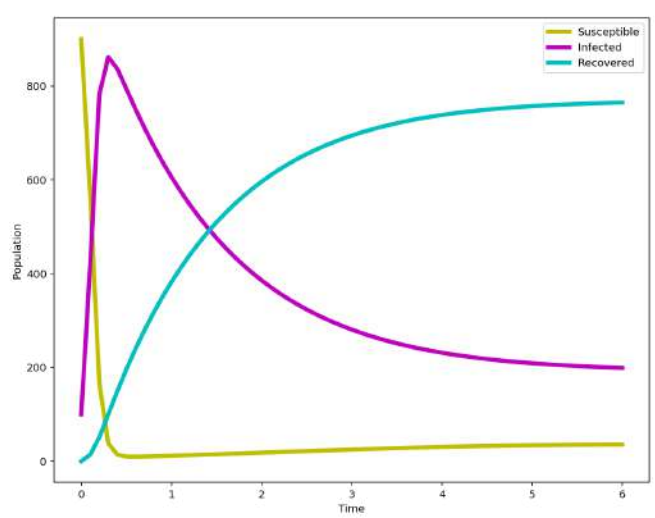

Figure 17

The data created from the I state, I_{data}, computed is as shown below.

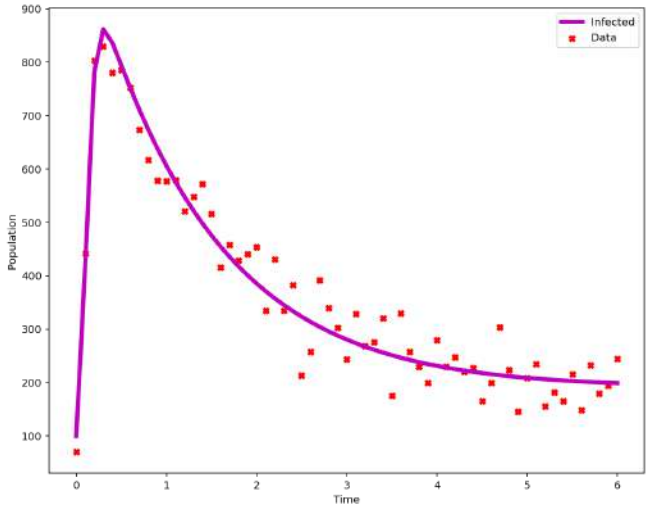
Infected
Data
Population
Time
900
800
700
600
500
400
300
200
100
0
1
2
3
4
5
6

Figure 18

3. DRAM for Bayesian Inference

In this section, the works done above are combined. Data created, $I_{data} = \{d_1, d_2, \ldots, d_i\}$, is used with the DRAM algorithm to learn the SIR model parameters. The prior for the parameters are Gaussian as

$$\theta \sim N(0,1)$$

$$f_\theta(\theta) = \prod_{i=1}^{k} f_\theta(\theta_i),\ where \quad k = 3\ for\ Section\ 3.1\ and\ k = 4\ for\ Section\ 3.2$$

The likelihood function is constructed as a log likelihood. The log of the prior is taken and summed with the likelihood to obtain posterior.

$$d_i \sim N(y_i, std),\ where \quad std = 50$$

$$f_D(D\,|\,\theta) = \prod_{i=1}^{k=61} f_D(d_i\,|\,\theta)$$

$$f_{\theta|D}(\theta\,|\,d) \sim f_D(D\,|\,\theta) f_\theta(\theta)$$

3.1. Identifiable Version

In this section, data created in section 2.1 is used with DRAM algorithm. The algorithm has an acceptance ratio of 0.32.

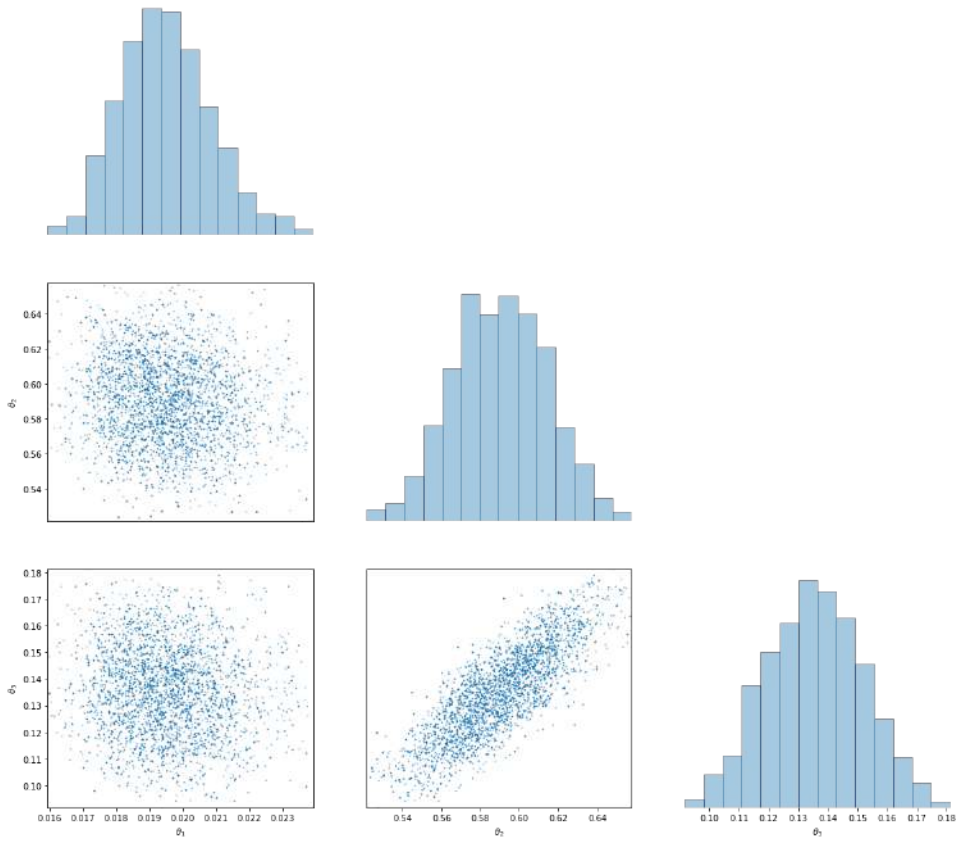

Figure 19: 1D and 2D Marginal Plots for the Identifiable Version

The IAC values are 10.03, 29.63, and 33.62 for the first (β), second (r), and third (δ) dimension respectively. The autocorrelation plots for the can be seen in Figure 20.

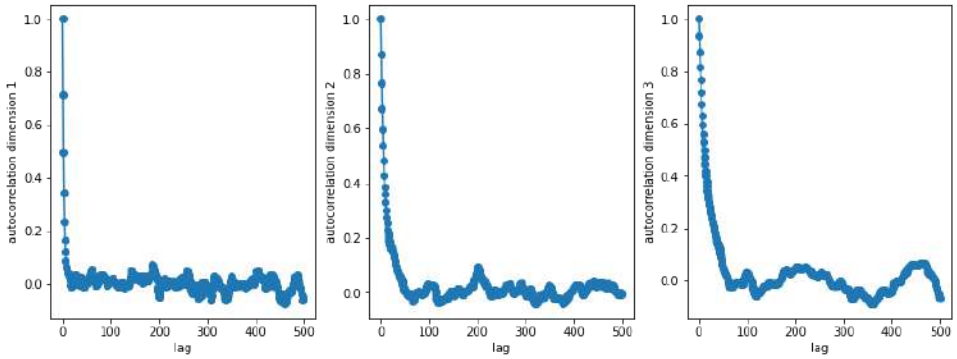

Figure 20: Autocorrelation Plots for the Identifiable Version

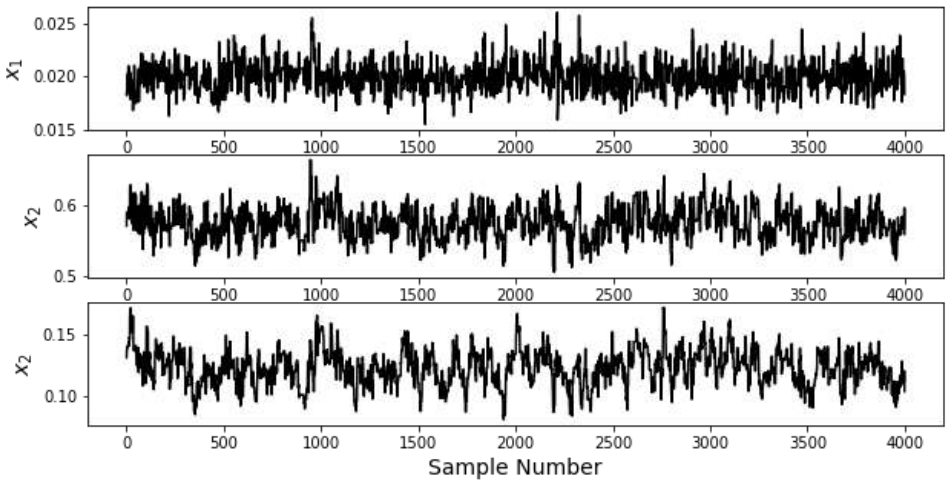

Figure 21: Visual Inspection of Mixing for the Identifiable Version

3.2.Non-Identifiable Version

In this section, data created in section 2.2 is used with DRAM algorithm. The algorithm has an acceptance ratio of 0.04.

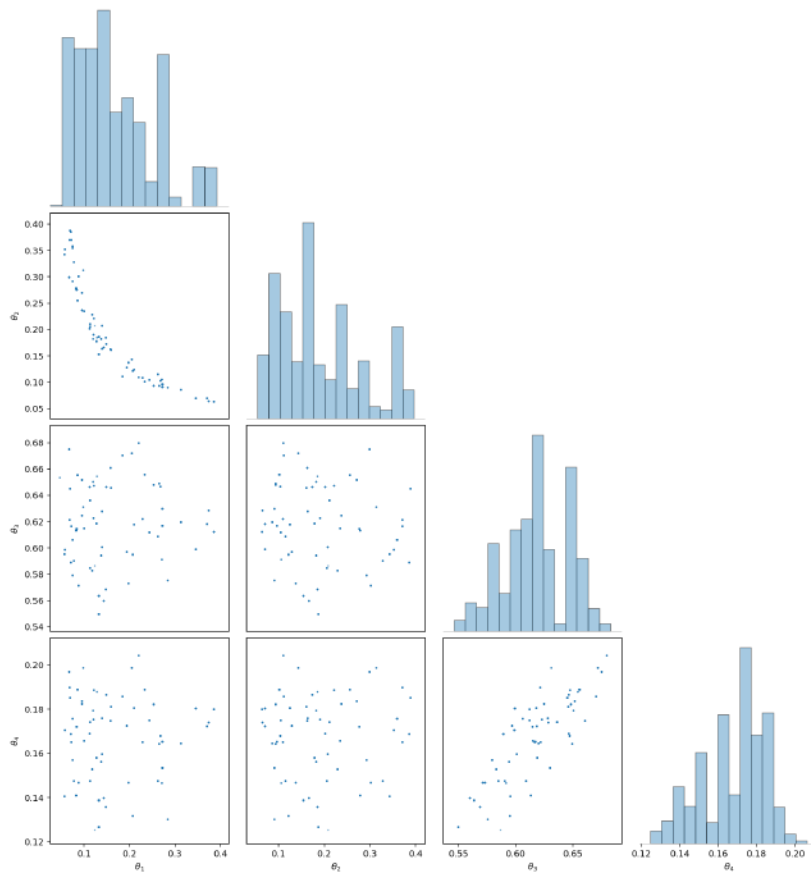

Figure 22: 1D and 2D Marginal Plots for the Non-Identifiable Version

The IAC values are 131.42, 62.97, 491.13 and 221.29 for the first (γ), second(κ), third(r), and fourth (δ) dimension respectively. The autocorrelation plots for the can be seen in Figure 23.

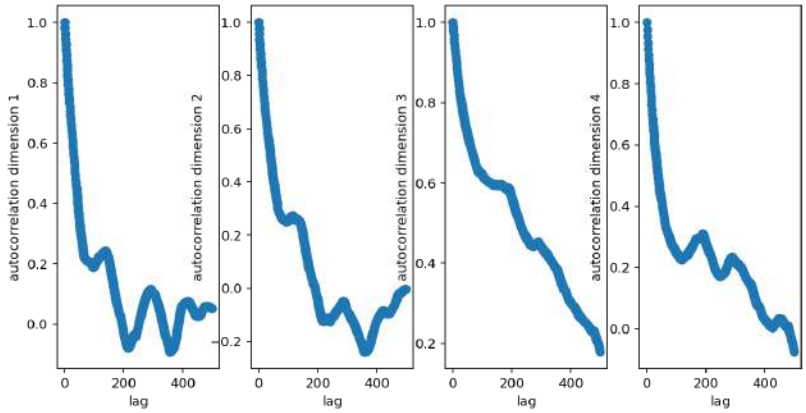

Figure 23: Autocorrelation Plots for the Non-Identifiable Version

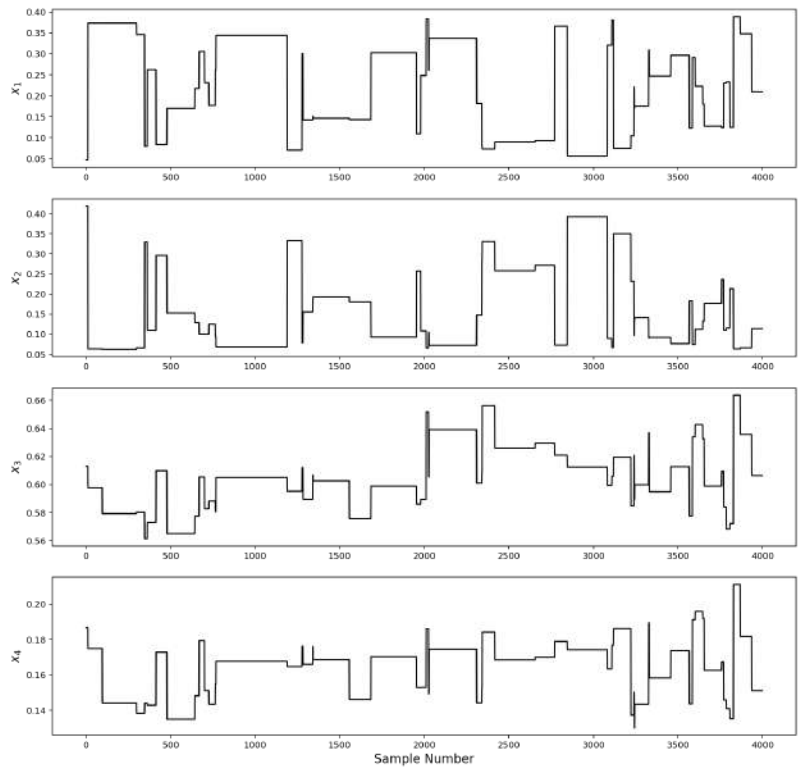

Figure 24: Visual Inspection of Mixing for the Non-Identifiable Version

Non-Identifiable case does not seem to be properly working, so it is hard to comment on it. But from the identifiable case, it could be seen from Figure 19 that the parameters are close to the original values. Also, the autocorrelation graphs in Figure 20 is decreasing rapidly, whereas it looks much more slower in Figure 23. The samples also look like white noise in the identifiable case as shown in Figure 21. Since the parameters are non-identifiable in the second case, these can be changed and the effect of this on the differential equation may not be observable if their multiplication is as desired. Hence, it is harder to infer their values. It can be seen from Figure 22 top left graph between θ_1, (γ), and θ_2, (κ), it is basically like a curve where $\gamma = \frac{\beta}{\kappa}$, since $\gamma\kappa$ is used in the differential equations instead of β, and this curve parameters multiplied fits into to the range its values.

4. Appendices

```
# ==============================
# ==============================
import numpy
import numpy.linalg
from mpmath import *
from matplotlib.pyplot import matplotlib, matplotlib.mpl, matplotlib.mpl_toolkits.mplot3d, pylab, scipy.linalg, scipy.fftpack, scipy.ndimage, scipy.optimize
# ==============================
# ==============================
# ==============================
```

```
%matplotlib inline
import scipy
import numpy as np
import matplotlib.pyplot as plt
import numpy.linalg as linalg, import scipy.linalg
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
from scipy.integrate import
```

```
# Some useful utilities
from mpmath.calc_ext.quad import mpmathify, build_goo_mat, logsumexpf, plot_bivariate_gauss, eval_func_on_grid, sub_sample_data

def compose(*functions):
    '''Compose a list of functions'
    return functools.reduce(lambda f, g: lambda x: f(g(x)), functions, lambda x: x)
```

```
def odeFunc(t,x):
    N = 100
    S = x[0]
    I = x[1]
    R = x[2]
    beta = 0.2
    gamma = 0.1
    delta = 0.15
    # theta = np.array([beta, I, delta]) # true parameter settings
    # print("S=",S,"I=",I,"R=",R,"beta=",beta )
    y1 = beta*S*I - I*delta*I
    y2 = y1 - delta*R                 # dai R
    return y0,y1,y2
```

```
def solveFunc(t):
    timeTmp = np.linspace(0,6,41)
    S_0 = 100
    I_0 = 100
    R_0 = 0
    x_0 = np.array([S_0, I_0, R_0])
    soln = scipy.integrate.odeint(odeFunc,(S,I,R),t_eval=timeTmp,method='RK45')
    return soln
```

```
soln1 = solveFunc(t)
data1 = np.zeros(41)
stdNoise = 50
for i1 in range(0,len(data1)):
    data1[i1]=soln1[i1,1]+stdNoise*np.random.randn()
```

```
plt.figure(num='Indentifiable Version',figsize=(10,8))
plt.plot(soln1[:,0],soln1[:,0],'y',linewidth=4,label='Susceptible')
plt.plot(soln1[:,0],soln1[:,1],'r',linewidth=4,label='Infected')
plt.plot(soln1[:,0],soln1[:,2],'b',linewidth=4,label='Recovered')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()
```

```
plt.figure(num='Data',figsize=(10,8))
plt.plot(soln1[:,0],soln1[:,1],'r',linewidth=4,label='Infected')
plt.scatter(soln1[:,0],data1, s=7, , marker='D',s=13,label='Data')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()
```

```
def odeFunc(t):
    timeTmp = np.linspace(0,6,41)
    S_0 = 900
    I_0 = 100
    R_0 = 0
    x_0 = np.array([S_0, I_0, R_0])
    soln = scipy.integrate.odeint(odeFunc,(S,I,R),t_eval=timeTmp,method='RK45')
    return soln
```

```
soln2 = solveFunc(t)
data2 = np.zeros(41)
stdNoise = 50
for i2 in range (0,len(data2)):
    data2[i2]=soln2[i2,1]+stdNoise*np.random.randn()
```

```
plt.figure(num='Indentifiable Version',figsize=(10,8))
plt.plot(soln2[:,0],soln2[:,0],'y',linewidth=4,label='Susceptible')
plt.plot(soln2[:,0],soln2[:,1],'r',linewidth=4,label='Infected')
plt.plot(soln2[:,0],soln2[:,2],'b',linewidth=4,label='Recovered')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()
```

```
plt.figure(num='Data',figsize=(10,8))
plt.plot(soln2[:,0],soln2[:,1],'r',linewidth=4,label='Infected')
plt.scatter(soln2[:,0],data2, s=7, , marker='D',s=13,label='Data')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()
```

```
beta = 0.2
r = 0.1
delta = 0.15
theta_0 = np.array([beta, r, delta]) # true parameter settings
#theta = np.random.randn(3)
gamma = 0.1
gamma = 0.2
kappa = 0.1
gamma_kappa = np.array([gamma, kappa,r, delta])  # true parameter settings
print(theta_0)
```

```
def log_likelihood(theta=theta,data):
    gamma = theta[0:2:1]
    beta = theta[0]
    r = theta[1]
    delta = theta[2]
    S_0 = 900
    I_0 = 100
    R_0 = 0
    x_0 = np.array([S_0, I_0, R_0])
    soln = scipy.integrate.odeint(odeFunc,(S,I),t_eval=timeTmp)

#     print(np.round(np.sum(soln[:,1],axis=0)))
#     print(np.round(np.sum(soln[:,1],axis=0)*r))
#     return 1.0
    if theta[0]>0 and theta[1]>= 0 and theta[2]>= 0:
        sum = 0
        # pre_mag_term = (1/np.sqrt(2*np.pi)*r)
        # mag_term = (np.exp(-(soln[:,1]-data)**2/(2.*r**2)))
        # print("pre_mag_term",pre_term
        for k in range(1,len(data)):
            if soln[k,1] <= 0:
                f_theta1 = np.log(np.stats.norm.pdf(data[k],soln.y[1,k], sd=r))
            f_theta1 = np.log(np.stats.norm.pdf(theta[0],0,1))
            f_theta2 = np.log(np.stats.norm.pdf(theta[1],0,1))
            f_theta3 = np.log(np.stats.norm.pdf(theta[2],0,1))
            f_theta = np.sum(np.array([f_theta1, f_theta2, f_theta3]))
            sum+=np.sum(f) + f_theta
        else:
            return -np.inf

        return sum
```

```
logsum1 = lambda params: log_likelihood(params, data1)
```

```
def multVarGaussPDF(x,y,cov):
    x_mu = x-y
    logpdf = np.log(((np.sqrt((2*np.pi)**2)*np.linalg.det(cov)))**(-np.exp(-0.5*np.dot((x_mu),T.np.dot(np.linalg.inv(cov),x_mu)))
    return logpdf
```

```
class DelayedMetJsonIndAppliedMetropolis:

    def __init__(self, logpdf, cov, iters=10, freq=5, sd=None, max_samples=10000, eps=1e-7):
        '''The class constructor, parameters are documented below'''
        # the log posterior function, the function we're fitting
        self.logpdf = logpdf # initial covariance
        self.cov_chol = np.linalg.cholesky(cov) # sqrt of the covariance
        self.dim = cov.shape[0] # number of parameters
        self.iters = iters # how to start adaptive
        self.freq = freq # frequency of covariance updates (should be an integer > 0)
        if sd == None:
            self.sd = (2.4**2) / self.dim
        else:
            self.sd = sd # scale for the covariance
        self.max_samples = max_samples # maximum number of samples
        self.eps = eps # small value to add to covariance diagonal
        self.num_samples = 0 # number of samples generated
        self.samples = np.zeros(max_samples, self.dim) # store the samples
        self.logpdf_vals = np.zeros(max_samples)

        self.sample_mean = self.samples[0,:]

    def sample(self, initial_sample, num_samples):
        assert num_samples >= self.max_samples, 'Requesting more samples than space is allocated for'

        self.samples[0, :] = initial_sample
        self.logpdf_vals[0] = self.logpdf(initial_sample)

        accept = 1
        for i in range(1, num_samples):

            # propose
            y = self.samples[i-1:, :] + np.dot(self.cov_chol, np.random.randn(self.dim))
            y_logpdf = self.logpdf(y)

            # compute accept-reject probability, using the fact that we have a symmetric proposal
            a = np.exp(y_logpdf - self.logpdf_vals[i-1])
            u = np.random.rand(1)

            u = np.random.rand(1)

            if a >= 1:
                self.samples[i, :] = y
                self.logpdf_vals[i] = y_logpdf
                accept += 1
            else:
                # propose
                cov_num_samples = i
                choldecl = np.linalg.cholesky(0.5*self.cov) # sqrt of the covariance
                y1 = self.samples[i-1, :] + np.dot(choldecl, np.random.randn(self.dim))
                y_logpdf1 = self.logpdf(y1)

                a1y1 = np.exp(y_logpdf - y_logpdf1)
                a1y1 = np.exp(y_logpdf - self.logpdf_vals[i-1])

                q1 = multVarGaussPDF(y1,self.cov)
                q2 = multVarGaussPDF(y,self.samples[i-1, :],self.cov)

                a2 = np.exp(y_logpdf2+np.log(q1)-q2)*np.log(-(self.logpdf_vals[i-1]+np.log(-a1xy)) + q1 - q2)
                a2 = min(a2,1)

                u2 = np.random.rand(1)

                if u2 <= a2:
                    self.samples[i, :] = y1
                    self.logpdf_vals[i] = y_logpdf2
                    accept += 1
                else:
                    self.samples[i, :] = self.samples[i-1, :]
                    self.logpdf_vals[i] = self.logpdf_vals[i-1]

            cov_num_samples += 1

            # adapt covariance if its time
            if (i % self.freq == 0 and (i > self.iters) <= 0:
                samplesNumber = i[:(i[(i:]]*self.samples[i:, :]+(i[(i:]]*self.sample_mean
                cov0part = (1/(i-1))*(np.dot(self.samples[i:, :].T, self.samples[i:, :])-(np.dot(self.sample_mean[:, ~\
                                    (i[(i:]]*np.dot(self.sample_mean,self.sample_mean.T)
                                    (np.dot(self.samples[i:, :],self.samples[i:, :]))
                self.cov = (i-1)/(i-samplesNumber-1)*self.cov + cov0part
                self.cov_chol = np.linalg.cholesky(self.cov)
                self.sample_mean = samplesNumber
            if (i % 100 == 0):
                print("Finished sample", i,"acceptance ratio = (accept / self.num_samples)")

        return self.samples, accept / float (self.num_samples)

def laplace_approx(initial_guess, logpost):
    '''Perform the laplace approximation,
       returning the MAP point and an approximation of the covariance

    Inputs
    ------
    initial_guess: (nparam, ) array of initial parameters
    logpost: function (param) -> log posterior

    Inputs
    ------
    map_point: (nparam, ) MAP of the posterior
    cov_approx: (nparam, nparam), covariance matrix for Gaussian fit at MAP

    map_point,xx=
        '''Negative posterior because optimizer is a minimizer'''
        return -logpost(x)

    # Gradient-free method to obtain optimum
    xx = scipy.optimize.minimize(neg_post, initial_guess, method='Nelder-Mead')
    # Gaussian method which also approximates the inverse of the hessian
    res = scipy.optimize.minimize(neg_post, res.x)

    map_point = res.x
    cov_approx = res.hess_inv
    return map_point, cov_approx
```

```
num_samples = 10000
dim=3
queue = np.random.randn(dim) # random queue
map_point, cov_laplace = laplace_approx(gamma, logsum1)
initial_sample = map_point
print(map_point)
print(cov_laplace)
print(map_point)
```

```
dram = DelayedMetJsonIndAppliedMetropolis(logsum1, cov=cs, freq=5, iters=20, sd = None, eps = 1e-7, max_samples=num_samples)

samples, ar = dram.sample(initial_sample,num_samples)
print(samples)
```

```
def sub_sample_data(samples, frac_burnin=1.2, frac_use=0.7):
    '''Resample data by burning off the front fraction and using another fraction

    Inputs
    ------
    samples: (N,d) array of samples
    frac_burnin: the number of samples of samples from the front to ignore
    frac_use: percentage of samples to use after burning, uniformly spaced

    nsamples = samples.shape[0]
    nsubsamp = int(nsamples*(1.0-frac_use))
    start = int(frac_burn * nsamples)
    inds = (nsubsamp*1.0)
    nsamples = nsamples - start
    inds = inds*(nsamples - 1)/(nsamples * frac_use)
    inds2 = np.arange(0, nsubsamp)
    inds = inds.int8()
    return samples[inds, :]
```

```
samples = sub_sample_data(samples, frac_burnin=1.2, frac_use=0.7)
fig, axs, gs = scatter_matrix(samples,nbb=3, hist_kwds=dict(bins=50,alpha=1), labels=['$\theta_{0}$', '$\theta_{1}$', '$\theta_{2}$'])
fig.set_size_inches(16,16)
plt.subplots_adjust(left=0.01, right=0.99, wspace=0.2, hspace=0.2)
```

```
def autocorrelation(samples, maxlag=20, step=1):
    '''Compute the correlation of a set of samples

    Inputs
    ------
    samples: (N, d)
    maxlag: maximum distance to compute the correlation for
    step: step between distances from 0 to maxlag for which to compute teh correlations

    # Get the shapes
    nsize = samples.shape[0]
    nsamples = samples.shape[1]

    # Compute the mean
    mean = np.mean(samples, axis=0)

    # Compute the denominator, which is variance
    denominator = np.zeros(nsize)
    for i1 in range(nsize):
        denominator = denominator + (samples[i1, :] - mean)**2

    lags = np.arange(0, maxlag, step)
    acorr = np.zeros(len(lags), nsize)
    for i1, lag in enumerate(lags):
        print("Computing autocorrelation for lag", lag)
        # Compute the correlation between all samples "lag apart"
        for i1 in range(nsize - lag):
            acorr[i1] = acorr[i1] + (samples[i1,:] - mean)*(samples[i1 + lag,:] -mean)
            acorr[i1] = acorr[i1] / denominator
        return lags, acorr
```

```
maxlag=500
step=1
lags, acolag = autocorrelation(samples, maxlag=maxlag,step=step)
# print("shapes",lags.shape,acolag.shape)
sns.set(style='ticks', palette='Set1')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 1')
plt.plot(lags, acolag[:, 1], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 2')
plt.plot(lags, acolag[:, 2], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 3')
plt.show()

lag1 = int(np.sum(acolag[:,0]))
print(lag1)
lag2 = int(np.sum(acolag[:,1]))
print(lag2)
lag3 = int(np.sum(acolag[:,2]))
print(lag3)
```

```
fig, axs = plt.subplots(1, 1, figsize=(8,5))
sns.set_kwds(marker='o', ms=7)
sns.set_kwds('lag')
sns.set_ylabel('$\theta_{0}$', fontsize=14)
sns.set_kwds('lag')
sns.set_xlabel('Lag', fontsize=14)
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 1')
plt.plot(lags, acolag[:, 0], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 2')
plt.show()
```

```
def log_likelihood(theta=theta,data):
    gamma = theta[0:2:1]
    beta = theta[0]
    r = theta[1]
    delta = theta[2]
    S_0 = 900
    I_0 = 100
    R_0 = 0
    x_0 = np.array([S_0, I_0, R_0])
    soln = scipy.integrate.odeint(odeFunc,(S,I),t_eval=timeTmp)

#     print(np.round(np.sum(soln[:,1],axis=0)))
#     print(np.round(np.sum(soln[:,1],axis=0)*r))
#     return -inf
    if theta[0]>0 and theta[1]>= 0 and theta[2]>= 0 and theta[3]>= 0:
        sum = 0
        # pre_mag_term = (1/np.sqrt(2*np.pi)*r)
        # mag_term = (np.exp(-(soln[:,1]-data)**2/(2.*r**2)))
        # print("pre_mag_term",pre_term
        for k in range(1,len(data)):
            if soln[k,1] <= 0:
                f_theta1 = np.log(np.stats.norm.pdf(data[k],soln.y[1,k], sd=r))
            f_theta1 = np.log(np.stats.norm.pdf(theta[0],0,1))
            f_theta2 = np.log(np.stats.norm.pdf(theta[1],0,1))
            f_theta3 = np.log(np.stats.norm.pdf(theta[2],0,1))
            f_theta4 = np.log(np.stats.norm.pdf(theta[3],0,1))
            f_theta = np.sum(np.array([f_theta1, f_theta2, f_theta3, f_theta4]))
            sum+=np.sum(f) + f_theta

        else:
            return -np.inf

        return sum
```

```
logsum2 = lambda params: log_likelihood(params, data2)
```

```
num_samples = 5000
dim=4
queue = np.random.randn(dim) # random queue
map_point, cov_laplace = laplace_approx(gamma, logsum2)
initial_sample = map_point
print(map_point)
print(cov_laplace)
print(map_point)

dram = DelayedMetJsonIndAppliedMetropolis(logsum2, cov=cs, freq=5, iters=20, sd = None, eps = 1e-7, max_samples=num_samples)

samples, ar = dram.sample(theta_0[0,num_samples)
print(samples)

samples = sub_sample_data(samples, frac_burnin=1.2, frac_use=0.7)
fig, axs, gs = scatter_matrix(samples,nbb=3, hist_kwds=dict(bins=50,alpha=1),
fig.set_size_inches(16,16)
plt.subplots_adjust(left=0.01, right=0.99, wspace=0.2, hspace=0.2)
```

```
maxlag=500
step=1
lags, acolag = autocorrelation(samples, maxlag=maxlag,step=step)
# print("shapes",lags.shape,acolag.shape)
sns.set(style='ticks', palette='Set1')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 1')
plt.plot(lags, acolag[:, 1], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 2')
plt.plot(lags, acolag[:, 2], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 3')
plt.plot(lags, acolag[:, 3], '-o')
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 4')
plt.show()

lag1 = int(np.sum(acolag[:,0]))
print(lag1)
lag2 = int(np.sum(acolag[:,1]))
print(lag2)
lag3 = int(np.sum(acolag[:,2]))
print(lag3)
lag4 = int(np.sum(acolag[:,3]))
print(lag4)

fig, axs = plt.subplots(1, 1, figsize=(8,5))
sns.set_kwds(marker='o', ms=7)
sns.set_kwds('lag')
sns.set_ylabel('$\theta_{0}$', fontsize=14)
sns.set_kwds('lag')
sns.set_xlabel('Lag', fontsize=14)
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 1')
plt.plot(lags, acolag[:, 0], '-o')
sns.set_kwds('lag')
sns.set_ylabel('$\theta_{1}$', fontsize=14)
sns.set_kwds('lag')
sns.set_xlabel('Lag', fontsize=14)
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 2')
plt.plot(lags, acolag[:, 1], '-o')
sns.set_kwds('lag')
sns.set_ylabel('$\theta_{2}$', fontsize=14)
sns.set_kwds('lag')
sns.set_xlabel('Lag', fontsize=14)
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 3')
plt.plot(lags, acolag[:, 2], '-o')
sns.set_kwds('lag')
sns.set_ylabel('$\theta_{3}$', fontsize=14)
sns.set_kwds('lag')
sns.set_xlabel('Lag', fontsize=14)
sns.set_kwds('lag')
sns.set_ylabel('autocorrelation dimension 4')
plt.show()
```

UNIVERSITY OF MICHIGAN

AE567

Statistical Inference, Estimation, and Learning

Project 3: Filtering

Deniz Dost

UM ID: 82337583

Dec. 2, 2021
Deniz Dost, 82337583

1. Model

The time-discretized system dynamics for e pendulum with the angle measurements taken at every δ timesteps is governed by the equations below, where the two states are the angle (x_1) and the angular rate (x_2).

$$\Phi(X_{k+1}) = \begin{pmatrix} x_1^{k+1} \\ x_2^{k+1} \end{pmatrix} = \begin{pmatrix} x_1^k + x_2^k \Delta t \\ x_2^k - gsin(x_1^k)\Delta t \end{pmatrix} + q^k$$

Where $q^k \sim \mathcal{N}(0,\mathcal{Q})$

$$\mathcal{Q} = \begin{pmatrix} \frac{q^c \Delta t^3}{3} & \frac{q^c \Delta t^2}{2} \\ \frac{q^c \Delta t^2}{2} & q^c \Delta t \end{pmatrix}, \quad where \quad q_c = 0.1, \quad and \quad \Delta t = 0.01$$

$$h(X_k) = y^{\delta k} = sin(x_1^{\delta k}) + r^{\delta k}$$

Where $r^{\delta k} \sim \mathcal{N}(0,\mathcal{R})$

The dynamics and filtering are considered with the initial condition $(x_1^0, x_2^0) = (1.5,0)$ for 500 time steps. Gaussian filtering to be implemented follows the below equations for prediction

$$m_k^- = \mathbb{E}[X_k \,|\, \mathcal{Y}_{k-1}]$$

$$C_k^- = \mathbb{C}ov[X_k, X_k \,|\, \mathcal{Y}_{k-1}]$$

and update.

$$\mu = \mathbb{E}[Y_k]$$

$$U = \mathbb{C}ov[X_k, Y_k]$$

$$S = \mathbb{C}ov[Y_k, Y_k]$$

$$m_k = m_k^- + US^{-1}(y_k - \mu)$$

$$C_k = C_k^- - US^{-1}U^T$$

1.1. Linearized Extended Kalman Filtering Equations

The linearized system equations are shown below:

$$A_k = \nabla \Phi(X)|_{X_k = m_{k-1}}$$

$$A_k = \begin{bmatrix} 1 & \Delta t \\ -gcos(m_{k-1}^1)\Delta t & 1 \end{bmatrix}$$

$$H_k = \nabla h(X)|_{X_k = m_k^-}$$

$$H_k = \begin{bmatrix} cos(m_k^{1-}) & 0 \end{bmatrix}$$

For the filtering part, the prediction equations are as follows to compute the predicted mean and covariance from the linearized system dynamics:

$$m_k^- \approx \Phi(m_{k-1})$$

$$C_k^- \approx A_k C_{k-1} A_k^T + \Sigma$$

The update equations to compute the updated mean (estimates) and covariance from the predicted states and covariance are as follows:

$$\mu \approx h(m_k^-)$$

$$S \approx H_k C_k^- H_k^T + \Gamma$$

$$U \approx C_k^- H_k^T$$

$$m_k = m_k^- + US^{-1}(y_k - \mu)$$

$$C_k = C_k^- - US^{-1}U^T$$

1.2.Integrals Required for Gaussian Filtering

The integrals for the Gaussian filtering are introduced here with the given equations

Prediction equations computed with the previous state estimate to obtain mean and covariance predictions are

$$m_k^- = \int \Phi(X_{k-1}) \mathcal{N}(X_{k-1}; m_{k-1}, C_{k-1}) dX_{k-1}$$

$$C_k^- = \int (\Phi(X_{k-1}) - m_k^-)(\Phi(X_{k-1}) - m_k^-)^T \mathcal{N}(X_{k-1}; m_{k-1}, C_{k-1}) dX_{k-1} + \Sigma$$

Update equations computed with the predicted mean and covariance to obtain the updated mean (state estimates) and the covariance are as follows

$$\mu = \int h(X_k) \mathcal{N}(X_k; m_k^-, C_k^-) dX_k$$

$$U = \int (X_k - m_k^-)(h(X_k) - \mu)^T \mathcal{N}(X_k; m_k^-, C_k^-) dX_k$$

$$S = \int (h(X_k) - \mu)(h(X_k) - \mu)^T \mathcal{N}(X_k; m_k^-, C_k^-) dX_k + \Gamma$$

$$m_k = m_k^- + US^{-1}(y_k - \mu)$$

$$C_k = C_k^- - US^{-1}U^T$$

2. Gaussian Filtering

With the system introduced above, four types of filtering are done: Extended, Unscented, Gauss-Hermite (of order 3 and 5) filters. Gauss-Hermite is selected as the last type of filter here since it has been worked on in class material and similarities to the unscented, thus its familiarity. The actual initial state is used as the center of the prior, as the prior mean with standard deviation value as one by setting the prior covariance to the identity matrix. A total of 16 scenarios are run with the combinations of measurement timesteps defined as $\delta \in \{5,10,20,40\}$ and noise variance defined as $R \in \{1,0.1,0.01,0.001\}$ for each filtering algorithm. The system dynamics are simulated and data is created as shown in Figure 1.

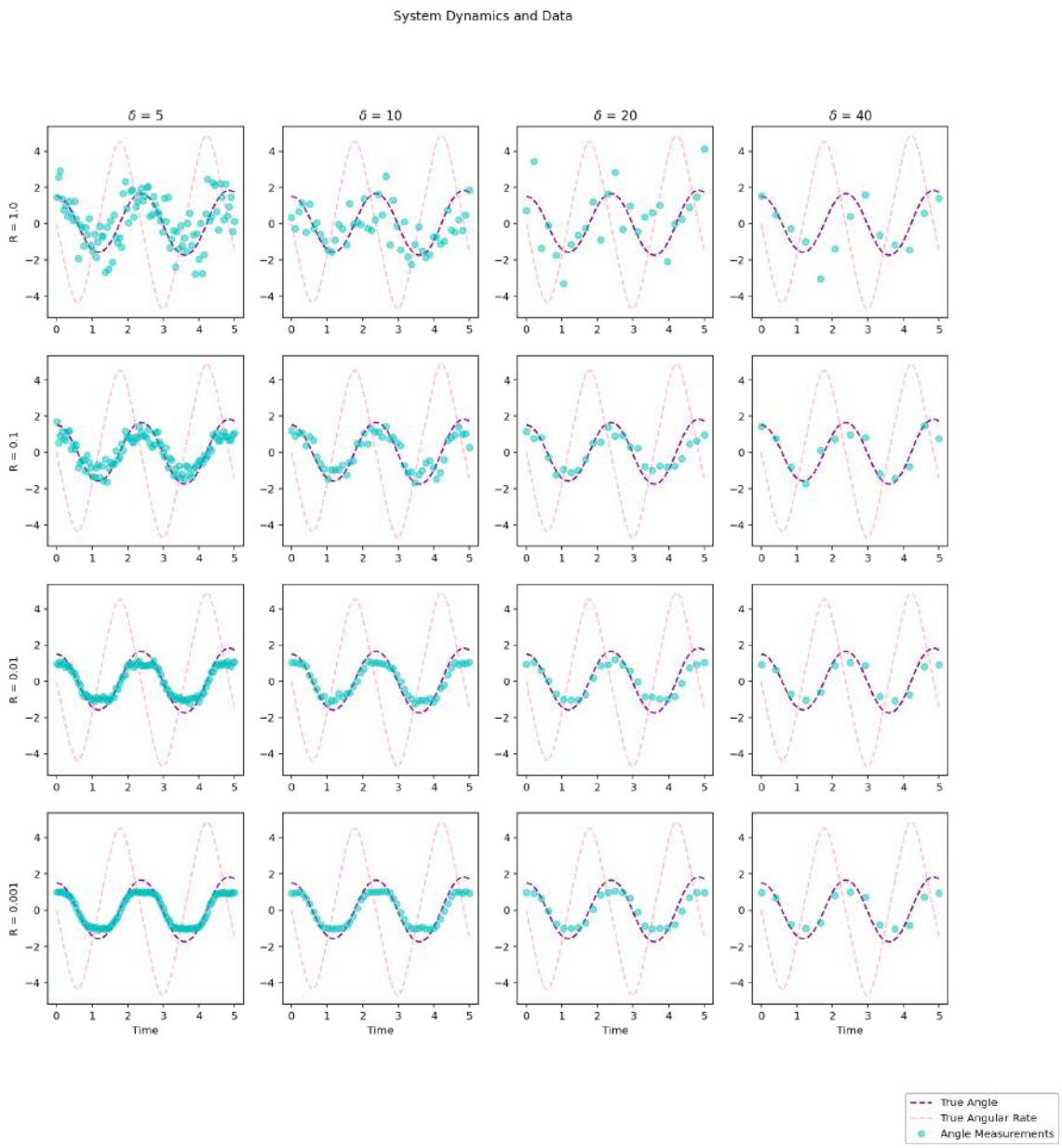
System Dynamics and Data
δ = 5
δ = 10
δ = 20
δ = 40
R = 1.0
R = 0.1
R = 0.01
R = 0.001
Time
True Angle
True Angular Rate
Angle Measurements

Figure 1

2.1.Extended Kalman Filter (EKF)

The EKF is based on integrating the linear integrands by linearizing the system around m_{k-1} for the dynamics and m_k^- for the measurement model with Taylor series approximation by using the Jacobians as shown above. The results with EKF is shown below in Figure 2.

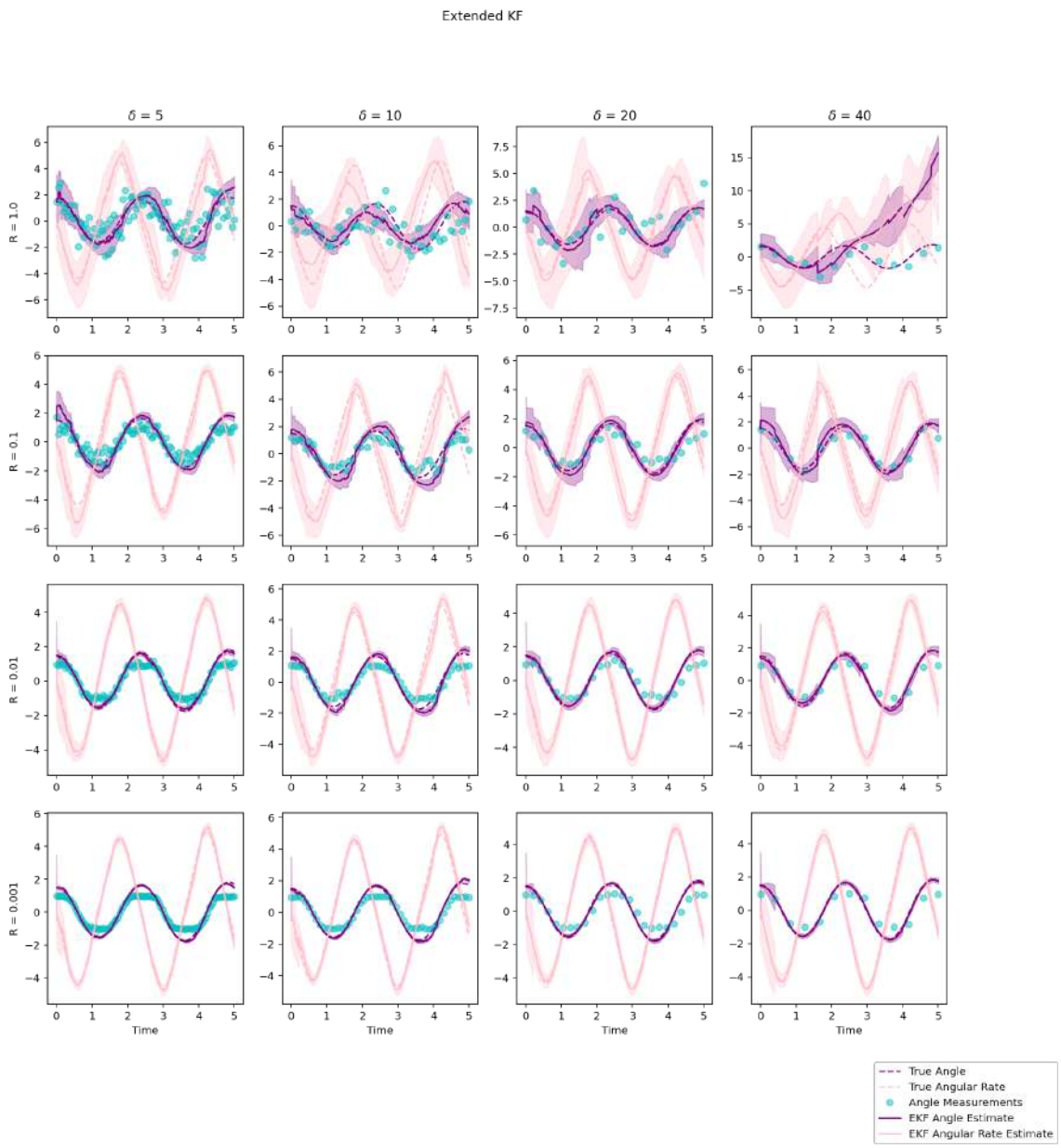

Figure 2

The mean squared error is calculated as shown below and the values for all of the scenarios with EKF are given for both of the states in Table 1.

$$MSE = \sum_{i=1}^{n} = \frac{(\hat{y}_i - y_i)^2}{n}$$

		Mean Squared Error	Measurement Timesteps			
		State	5	10	20	40
Noise Variance	1	1	0.1711	0.4165	0.0958	27.5325
		2	0.7877	2.8797	0.1300	22.8725
	0.1	1	0.0835	0.3000	0.0370	0.1046
		2	0.2272	1.4725	0.1305	0.3963
	0.01	1	0.0064	0.0523	0.0055	0.0123
		2	0.0158	0.2829	0.0205	0.0531
	0.001	1	0.0042	0.0135	0.0039	0.0033
		2	0.0372	0.0937	0.0173	0.0216

Table 1

2.2.Unscented Kalman Filter (UKF)

In order to evaluate the integrals approximately, quadrature rules are used.

$$\int h(X)dX \approx \sum_{i=0}^{2d} h\left(X^{(i)}\right) w^{(i)}$$

Where $w^{(i)}$ is the weight of each ith point, $x^{(i)}$. When integration is considered with respect to weights u in a standard Gaussian measure

$$\int h(X)p(X)dX = \int h(\mu + \Sigma u)\mathcal{N}(u\,|\,0,I)du = \int \tilde{h}(u)\mathcal{N}(u\,|\,0,I)du$$

Where u is transformed to X as

$$X = \mu + \sqrt{\Sigma}u$$

UKF uses $2d+1$ quadrature rule and it uses two types of weights: for mean and covariance. There is the additional point to the elements of the symmetric set $[M]$ at the origin, $u^{(0)}$.

$$[M] = \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots, \begin{pmatrix} -1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots \right\}, \quad and \quad u^{(0)} \begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

The weights for the mean are calculated as

$$w_m^{(i)} = \begin{cases} \frac{\lambda}{d+\lambda}, & i = 0) \\ \frac{1}{2(d+\lambda)}, & i = 1,2,...,2d \end{cases}$$

Whereas the weights for the covariance are calculated as

$$w_c^{(i)} = \begin{cases} \frac{\lambda}{d+\lambda} + (1-\alpha^2+\beta), & i = 0) \\ \frac{1}{2(d+\lambda)}, & i = 1,2,...,2d \end{cases}$$

Where α, β, and κ are free parameters to be chosen from literature and

$$\lambda = \alpha^2(d+\kappa) - d$$

Here are chosen as $\alpha = 0.001$, $\beta = 2$, $\kappa = 0$ from literature.

Then, the integration approximation becomes

$$\int h(X)\mathcal{N}(X\,|\,\mu,\Sigma)dX \approx \sum_{i=0}^{2d} h\left(\mu + \sqrt{\Sigma}\sqrt{d+\lambda}u^{(i)}\right) w^{(i)}$$

The results with the UKF are shown below in Figure 3.

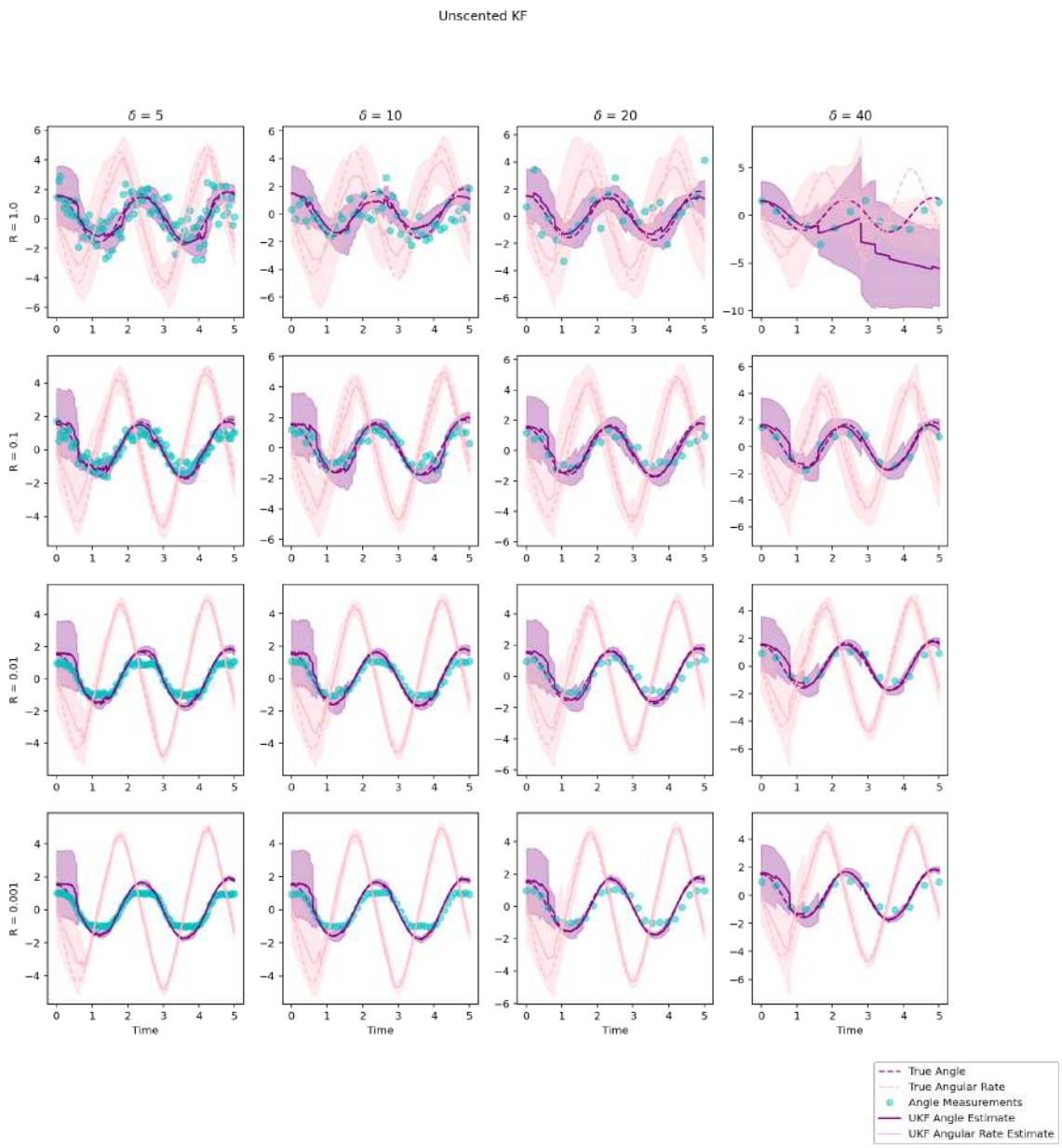
Unscented KF
δ = 5
δ = 10
δ = 20
δ = 40
R = 1.0
R = 0.1
R = 0.01
R = 0.001
Time
True Angle
True Angular Rate
Angle Measurements
UKF Angle Estimate
UKF Angular Rate Estimate

Figure 3

The mean squared error for all of the scenarios with UKF are given for both of the states in Table 2.

		Mean Squared Error	Measurement Timesteps			
		State	5	10	20	40
Noise Variance	1	1	0.1081	0.2052	0.1436	11.3669
		2	0.5219	1.3182	0.9611	8.4126
	0.1	1	0.0682	0.1631	0.0626	0.1158
		2	0.4386	0.4009	0.5053	0.5312
	0.01	1	0.0454	0.0560	0.0592	0.0723
		2	0.2145	0.2064	0.2320	0.2617
	0.001	1	0.0363	0.0406	0.0440	0.0767
		2	0.1925	0.2038	0.2394	0.2309

Table 2

2.3.Gauss-Hermite Kalman Filter of order 3 (GHKF3)

GHKF is discussed to be the most accurate and stable filters among the others introduced so far. However, it also happens to be the most expensive one because of the exponential increase in the evaluation points with respect to the dimensions. The Hermite polynomials are used recursively as:

$$H_{k+1}(x) = xH_k(x) - kH_{k-1}(x)$$

$$H_0(x) = 1$$

$$H_1(x) = x$$

$$H_2(x) = xH_1(x) - 1H_0(x) = x^2 - 1$$

$$H_3(x) = xH_2(x) - 2H_1(x) = x^3 - 3x$$

$$H_4(x) = xH_3(x) - 3H_2(x) = x^4 - 6x^2 + 3$$

$$H_5(x) = xH_4(x) - 4H_3(x) = x^5 - 10x^3 + 15x$$

And it is exact for polynomials up to order $2n-1$ with the nth order Hermite polynomial $H_n(x)$. When the integrands are approximated in an Hermite polynomial expansion and integrated, the multi-dimensional Gaussian distribution can be approximated as

$$\int h(X)\mathcal{N}(X\,|\,\mu,\Sigma)dX \approx \sum_{i=1}^{d} h\left(\mu+\sqrt{\Sigma}u^{(i)}\right)w^{(i)}$$

Here, two orders (of three and five) are implemented where

$$order = 2n-1$$

$$n = \frac{order+1}{2}$$

The results with GHKF with third order are shown below in Figure 4.

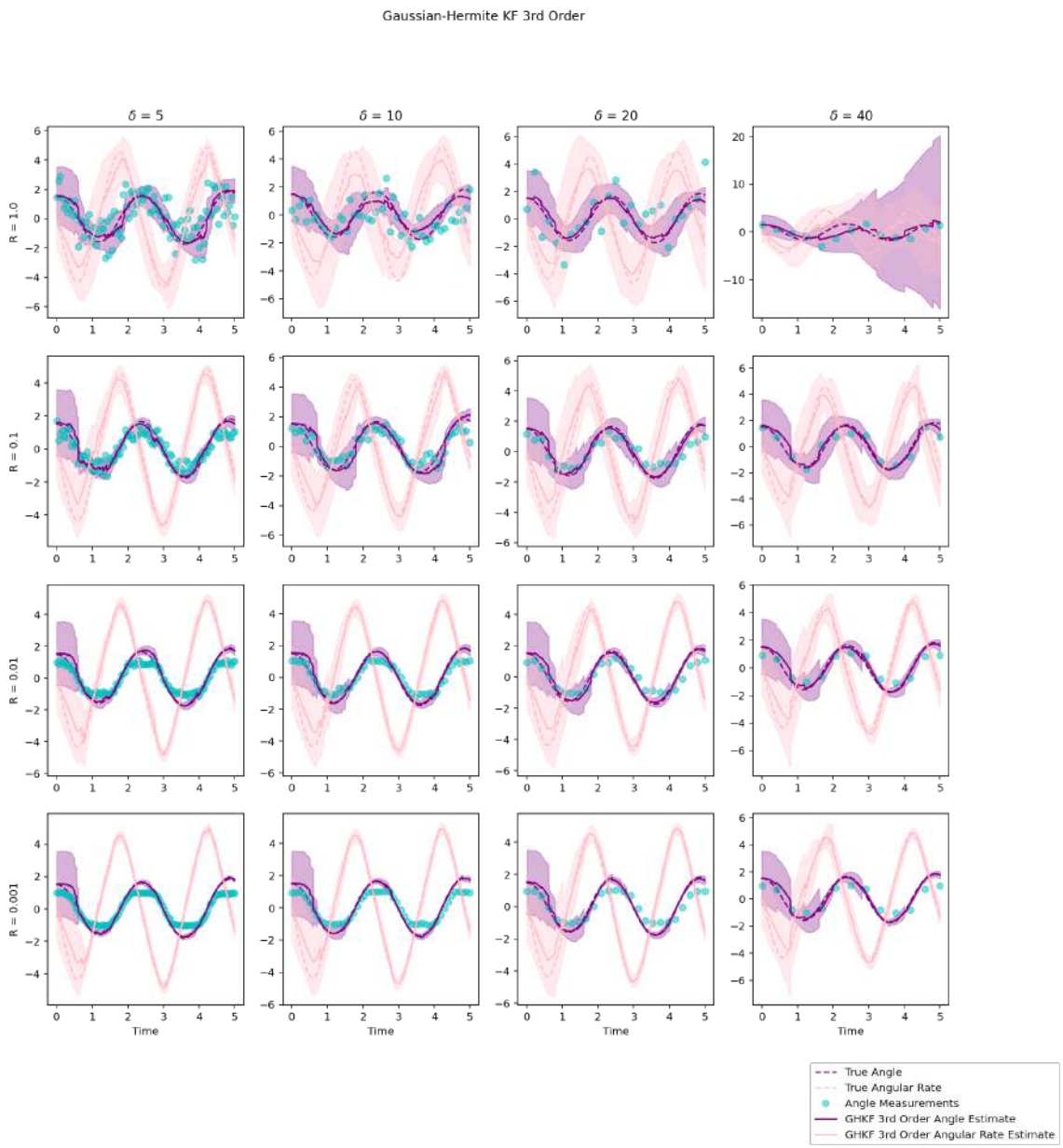

Figure 4

The mean squared error for all of the scenarios with GHKF third order are given for both of the states in Table 3.

		Mean Squared Error	Measurement Timesteps			
		State	5	10	20	40
Noise Variance	1	1	0.0970	0.1722	0.0783	0.7345
		2	0.4526	1.1291	0.6233	6.4787
	0.1	1	0.0621	0.1490	0.0474	0.0711
		2	0.4068	0.4460	0.4668	0.4613
	0.01	1	0.0349	0.0383	0.0431	0.0557
		2	0.1809	0.1706	0.2096	0.2646
	0.001	1	0.0263	0.0266	0.0285	0.0547
		2	0.1641	0.1690	0.2172	0.2347

Table 3

2.4.Gauss-Hermite Kalman Filter of order 5 (GHKF5)

The results with GHKF with fifth order are shown below in Figure 5.

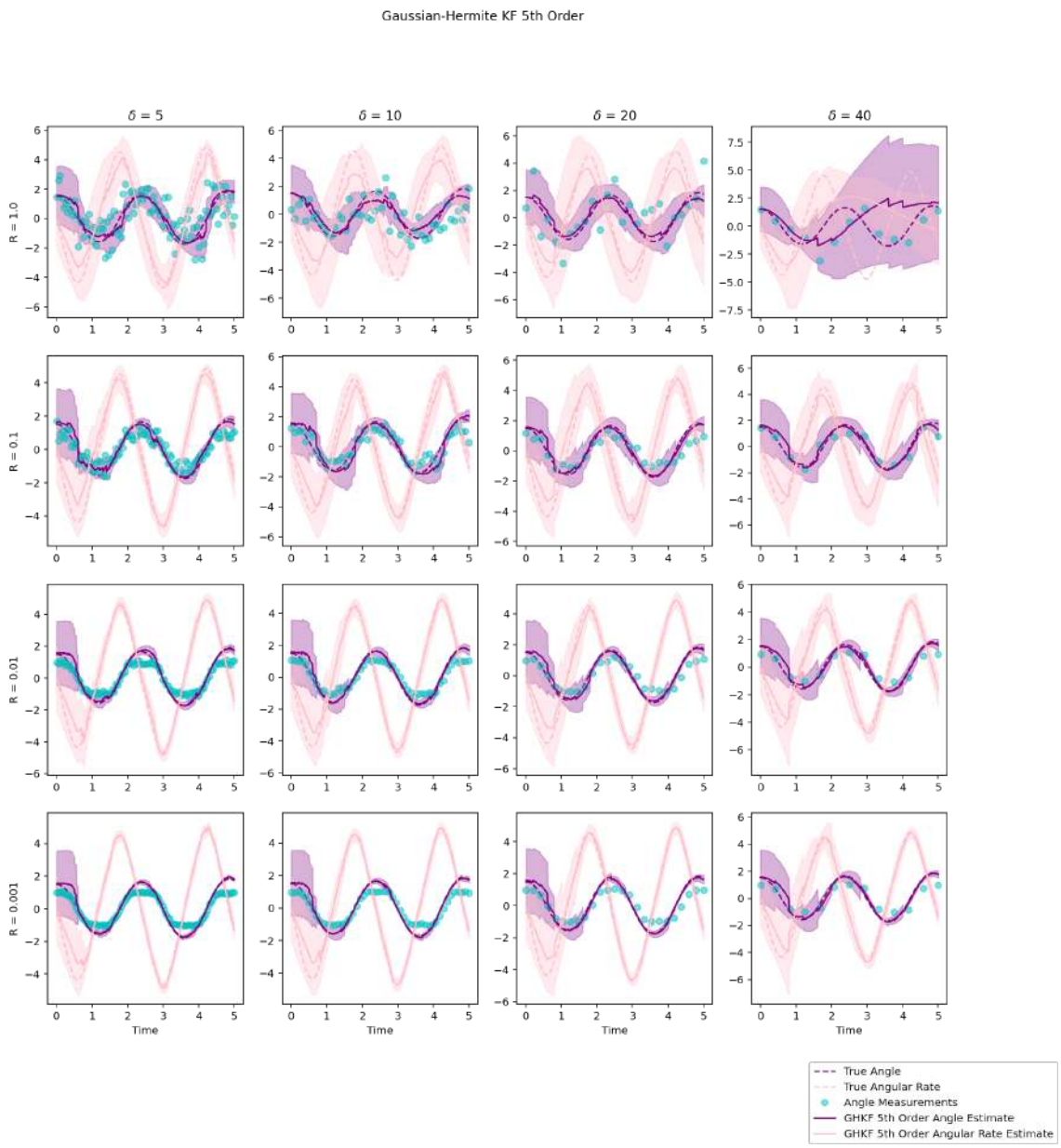

Figure 5

The mean squared error for all of the scenarios with GHKF fifth order are given for both of the states in Table 4.

		Mean Squared Error	Measurement Timesteps			
		State	5	10	20	40
Noise Variance	1	1	0.0982	0.1812	0.0906	3.4058
		2	0.4562	1.1877	0.6689	9.9906
	0.1	1	0.0649	0.1535	0.0527	0.0853
		2	0.3992	0.4197	0.4474	0.4820
	0.01	1	0.0393	0.0467	0.0495	0.0619
		2	0.1865	0.1770	0.2008	0.2564
	0.001	1	0.0306	0.0327	0.0354	0.0639
		2	0.1667	0.1719	0.2123	0.2041

Table 4

In order to compare the computational cost, the run times of each algorithm for all 16 scenarios are given below in Table 5.It can easily be seen that the computational cost drastically increases from EKF to GHKF of fifth order. This was expected considering the algorithm complexities of each filter and evaluation points.

Filter Type	Duration [sec]
EKF	0.12
UKF	0.64
GHKF3	0.73
GHKF5	1.44

Table 5

It can be seen from the mean squared errors and the graphs that as the time interval between the measures and the measurement noise gets bigger, the estimates get worse. The converse also holds, where the measurement noise and time between measurements are small, the estimates are better. This makes sense intuitively where good and more frequent measurements would lead to better estimates. With the lowest quality and least frequent measurements, the filters diverge (EKF almost always

whereas the others sometimes). The narrowing of the 2σ bands with better measurements can also be observed visually. GHKF is the most robust as expected theoretically and this can be observed when different runs are done where it does not diverge whereas the others (mostly EKF) sometimes diverge with cases other than the worst case, i.e. the least frequent and least quality measurements. Given the time is doubled from third to fifth order, and the filter performance on the estimates, third order GHKF can be the choice of filtering.

3. Particle Filtering

There is also another filtering alternative, particle filtering, which can be use when Gaussian approximations are not working well. Here, importing sampling is done to be used as the empirical distribution to have points and weights.Importance sampling was discussed on the previous projects, and it is once again implemented here. It uses importance sampling sequentially. When new data is available, the distribution is updated. The marginal likelihood is

$$\hat{P}(y_n, \mathscr{Y}_{n-1}) = \frac{1}{N}\sum_{i=1}^{N} v^{(i)}\bar{w}_{n-1}^{(i)}$$

$$where \quad w^{(i)} = v^{(i)}\bar{w}_{n-1}^{(i)} \quad and \quad \bar{w}_n^{(i)} = \sum_{i=1}^{N} w_n^{(i)}$$

$$v_n^{(i)} = \frac{P(y_n \,|\, x_n^{(i)})P(x_n^{(i)} \,|\, x_{n-1}^{(i)})}{\pi(x_n^{(i)} \,|\, \mathscr{X}_{n-1}^{(i)})}$$

Filter basically puts more importance with places with greater weights. The samples are got from these areas and use to propagate. Particle filtering for the pendulum is done with the system dynamics as the proposal. If no data is available, filter basically uses the proposal, the system dynamics. The three scenarios chosen for this purpose are:

$$\delta_1 = 40,\ R_1 = 1$$

$$\delta_2 = 10,\ R_2 = 0.1$$

$$\delta_3 = 5,\ R_3 = 0.001$$

which are the worst, medium and best cases respectively.

The particle filter run for the pendulum results in the estimates given below in Figure 6.

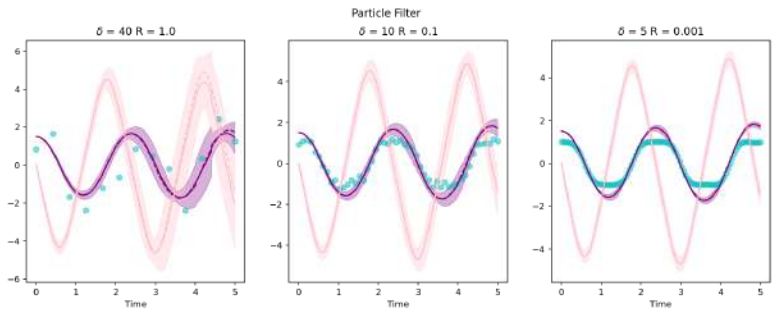

Figure 6

The joint posterior at five equally spaced time instances are shown below in Figure 7 for all of the scenarios as indicated.

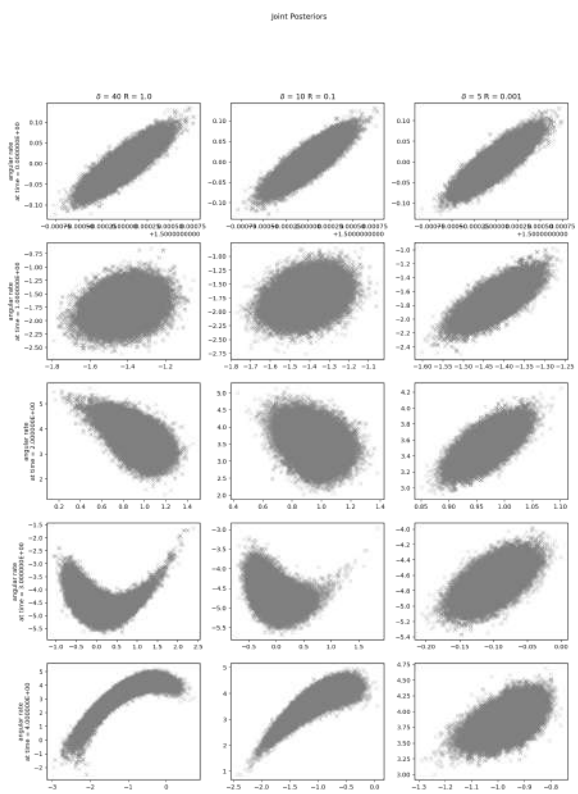

Figure7

The UKF run for the same conditions results in the estimates shown below in Figure 8 and the overlay of UKF approximations at the time instances shown in Figure 7 are given in Figure 9.

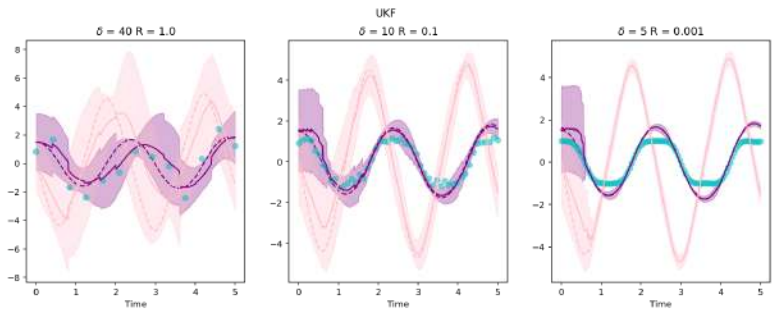

Figure 8

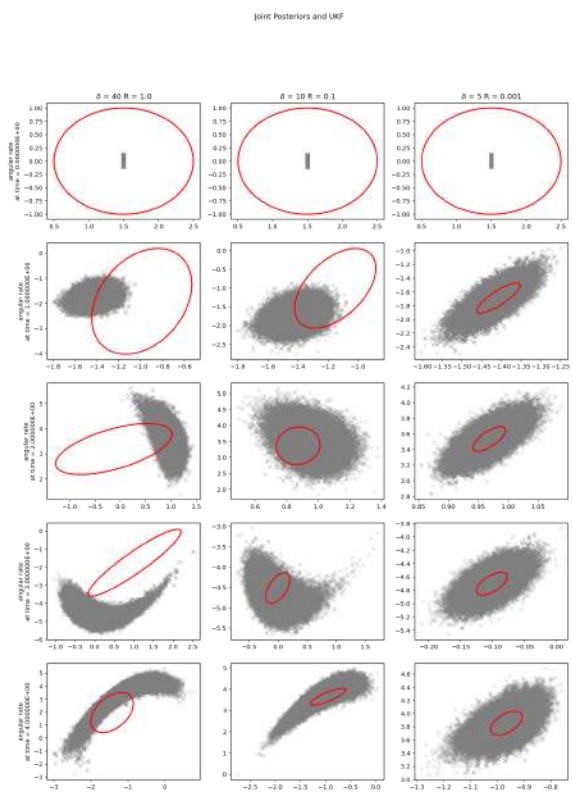

Figure 9

It can be seen that the first row encapsulates the points with the eye matrix initial covariance circle. Then, the last scenario with the most frequent and accurate measurements have the most Gaussian looking posteriors whereas the first one has the least. If all of the filters introduced are run for the fifth time instance, i.e. t=4 seconds, where the joint posterior is non-Gaussian, the moments are as shown in Figure 10. It can be seen that GHKF and UKF is better than EKF. Along different trials, GHKF is the most stable one in this graph. So, this also supports the claim that GHKF is the most robust one among the others.

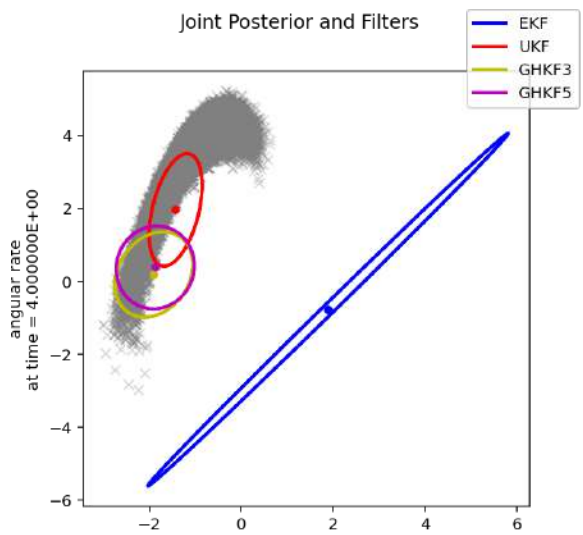

Figure 10

For the last scenario, a performance analysis is done with varying sample numbers. It is expected to have better convergence with increased number of samples. The convergence for the mean and covariance of the PF can be seen in Figure 11.

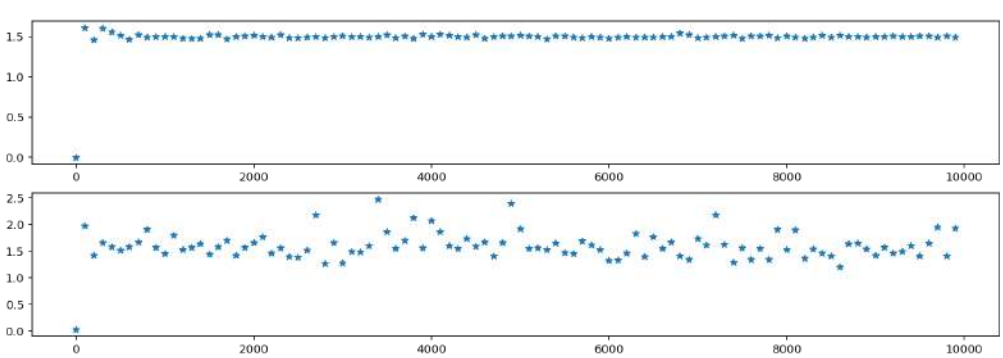

Figure 11

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.mpl_toolkits.mplot3d.axes3d, time, numpy, pandas
import numpy.random as rnd
import numpy.linalg as la
import matplotlib.pyplot as plt
import pandas
import matplotlib.mpl_toolkits.mplot3d.axes3d
import sklearn.cross_validation
import sklearn.tree
import sklearn.ensemble
import sklearn.metrics
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.cross_validation import train_test_split
from sklearn.metrics import roc_curve, auc
```

Features

```
def gradientplot(ax, df, target_column, bins=10):
    ax = ax.ravel()[:-1]
    for i, c in enumerate(df.columns):
        ax[i].hist(df[c], bins=bins)
        ax[i].set_title(c)
        ax[i].set_xticks([])
        ax[i].set_yticks([])
    for ii in range(len(df.columns), len(ax)):
        ax[ii].hist(df[df[target_column].isnull()][c], bins=bins)
        ax[ii].set_title("%s|nan" % c)
        ax[ii].set_xticks([])
        ax[ii].set_yticks([])
    print_text = "Blue: %s, Green: %s" % (str(df[target_column].unique()[0])[:20],
                                          str(df[target_column].unique()[1])[:20])
    if len(df[target_column].unique()) > 2:
        print_text = "Too many values in %s" % target_column
    ax[-1].text(0.5, 0.5, print_text, ha='center')
    ax[-1].set_xticks([])
    ax[-1].set_yticks([])
    return ax[:len(df.columns)]
```

2d

```
def linear_predictor(df, mean, std, target, debug=):
    '''Generate a linear predictor based on the data and the mean and the stdev
    df: a dataframe
    mean: a series of means, indexed by column
    std: a series of stdevs, indexed by column
    target: the column to predict

    Returns:
    lin_df: the linear model
    lin_mean: the mean of the model
    lin_std: the stdev of the model, normalized against mean=0 stdev=1
    lin_predictor: a function that takes a dataframe and returns a predictor dataseries

    Warning:
    lin_mean, lin_std: a dataframe indexed by column
    lin_predictor: a function that takes a dataframe

    print_debug = lin_debug(debug, 'linear')
    lin_df = df[mean.index].dropna() * mean[df[mean.index].dropna().columns] +
                                       std[df[mean.index].dropna().columns]
    print_debug(lin_df.describe())

    lin_mean = lin_df.mean(), lin_df.mean(axis=1).mean()
    lin_std = lin_df.std(), lin_df.std(axis=1).mean()

    return lin_df, lin_mean, lin_std, lin_predictor
```

2d

```
def linear_predictor_step(df, mean, std, target, debug=):
    '''Generate a linear predictor based on the data and the mean and the stdev
    df: a dataframe
    mean: a series of means, indexed by column
    std: a series of stdevs, indexed by column
    target: the column to predict

    Returns:
    lin_df: the linear model
    lin_mean: the mean of the model
    lin_std: the stdev of the model, normalized against mean=0 stdev=1
    lin_predictor: a function that takes a dataframe and returns a predictor dataseries

    Warning:
    lin_mean, lin_std: a dataframe indexed by column
    lin_predictor: a function that takes a dataframe

    print_debug = lin_debug(debug, 'linear')
    lin_df = df[mean.index].dropna() * mean[df[mean.index].dropna().columns] +
                                       std[df[mean.index].dropna().columns]
    print_debug(lin_df.describe())

    lin_mean = lin_df.mean(), lin_df.mean(axis=1).mean()
    lin_std = lin_df.std(), lin_df.std(axis=1).mean()

    return lin_df, lin_mean, lin_std, lin_predictor
```

2d

```
def binary_predictor(df, mean, std, target, bins=10, print_bins, debug=):
    '''Generate a binary predictor based on the data and the mean and the stdev
    df: a dataframe
    mean: a series of means, indexed by column
    std: a series of stdevs, indexed by column
    target: the column to predict
    bins: the number of bins to use when generating the predictor
    print_bins: if, along with debug, to print the bins
    debug: if to print debug messages

    Returns:
    bin_df: the binary model
    bin_mean: the mean of the model
    bin_std: the stdev of the model, normalized against mean=0 stdev=1
    bin_predictor: a function that takes a dataframe and returns a predictor dataseries

    print_debug = lin_debug(debug, 'binary')
    bin_df = df[mean.index].dropna() * mean[df[mean.index].dropna().columns] +
                                       std[df[mean.index].dropna().columns]
    print_debug(bin_df.describe())

    bin_mean = bin_df.mean(), bin_df.mean(axis=1).mean()
    bin_std = bin_df.std(), bin_df.std(axis=1).mean()

    print_bins = lin_debug(print_bins, 'bins')
    for c in bin_df.columns:
        bin_df[c] = pandas.cut(bin_df[c], bins, retbins=True)[1][:-1]
        print_bins(c, bin_df[c].unique())

    bin_predictor = lambda df: df[bin_df.columns].isin(bin_df[bin_df.columns])

    return bin_df, bin_mean, bin_std, bin_predictor
```

2d

```
def binary(df):
    '''Binarize df
    df: a dataframe

    Returns:
    bin_df: a binarized version of df

    Warning:
    bin_df = df.copy()
    for c in df.columns:
        bin_df[c] = df[c].isnull()
        if len(df[c].unique()) == 2:
            bin_df[c] = df[c] == df[c].unique()[0]
        elif len(df[c].unique()) > 2:
            for u in df[c].unique():
                bin_df[c + '|' + str(u)] = df[c] == u
    return bin_df
```

2d

```
def binary_predictor_step(df, mean, std, target, debug=):
    '''Generate a binary predictor based on the data and the mean and the stdev
    df: a dataframe
    mean: a series of means, indexed by column
    std: a series of stdevs, indexed by column
    target: the column to predict
    debug: if to print debug messages

    Returns:
    bin_df: the binary model
    bin_mean: the mean of the model
    bin_std: the stdev of the model, normalized against mean=0 stdev=1
    bin_predictor: a function that takes a dataframe and returns a predictor dataseries

    print_debug = lin_debug(debug, 'binary')
    bin_df = df[mean.index].dropna() * mean[df[mean.index].dropna().columns] +
                                       std[df[mean.index].dropna().columns]
    print_debug(bin_df.describe())

    bin_mean = bin_df.mean(), bin_df.mean(axis=1).mean()
    bin_std = bin_df.std(), bin_df.std(axis=1).mean()

    bin_predictor = lambda df: df[bin_df.columns].isin(bin_df[bin_df.columns])

    return bin_df, bin_mean, bin_std, bin_predictor
```

2d

```
def binary_predictor_step(df, mean, std, target, bins=10, print_bins, debug=):
    '''Generate a binary predictor based on the data and the mean and the stdev
    df: a dataframe
    mean: a series of means, indexed by column
    std: a series of stdevs, indexed by column
    target: the column to predict
    bins: the number of bins to use when generating the predictor
    print_bins: if, along with debug, to print the bins
    debug: if to print debug messages

    Returns:
    bin_df: the binary model
    bin_mean: the mean of the model
    bin_std: the stdev of the model, normalized against mean=0 stdev=1
    bin_predictor: a function that takes a dataframe and returns a predictor dataseries

    print_debug = lin_debug(debug, 'binary')
    bin_df = df[mean.index].dropna() * mean[df[mean.index].dropna().columns] +
                                       std[df[mean.index].dropna().columns]
    print_debug(bin_df.describe())

    print_bins = lin_debug(print_bins, 'bins')
    for c in bin_df.columns:
        bin_df[c] = pandas.cut(bin_df[c], bins, retbins=True)[1][:-1]
        print_bins(c, bin_df[c].unique())

    bin_predictor = lambda df: df[bin_df.columns].isin(bin_df[bin_df.columns])

    return bin_df, bin_mean, bin_std, bin_predictor
```

2d

```
def binarize(df):
    '''Binarize df
    df: a dataframe

    Returns:
    bin_df: a binarized version of df

    Warning:
    bin_df = df.copy()
    for c in df.columns:
        bin_df[c] = df[c].isnull()
        if len(df[c].unique()) == 2:
            bin_df[c] = df[c] == df[c].unique()[0]
        elif len(df[c].unique()) > 2:
            for u in df[c].unique():
                bin_df[c + '|' + str(u)] = df[c] == u
    return bin_df
```

2d

```
def binary(df):
    '''Binarize df
    df: a dataframe

    Returns:
    bin_df: a binarized version of df

    Warning:
    bin_df = df.copy()
    for c in df.columns:
        bin_df[c] = df[c].isnull()
        if len(df[c].unique()) == 2:
            bin_df[c] = df[c] == df[c].unique()[0]
        elif len(df[c].unique()) > 2:
            for u in df[c].unique():
                bin_df[c + '|' + str(u)] = df[c] == u
    return bin_df
```

In [28]:

```
    Mean[ii, i] = np.dot(weights[:, ii], samples[:, i, ii])
    Var[ii, i] = np.exp(np.dot(weights[:, ii], (samples[:, i, ii] - np.tile(Mean[ii, i], (N, 1)))**2))
    return means, stds


def step(proxy, propgdf, current_samples, current_weights, likelihood, data, propagator, R):
    """
    Propagate a particle filter

    @param:[in] prop            - proposal function (current_state, data)
    @param:[in] propgdf         - proposal function logpdf
    @param:[in] current_samples - ensemble of samples
    @param:[in] current_weights - ensemble of weights
    @param:[in] likelihood      - function to evaluate the log likelihood (samples, data)
    @param:[in] data            - observation
    @param:[in] propagator      - dynamics logpdf

    Evercurse samples and weights after assimilating the data
    """
    new_samples = prop(current_samples, data)
    new_weights = likelihood(new_samples, data, R) + propagator(new_samples, current_samples) - \
                  propgdf(new_samples, current_samples, data)
    new_weights = np.exp(new_weights) * current_weights
    new_weights = new_weights / np.sum(new_weights)
    return new_samples, new_weights


def particle_filter(data, prior_mean, prior_cov,
                    prop, propgdf, likelihood, propagator, delta,R,
                    nsamples=100, resampling_threshold_frac=0.2):
    """Particle Filter

    Inputs
    ------
    data: (nsteps, m) array of data points, N is the time index, m is the dimensionality of the data
    prior_mean: (d), prior mean
    prior_cov: (d, d), prior mean
    Nsamples: integer, number of samples in the empirical distribution
    resampling_threshold_frac: float between 0 and 1 indicating to resample when effective sample size below frac of nsamples

    Returns
    -------
    samples: (nsamples, d, nsteps)
    weights: (nsamples, nsteps)
    eff: (nsamples), effective sample size

    Notes
    -----
    For documentation of prop, propgdf, likelihood, and propagator -- see the step function
    """
    d = prior_mean.shape[0]
    nsteps = data.shape[0]
    # Allocate memory
    samples = np.zeros((nsamples, d, nsteps+1))
    weights = np.zeros((nsamples, nsteps+1))
    eff = np.zeros((nsteps+1)) # keep track of effective sample size at each step
    # Generate initial samples from the prior
    L = np.linalg.cholesky(prior_cov)
    samples[:, :, 0] = np.tile(prior_mean, (nsamples, 1))+ np.dot(L, np.random.randn(d, nsamples),).T
    weights[:, 0] = 1.0 / nsamples # all weights are equal because of independent sampling from prior
    eff[0] = nsamples
    resamp_thresh = int(nsamples * resampling_threshold_frac)
    for ii in range(0, nsteps):
        if ii%delta==1:
            samples[:, :, ii], weights[:, ii] = step(prop,  propgdf, samples[:, :, ii-1], weights[:, ii-1],
                                                     likelihood, data[ii-1,:], propagator,R)

        else:
            samples[:,:,ii] = prop(samples[:, :, ii-1], data[ii-1,:])
            weights[:,ii]=weights[:,ii-1]

        # compute the effective sample size
        eff[ii] = 1.0 / np.sum(weights[:, ii]**2)

        # if ii % 50 == 0:
            #print('eff = ', ii, eff[ii])
        #print('eff/mean = ', eff[ii], mean(ii, :))

        # resample if effective sample size is below threshold
        if eff[ii] < resamp_threshold:
            samples[:, :, ii], weights[:, ii] = resample(nsamples, samples[:, :, ii], weights[:, ii])
    return samples, weights, eff
```

In [30]:

```
# Process noise
proc_var=0.1
proc_mat = np.zeros((1,2))
proc_mat[0, 0] = proc_var/3.0*dt**3
proc_mat[0, 1] = proc_var/2.0*dt**2
proc_mat[1, 0] = proc_var/2.0*dt**2
proc_mat[1, 1] = proc_var*dt
proc_mat_inv = np.linalg.pinv(proc_mat)
lprocc = np.linalg.cholesky(proc_mat)

def proposal(current_state, data=None, dt=dt):
    """ Bootstrap Particle Filter the proposal is the dynamics!"""

    if current_state.ndim == 1:
        return pendulum_dyn(current_state, dt=dt) + np.dot(lprocc, np.random.randn(2))
    else:
        nsamples = current_state.shape[0]
        return pendulum_dyn(current_state, dt=dt) + np.dot(lprocc, np.random.randn(2, nsamples).T)

def proposal_logpdf(current, previous, data=None, proc_var=0.1):
    """ Bootstrap Particle Filter: the proposal is the dynamics"""
    next1 = pendulum_dyn(previous, dt=dt)
    delta = next1 - current
    if current.ndim == 1:
        return -0.5 * np.dot(delta, np.dot(proc_mat_inv, delta))
    else:
        return -0.5 * np.sum(delta * np.dot(delta, proc_mat_inv.T), axis=1)

def likelihood(state, data, noise_var):
    """Gaussian Likelihood through nonlinear model"""
    dpropose = observe(state)
    delta = dpropose - data
    if state.ndim == 1:
        return -0.5 * np.dot(delta, delta) / noise_var
    else:
        return -0.5 * np.sum(delta * delta, axis=1) / noise_var
```

In [34]:

```
custom_lines = [Line2D([0], [0], color='b', lw=2),
                Line2D([0], [0], color='r', lw=2),
                Line2D([0], [0], color='y', lw=2),
                Line2D([0], [0], color='m', lw=2)]
```

In [36]:

```
# random particle
x0 = np.array([1.5, 0])
Nsteps = 500
N = Nsteps
qc = 0.1
deltat = 0.01
dt = deltat
delta_array = np.array([45,10,5])
Marray = np.array([1, 0.1, 0.001])
Q = np.array([[(qc*deltat**3)/3 , (qc*deltat**2)/2 ],
              [(qc*deltat**2)/2 ,  qc*deltat]])

proc_var = qc
xlim = [-6, 6]
ylim = [-6, 6]

fig0, ap0 = plt.subplots(1,3,figsize=(15,5))
fig1, ap1 = plt.subplots(1,3,figsize=(15,5))
fig2, ap2 = plt.subplots(1,3,figsize=(15,20))
fig3, ap3 = plt.subplots(1,3,figsize=(15,20))
fig4, ap4 = plt.subplots(1,3,figsize=(5,5))


for i in range(3):
    delta = deltaArray[i]
    R = Marray[i]
    obs = np.zeros((Nsteps, 2))
    true[0, :] = x0
    times = np.arange(0, Nsteps*dt, dt)
    data = np.zeros((Nsteps-1, 1))
    na = R
    noise_var = na*na

    proc_mat = Q
    proc_mat_inv = np.linalg.pinv(proc_mat)
    lprocc = np.linalg.cholesky(proc_mat)

    for ii in range(1, Nsteps):
        true[ii, :] = pendulum_dyn(true[ii-1, :],dt=dt)
        data[ii-1] = observe(true[ii, :]) + np.random.randn()*R

    T = N*deltat
    T = np.linspace(0, T, int(np.ceil(N/delta)))

    # PF
    prior_mean = true[0, :] # start at the truth -- this is obviously not possible in reality
    prior_cov = Q #np.eye(2) # identity covariance
    samples, weights, eff = particle_filter(data, prior_mean, prior_cov,
                                            proposal, proposal_logpdf, likelihood, proposal_logpdf,delta,R,
                                            nsamples=10000, resampling_threshold_frac=0.1)

    MEANPF, STDPF= compute_mean_std(samples, weights)
    MERANPF = MEANPF.T
    j=0
    ap0[i].plot(times, true[:, 0],'--', color='purple', label='True Angle')
    ap0[i].plot(times, true[:, 1],'--', color='pink', label='True Angular Rate')
    ty = np.linspace(0, T, int(np.ceil(N/delta)))
    ap1[i].plot(ty, data[:-deltaj, 0], 'o', alpha=0.5, label='Angle Measurements')
    ap1[i].plot(ty, MERANPF[:,0], color='purple', label='UKF Angular Rate Estimate')
    ap1[i].plot(times, MERANPF[:,1], color='pink', label='UKF Angular Rate Estimate')
    ap1[i].fill_between(times, MERANPF[:,0] - 2 * STDPF[:, 0], MERANPF[:,0] + 2*STDPF[:, 0],
                        color='purple', alpha=0.2)
    ap1[i].fill_between(times, MERANPF[:,1] - 2 * STDPF[:, 1],  MERANPF[:,1] + 2*STDPF[:, 1],
                        color='pink', alpha=.3)
    if j == 0: #ap.set(deltaArray[i])
        ap0[i].set_xlabel('Time', fontsize=10)
    if j == 0:
        ap0[i].set_title('$\deltat = ' +str(delta) + ' R = ' +str(R)

    # UKF
    prior_mean = x0.T # prior mean
    prior_cov = np.eye(2) # prior covariance
    proc_cov = Q # lw=1 * np.eye(2) # process noise covariance
    meas_cov = R # lw=1 * np.eye(R.shape[0]) #measurement noise covariance
    ukf = UKF(dt=dt,
    meanpost, covpost = UKF(R, yuse, proc_cov, meas_cov, prior_mean, prior_cov, delta)
    std_dera = get_std(covpost) #extract the standard deviations at all the states
    T = N*deltat
    T = np.linspace(0, T, N+1)
    ap1[i].plot(times, true[:, 0],'--', color='purple', label='True Angle')
    ap1[i].plot(times, true[:, 1],'--', color='pink', label='True Angular Rate')
    ap1[i].plot(ty, yuse, 'oo', alpha=0.5, label='Angle Measurements')
    ap1[i].plot(t, meanpost[0,:], color='purple', label='UKF Angle Estimate')
    ap1[i].plot(t, meanpost[1,:], color='pink', label='UKF Angular Rate Estimate')
    ap1[i].fill_between(t, meanpost[0,:] - 2 * std_dera[:, 0],  meanpost[0,:] + 2*std_dera[:, 0],
                        color='purple', alpha=0.3)
    ap1[i].fill_between(t, meanpost[1,:] - 2 * std_dera[:, 1],  meanpost[1,:] + 2*std_dera[:, 1],
                        color='pink', alpha=0.3)
    if j == 0: #ap.set(deltaArray[i])
        ap1[i].set_xlabel('Time', fontsize=10)
    if j == 0:
        ap1[i].set_title('$\deltat = ' +str(delta) + ' R = ' +str(R)

    # PART 2
    d, N = MEANPF.shape
    kk = 0
    for ii in range(N):
        if ii % 100 == 0:
            Nsamples = 100000
            weightsii = weights[:, ii]
            samplesii = samples[:, :, ii]
            s, w = resample(Nsamples, samplesii, weightsii) # resample to obtain equal weights
            xspace = np.linspace(xlim[0], xlim[1],100)
            yspace = np.linspace(ylim[0], ylim[1],100)
            XX, YY = np.meshgrid(xspace, yspace)
            positions = np.vstack([XX.ravel(), YY.ravel()])
            values = np.vstack([s[:, 0], s[:, 1]])
            kernel = kde(values, bw=[0.1, 0.1])
            f = np.reshape(kernel(positions).T, XX.shape)
            ap0.plot(s[:, 0], s[:, 1], 'x', color='grey', alpha=0.3,zorder=0)

            if i == 0:
                ap0[kk,i].set_ylabel('angular rate\nat time = {:5E}'.format(times[ii]))
            kk = kk+1
    kk = 0
    for i in range(N):
        if i % 100 == 0:
            Nsamples = 100000
            weightsii = weights[:, ii]
            samplesii = samples[:, :, ii]
            s, w = resample(Nsamples, samplesii, weightsii) # resample to obtain equal weights

            xspace = np.linspace(xlim[0], xlim[1],100)
            yspace = np.linspace(ylim[0], ylim[1],100)
            XX, YY = np.meshgrid(xspace, yspace)

            positions = np.vstack([XX.ravel(), YY.ravel()])
            values = np.vstack([s[:, 0], s[:, 1]])
            kernel = stats.gaussian_kde(values) # kernel density estimate to get contours
            f = np.reshape(kernel(positions).T, XX.shape)
            ap1[kk,i].plot(s[:, 0], s[:, 1], 'x', color='grey', alpha=0.3,zorder=0)

            if i == 0:
                ap1[kk,i].set_ylabel('angular rate\nat time = {:5E}'.format(times[ii]))
            kk = kk+1
    if j == 0:
        ap0[j,i].set_title('$\deltat = ' +str(delta) + ' R = ' +str(R)

    if j == 0:
        ap0[j,i].set_title('$\deltat = ' +str(delta) + ' R = ' +str(R)

    # PART 3
    if i == 0:
        #EKF
        meanpostEKF, covpostEKF = kalman_filter(R, yuse, proc_cov, meas_cov, prior_mean, prior_cov, deltat, delta)
        std_deveEKF = get_std(covpostEKF) #extract the standard deviations at all the states

        #UKF
        meanpostUKF, covpostUKF = UKF(R, yuse, proc_cov, meas_cov, prior_mean, prior_cov, delta)
        std_deveUKF = get_std(covpostUKF) #extract the standard deviations at all the states

        #GHKF 1
        order = 3
        orderd = int(order+1)/2
        meanpostGHKF1, covpostGHKF1 = GHKF(R, yuse, proc_cov, meas_cov, prior_mean, prior_cov, delta)
        std_devaGHKF1 = get_std(covpostGHKF1) #extract the standard deviations at all the states

        #GHKF 5
        order = 5
        orderd = int(order+1)/2
        meanpostGHKF5, covpostGHKF5 = GHKF(R, yuse, proc_cov, meas_cov, prior_mean, prior_cov, delta)
        std_devaGHKF5 = get_std(covpostGHKF5) #extract the standard deviations at all the states

    for ii in range(N):
        if ii == 400:
            Nsamples = 100000
            weightsii = weights[:, ii]
            samplesii = samples[:, :, ii]
            s, w = resample(Nsamples, samplesii, weightsii) # resample to obtain equal weights
            xspace = np.linspace(xlim[0], xlim[1],100)
            yspace = np.linspace(ylim[0], ylim[1],100)
            XX, YY = np.meshgrid(xspace, yspace)
            positions = np.vstack([XX.ravel(), YY.ravel()])
            values = np.vstack([s[:, 0], s[:, 1]])
            kernel = stats.gaussian_kde(values) # kernel density estimate to get contours
            f = np.reshape(kernel(positions).T, XX.shape)
            ap9.plot(s[:, 0], s[:, 1], 'x', color='grey', alpha=0.3,zorder=0)


            lambda_EKF, vEKF = np.linalg.eig(covpostEKF[:,:,ii])
            lambda_EKF = np.sqrt(lambda_EKF)
            ellEKF = Ellipse(xy=(meanpostEKF[0,ii], meanpostEKF[1,ii]),
                             width=lambda_EKF[0]*1*2, height=lambda_EKF[1]*1*2,
                             angle=np.rad2deg(np.arccos(vEKF[0, 0])), edgecolor='b', fc='None', lw=2)

            lambda_UKF, vUKF = np.linalg.eig(covpostUKF[:,:,ii])
            lambda_UKF = np.sqrt(lambda_UKF)
            ellUKF = Ellipse(xy=(meanpostUKF[0,ii], meanpostUKF[1,ii]),
                             width=lambda_UKF[0]*1*2, height=lambda_UKF[1]*1*2,
                             angle=np.rad2deg(np.arccos(vUKF[0, 0])), edgecolor='r', fc='None', lw=2)


            lambda_GHKF1, vGHKF1 = np.linalg.eig(covpostGHKF1[:,:,ii])
            lambda_GHKF1 = np.sqrt(lambda_GHKF1)
            ellGHKF1 = Ellipse(xy=(meanpostGHKF1[0,ii], meanpostGHKF1[1,ii]),
                               width=lambda_GHKF1[0]*1*2, height=lambda_GHKF1[1]*1*2,
                               angle=np.rad2deg(np.arccos(vGHKF1[0, 0])), edgecolor='y', fc='None', lw=2)

            lambda_GHKF5, vGHKF5 = np.linalg.eig(covpostGHKF5[:,:,ii])
            lambda_GHKF5 = np.sqrt(lambda_GHKF5)
            ellGHKF5 = Ellipse(xy=(meanpostGHKF5[0,ii], meanpostGHKF5[1,ii]),
                               width=lambda_GHKF5[0]*1*2, height=lambda_GHKF5[1]*1*2,
                               angle=np.rad2deg(np.arccos(vGHKF5[0, 0])), edgecolor='m', fc='None', lw=2)

            ellEKF.set_facecolor('none')
            ellUKF.set_facecolor('none')
            ellGHKF1.set_facecolor('none')
            ellGHKF5.set_facecolor('none')
            ap9.add_patch(ellEKF)
            ap9.add_patch(ellUKF)
            ap9.add_patch(ellGHKF1)
            ap9.add_patch(ellGHKF5)
            ap9.plot(meanpostEKF[0,ii],meanpostEKF[1,ii],'o',color = 'b',markersize=4,label='EKF')
            ap9.plot(meanpostUKF[0,ii],meanpostUKF[1,ii],'o',color = 'r',markersize=4,label='UKF')
            ap9.plot(meanpostGHKF1[0,ii],meanpostGHKF1[1,ii],'o',color = 'y',markersize=4,label='GHKF1')
            ap9.plot(meanpostGHKF5[0,ii],meanpostGHKF5[1,ii],'o',color = 'm',markersize=4,label='GHKF5')


            ap9.set_ylabel('angular rate\nat time = {:5E}'.format(times[ii]))

    ap9[i].legend(bbox_to_anchor=(2,0), loc='lower right',
                  bbox_transform=fig.transFigure, ncol=1)
    ap4[i].legend(bbox_to_anchor=(2,0), loc='lower right',
                  bbox_transform=fig.transFigure, ncol=1)
    fig9.legend(custom_lines, ['EKF', 'UKF', 'GHKF1', 'GHKF5'])

    # fig9.legend()
    fig9.suptitle('Particle Filter',fontsize=12)
    fig9.subplots_adjust(hspace=1.2)
    fig9.suptitle('Joint Posteriors',fontsize=12)
    fig8.suptitle('Joint Posterior and UKF',fontsize=12)
    fig8.suptitle('Joint Posterior and Filters',fontsize=12)
```

Out[36]:

```
Text(0.5, 0.98, 'Joint Posterior and Filters')
```

In [99]:

```
iterator = 1
num_iterations = 100
meas_array = np.zeros(num_iterations)
cov_array = np.zeros(num_iterations)
for i in range(num_iterations):
    nsamples = 10*iterator[-1 + 1
    samples, weights, eff = particle_filter(data, prior_mean, prior_cov,
                                            proposal, proposal_logpdf, likelihood, proposal_logpdf,delta,R,
                                            nsamples=nsamples, resampling_threshold_frac=0.1)
    means, stds = compute_mean_std(samples, weights)
    x, w = resample(nsamples, samples[:,:,499], weights[:,499])
    cov = np.cov(x[:,0],x[:,1])
    mean_array[i] = means[0][0]
    norm_cov = np.linalg.norm(np.cov(samples[0,0,:],weights[0,:]))
    cov_Array[i] = norm_cov
fig10, ap10 = plt.subplots(2,1,figsize=(15,5))
t1 = np.arange(1, iterator-1 + 1, 1)[1:]
ap10[0].plot(t1,mean_array[1:])
ap10[1].plot(t1,cov_array[1:])
```

Out[99]:

```
[<matplotlib.lines.Line2D at 0x2d03bcd0>]
```

In [ ]:

UNIVERSITY OF MICHIGAN

AE567

Statistical Inference, Estimation, and Learning

Final Project:
Literature Review on Initial Alignment Methods

Deniz Dost

UM ID: 82337583

I. BACKGROUND & PROBLEM STATEMENT

Initial alignment is the procedure of determining the initial orientation of the system before starting navigation, guidance and control. It is a procedure which holds a crucial role in aerospace missions, such as satellite launchers, to have better navigation accuracy. The output of this system comprises the initialization, and the success, of the navigation procedure. There are multiple methods to find the attitude of the system including theodolite and autocollimators, magnetometers, star trackers etc. However, this paper focuses on the attitude estimation on a stationary base with strap-down inertial navigation system (SINS) which constitutes of a three axis accelerometer and gyroscope inertial measurement unit (IMU).

There are two parts to initial alignment; coarse and fine. In the coarse alignment phase, the attitude, usually the Euler 3-2-1 rotation angles between the body and the local navigation frames, is calculated roughly from the IMU measurements analytically. Fine alignment uses this initial value to do estimation and decrease the error of the alignment process. The third Euler angle is usually the most challenging one. The purpose of this report is to investigate different alignment techniques with various states, dynamics, procedures, and measurements as well as different filtering techniques, their effectiveness and comparison from the results given in the works in a literature review to gain a broader perspective on different types of filtering procedures in addition to linking the ones that we have covered so far in the course. Therefore, the main focus will be on the fine alignment part. However, a brief explanation of the coarse alignment is also provided to construct the background of alignment as a concept as well as previously implemented satellite launcher mission alignment procedures.

II. REAL-LIFE APPLICATIONS

A conventional, traditional, way of initial alignment is the optical/physical alignment done with the help of a theodolite. This can be done either in long and short range as shown in Figure 1 respectively [1]. This procedure is used for Saturn vehicles, and takes 60 minutes and has met the ±0.00055° accuracy requirement on all of the missions used [2]. However, this is not a very practical alignment technique regarding the physical constraints such as launch platform/vehicle, time and labor restrictions. So, coming up with an algorithmic solution to the alignment is desired.

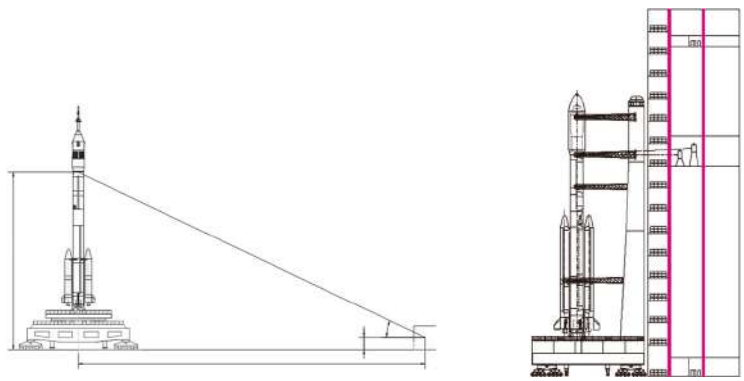

Figure 1

Orion system, for NASA, has a navigation proposal [3] in which a coarse alignment procedure with a low pass second order filter is applied to the IMU measurements in order to eliminate the effects of noise and twist and sway induced oscillations. Three methods for initial alignment EKF is introduced for EM-1 (Exploration Mission).

- Integrated Velocity: Exploration Flight Test-1 used this type of measurement for initial alignment. This measurement gives the Earth fixed frame position difference in EKF call rate, which is chosen to be one second. This measurement is not linear, but the nonlinearities are not strong in one second intervals. This is set to zero, knowing the position is

not changing. Noise is obtained by the position change caused by twist and sway of IMU.

- Zero Velocity: Using the knowledge of IMU velocity being zero, this is a linear measurement. Noise is obtained by the sway and twist induced velocity.
- Pad Position: Using the IMU position, the measurement includes errors due to the position survey errors (bias) in addition to the twist and sway motion (white noise).

Then, a trade factor analysis is conducted to choose Pad Position to apply for the next mission [3]. The initial alignment procedure is approximately 40 minutes [4].

Ares I proposes to use an hour period of time for initial alignment on ground and a several initial alignment options as explained below [5].

- Usage of conventional way of optical alignment: as introduced above for Saturn system [2]. However, the shortcomings are also applicable to here in terms of practical use.
- Usage of two orientations: two gyros rotated 90° with respect to each other around the Up axis. This method is shown to reduce the azimuth error from 0.083° to 0.028° when compared to single orientation.
- Usage of multiple IMUs: the system may include multiple medium quality IMUs for the mission, so utilizing them to increase initializing accuracy (to 0.017°) is also possible given that a precise alignment (around 0.028°) is done between the IMUs.
- Using GPS on the ascent: using the errors created by the initialization errors on the ascent phase and comparing these to the GPS measurements.

- Using GPS: using GPS data before launch can increase the convergence rate, however, it does not provide any improvements compared ZVU. It also creates some disadvantages, since this can cause some delays when GPS is not available.
- Using high quality IMU: a direct solution of using a high quality IMU which provides better measurements.

The report also obtains the theoretical error of gyro-compassing as a function of IMU parameters are [5]:

$$Gyrocompassing\ Error = \frac{East\ Gyro\ Drift}{\omega_e \cos(lat)}$$

$$Leveling\ Error = \arctan\left(\frac{Accelerometer\ Bias}{g}\right)$$

III. COARSE ALIGNMENT METHODS

Coarse alignment, also known as gyro compassing, takes the measurements from IMU to coincide the zero acceleration in North and East and zero angular velocity in East axes) to find the orientation from the local navigation frame to the body frame. This can usually be represented as angles or direction cosine matrix C, also known as the rotation matrix.

For instance, [6] computes the C_N^B as

$$\begin{array}{lll} C_{11} = \frac{\omega_x}{\Omega cosL} - \frac{g_x tanL}{g} & C_{21} = -C_{12}C_{33} + C_{13}C_{32} & C_{31} = -\frac{g_x}{g} \\ C_{11} = \frac{\omega_y}{\Omega cosL} - \frac{g_y tanL}{g} & C_{22} = C_{11}C_{33} - C_{31}C_{13} & C_{32} = -\frac{g_y}{g} \\ C_{11} = \frac{\omega_z}{\Omega cosL} - \frac{g_z tanL}{g} & C_{22} = -C_{11}C_{32} - C_{31}C_{11} & C_{33} = -\frac{g_z}{g} \end{array}$$

From the measurements $g^B = \begin{bmatrix} g_x \\ g_y \\ g_z \end{bmatrix}\ and\ \omega_{IE}^B = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$

Given $g^N = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$, $g^N = \begin{bmatrix} \Omega cosL \\ 0 \\ -\Omega sinL \end{bmatrix}$ where Ω and L denote Earth's rotation rate and latitude respectively on NED navigation frame.

In an alternative way with the same logic, [7] computes the Euler angles from measurements from the accelerometer and gyro measurements , f^B_{IB} and ω^B_{IB}, as shown below:

$$\theta_{NB} = arctan\left(\frac{-f^B_{IB,x}}{\sqrt{(f^B_{IB,y})^2 + (f^B_{IB,z})^2}}\right) \quad \phi_{NB} = arctan2(-f^B_{IB,y}, f^B_{IB,z}) \quad \psi_{NB} = arctan2(sin\psi_{NB}, cos\psi_{NB})$$

$$where,\ sin\psi_{NB} = -\omega^B_{IB,y}cos\phi_{NB} + \omega^B_{IB,z}sin\phi_{NB} \quad and \quad cos\psi_{NB} = \omega^B_{IB,x}cos\theta_{NB} - \omega^B_{IB,y}sin\phi_{NB}sin\theta_{NB} + \omega^B_{IB,z}cos\phi_{NB}sin\theta_{NB}$$

where $\phi_{NB}, \theta_{NB}, \psi_{NB}$ are the roll, pitch, and yaw Euler angles respectively.

In this case, the accuracy of the alignment is directly affected by the accelerometer and gyro errors and can be represented as

$$\sigma^2_{gyrocompass} > \sigma^2_{acc} + \frac{\sigma^2_{gyro}}{15^2\cos^2(\phi_{geodetic})}$$

The coarse alignment may produce a large error, and this will cause the linear error model and KF to be inefficient which is created based on assuming small misalignment [8].

IV. FINE ALIGNMENT METHODS & RESULTS

Kalman Filter (KF) is the mostly used technique for attitude estimation in initial alignment procedure. However, there are variations of KF. Below, the general linearized KF equations are provided.

General linearized filtering prediction equations:

$$m^-_k \approx \Phi(m_{k-1}) \tag{1}$$

$$C_k^- \approx A_k C_{k-1} A_k^T + \Sigma \tag{2}$$

General linearized filtering update equations:

$$\mu \approx h(m_k^-) \tag{3}$$

$$S \approx H_k C_k^- H_k^T + \Gamma \tag{4}$$

$$U \approx C_k^- H_k^T \tag{5}$$

$$m_k = m_k^- + US^{-1}(y_k - \mu) \tag{6}$$

$$C_k = C_k^- - US^{-1}U^T \tag{7}$$

Also, the the Gaussian filtering equations are listed below for other types of filtering to be discussed.

General Gaussian filtering prediction equations:

$$m_k^- = \int \Phi(X_{k-1}) \mathcal{N}(X_{k-1}; m_{k-1}, C_{k-1}) dX_{k-1} \tag{8}$$

$$C_k^- = \int (\Phi(X_{k-1}) - m_k^-)(\Phi(X_{k-1}) - m_k^-)^T \mathcal{N}(X_{k-1}; m_{k-1}, C_{k-1}) dX_{k-1} + \Sigma \tag{9}$$

General Gaussian filtering update equations:

$$\mu = \int h(X_k) \mathcal{N}(X_k; m_k^-, C_k^-) dX_k \tag{10}$$

$$U = \int (X_k - m_k^-)(h(X_k) - \mu)^T \mathcal{N}(X_k; m_k^-, C_k^-) dX_k \tag{11}$$

$$S = \int (h(X_k) - \mu)(h(X_k) - \mu)^T \mathcal{N}(X_k; m_k^-, C_k^-) dX_k + \Gamma \tag{12}$$

$$m_k = m_k^- + US^{-1}(y_k - \mu) \tag{13}$$

$$C_k = C_k^- - US^{-1}U^T \tag{14}$$

A full state KF for initial alignment and/or navigation purposes would consists of 15 error states with position, velocity, attitude, accelerometer and gyro error as:

$$X = [\delta r_N \ \delta r_E \ \delta r_D \ \delta V_N \ \delta V_E \ \delta V_D \ \psi_N \ \psi_E \ \psi_D \ \nabla_N \ \nabla_E \ \nabla_D \ \epsilon_N \ \epsilon_E \ \epsilon_D]$$

IV.I.Observability Aspect

One of the parameters for KF accuracy is the observability of the dynamical systems and the observability analysis shows that the system is not fully observable. Literature shows that there are some work to overcome this issue.

One of the most cited works [9] suggests the use of a 10 state KF, by removing; the position error states since the location would be known in a stationary alignment, and the vertical velocity error due to horizontal velocities being of interest and its weak coupling with them. This state vector then becomes:

$$X = [\delta V_N \ \delta V_E \ \psi_N \ \psi_E \ \psi_D \ \nabla_N \ \nabla_E \ \epsilon_N \ \epsilon_E \ \epsilon_D]$$

Then, using the horizontal velocity measurements

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

However, the rank of the observability matrix in this case is only 7, which suggests 3 states, namely $[\nabla_E \ \epsilon_E \ \epsilon_D]$, are unobservable [9][10] and this alters the filter performance. So, a rate state vector is introduced to combine the originally unobservable states with the observable ones. There are may works in literature which uses the 10 state filter. Literature has a lot of work dedicated to increasing the observability of the system to have accurate estimation performance. One of the works, focusing on state selection for that purpose, states that vertical velocity error should not be removed from the states due to its coupling with others. So, a 12 state filter is suggested to be used for initial alignment purposes. In case of INS/GNSS integration for navigation purposes after the alignment, a full order KF is suggested [11]. The following work from the same authors also include an observability and estimability analysis for the states [12].

Removal of unobservable states to increase filter performance is also studied where the state vector is initially defined as [10]:

$$X = [\phi_N \ \phi_E \ \phi_D \ \epsilon_N \ \epsilon_E \ \epsilon_D \nabla_N \ \nabla_E \ ]$$

$$\dot{X}(t) = FX + Gw \quad where,\ F = \begin{bmatrix} 0 & -\omega_{ie}sinL & 0 & -1 & 0 & 0 & 0 & 0 \\ \omega_{ie}sinL & 0 & \omega_{ie}cosL & 0 & -1 & 0 & 0 & 0 \\ 0 & -\omega_{ie}cosL & 0 & 0 & 0 & -1 & 0 & 0 \\ & & 0_{5x8} & & & & & \end{bmatrix}$$

Then, after conducting the observability analysis, when the unobservable states are removed the state vector becomes:

$$X = [\phi_N\ \phi_E\ \phi_D\ \epsilon_N\ \epsilon_D]$$

Then, the discrete KF algorithm is used as (1)–(7) with renewed system dynamics as

$$F = \begin{bmatrix} 0 & -\omega_{ie}sinL & 0 & -1 & 0 \\ \omega_{ie}sinL & 0 & \omega_{ie}cosL & 0 & 0 \\ 0 & -\omega_{ie}cosL & 0 & 0 & 0 \\ & 0_{1x5} & & & \end{bmatrix}$$

This algorithm is used for large initial attitude errors, 5°, 5°, and 50° for roll, pitch, and yaw respectively. The convergence time is reduced with the removal of the states: for the first two it is 40 seconds, whereas it is 100 seconds for the last one, longer than the first two as expected.

All of the states can be made observable by using multi position alignment. The second position of the IMU can be set to be the actual working position and the position by rotation around the azimuth axis can be used as the first position to obtain fully observable system. The IMU rotation by 15° degree increments showed that 180° rotation yields the best results, as shown in Figure 2 below, with 0.03° azimuth error [13].

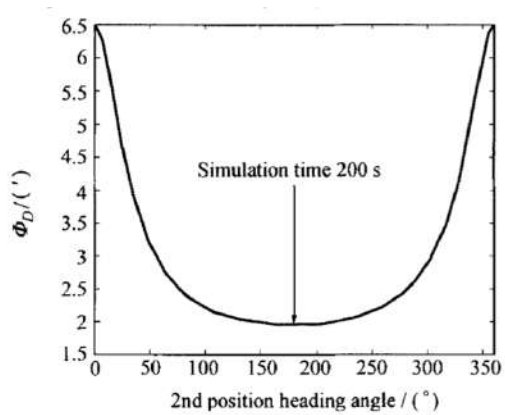

Figure 2

There are also other works using two position alignment, such as using the East gyro information as a new measurement [14]. The observation then becomes

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\Omega_U & \Omega_N & 0 & 0 & C_{11} & C_{12} & C_{13} \end{bmatrix}$$

The accuracies of the conventional two position and proposed two position alignment is similar, however, the proposed method is faster than the conventional one. There are also works regarding 90° rotation to account for the erecting maneuver and three position alignment for down axis gyro error estimation [15][16].

The effect of multi positioning on states' observability can be seen in Table 1 [17].

Difference alignment methods	The observability of states				
	ϕ_E, ϕ_N	ϕ_U	∇_x, ∇_y	$\varepsilon_x, \varepsilon_y$	ε_z
Fixed position alignment	observable	Low-observability	unobservable	observable	Low-observability
Two-position alignment	observable	observable	Low-observability	observable	High-observability

Table 1

IV.II.Kalman Filtering Variations

Linear KF is a widely used algorithm for initial alignment. Extended KF (EKF) is another type of commonly used algorithm for initial alignment. As discussed in the course material, EKF is based on integrating the linear integrands by linearizing the system around m_{k-1} for the dynamics and m_k^- for the measurement model with Taylor series approximation by using the Jacobians as shown below.

$$A_k = \nabla \Phi(X)|_{X_k = m_{k-1}}$$

$$H_k = \nabla h(X)|_{X_k = m_k^-}$$

For Ares I, a 2 minute coarse and 18 fine initial alignment EKF algorithm was proposed with ZVU. The filter introduced for this mission in [18] uses only three axis velocity and attitude errors. The state transition matrix, Φ, is computed with a Taylor series expansion up until to the third power

$$\Phi = \left(I + F\Delta t + \frac{1}{2}F^2\Delta t^2 + \frac{1}{6}F^3\Delta t^3 \right) \; where, \quad F = \frac{\delta f(x)}{\delta x} = \begin{bmatrix} 0 & \Delta t[\times (C_S^{Nav})\overrightarrow{a}_S] \\ 0 & 0 \end{bmatrix}$$

Where $\overrightarrow{a}_S$ is the sensor output specific force.

$$Y = HX, \quad H = \begin{bmatrix} I_{3x3} & 0_{3x3} \end{bmatrix}$$

Quaternions are used for attitude updates and $\mu = h(x)$ is the velocity obtained from the navigation algorithm for the update part of the filter. Which yields the time KF equations as:

$$C^- = \Phi C^+ \Phi^T + \Sigma(t)\Delta t$$

$$\hat{x} = K(y - \mu)$$

$$C^+ = (I - KH)C^-$$

$$K = CH^T(HC^-H^T + \Gamma)^{-1}$$

Where $\hat{x}$ denotes the estimate (updated) states, and + and – posterior and prior respectively. The errors on roll, pitch, and yaw was found to be 0.03°, 0.004° ve 0.004° respectively on Monte Carlo analysis [18].

There are several filters designed with fading factors for initial alignment in literature. Multiple fading factors for the use with KF presented in [19] with initial filter equations as usual with F as the system disturbance matrix:

$$m_k^- = \Phi m_{k-1}$$

$$C_k^- = \Phi_k^- C_{k-1} \Phi_k^{-T} + F_{k-1} \Sigma_{k-1} F_{k-1}^T$$

$$C_k = (I - K_k H_k) C_k^- (I - K_k H_k)^T - K_k \Gamma_k K_k^T$$

$$m_k = m_k^- + K_k (y_k - \mu)$$

It should be noted that his paper, along with many others, uses the Joseph form, not the normal form as we have used in class, to update the error covariance matrix. Joseph form prevents the loss of symmetry and positive definiteness with cost of computation. Where

$$\mu = H m_k^-$$

$$K_k = C^- H_k^T (H_k C^- H_k^T + \Gamma_k)^{-1}$$

Divergence of the filter might happen due to the reliance on the past data. In order to overcome this, exponential fading factor λ is introduced. This way, the memory of the filter would be limited. The fading factor is used on the covariance prediction step as

$$C_k^{-*} = \lambda_k \Phi_k^- C_{k-1} \Phi_k^{-T} + F_k \Sigma_k F_k^T \quad \lambda_k > 1$$

Where Φ is the state transition matrix. This one uses 12 states, three axes velocity, attitude, accelerometer and gyro errors as shown below.

$$X = [\delta V_E \ \delta V_N \ \delta V_U \ \phi_E \ \phi_N \ \phi_U \ \ \nabla_x \ \nabla_y \ \nabla_z \ \epsilon_x \ \epsilon_y \ \epsilon_z]$$

$$\dot{X}(t) = AX + Gw \quad where,\ A = \begin{bmatrix} A_1 & A_2 & C_B^N & 0_{3x3} \\ 0_{3x3} & A_3 & 0_{3x3} & -C_B^N \\ 0_{3x3} & 0_{3x3} & 0_{3x3} & 0_{3x3} \\ 0_{3x3} & 0_{3x3} & 0_{3x3} & 0_{3x3} \end{bmatrix} \quad G = \begin{bmatrix} C_B^N & 0_{3x3} \\ 0_{3x3} & C_B^N \\ 0_{6x3} & 0_{6x3} \end{bmatrix}$$

$$A_1 = \begin{bmatrix} 0 & 2\omega_{ie}sinL & -2\omega_{ie}cosL \\ -2\omega_{ie}sinL & 0 & 0 \\ 2\omega_{ie}cosL & 0 & 0 \end{bmatrix},\ A_2 = \begin{bmatrix} 0 & -g & 0 \\ g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},\ A_3 = \begin{bmatrix} 0 & \omega_{ie}sinL & -\omega_{ie}cosL \\ -\omega_{ie}sinL & 0 & 0 \\ \omega_{ie}cosL & 0 & 0 \end{bmatrix}$$

$$Y = HX + v, \quad H = \begin{bmatrix} I_{3x3} & 0_{3x9} \end{bmatrix}$$

If the fading factor is constant as shown above, it may not achieve the desired performance under unpredicted disturbance effects. So, [19] proposes the usage of multiple varying fading factors to work on the memory length. A slow fading factor would be used when the data fits to the model better than the rapid fading factor cases. Below is an algorithm structure with optimal fading factor

$$N_k = \hat{\delta}_{v_k} - B_k$$

$$B_k = H_k F_k \Sigma_k F_k^T H_k^T + R_k$$

$$M_k = H_k \Phi_k^- C_k^- \Phi_k^{-T} H_k^T$$

$$J_k = \Phi_k^- C_k^- \Phi_k^{-T}$$

$$\lambda_k = max\left(1, \frac{tr[N_k M_k^{-1}]}{m}\right) \quad where \quad m : dim(observation\ vector)$$

Where $\hat{\delta}_{v_k}$ is the innovation sequence covariance estimation. This single fading factor usage may cause the filter to diverge given the estimation effect of each state being different. In order to account for these differences, and also to keep covariance metric symmetric, a diagonal fading factor matrix, Λ_k, can be used to propagate the covariance matrix.

$$C_k^{-*} = \Lambda_k \Phi_k^- C_{k-1} \Phi_k^{-T} \Lambda_k^T + F_k \Sigma_k F_k^T$$

$$\Lambda_k = \begin{bmatrix} \lambda_k^1 & & & & & & \\ & \ddots & & & & & \\ & & \lambda_k^m & & & & \\ & & & \ddots & & & \\ & & & & 1 & & \\ & & & & & \ddots & \\ & & & & & & 1 \end{bmatrix}$$

$$H = \begin{bmatrix} S_{mxm} & 0_{mx(n-m)} \end{bmatrix}$$

$$\lambda_k^i = max\left(a, \frac{\alpha_i tr[N_k]}{\sum_{i=1}^n \alpha_i M_k^{ii}}\right)$$

$$where \quad \lambda_k^i = \begin{cases} max\left(1, \sqrt{\frac{\hat{\delta}_{v_k}^{ii} - b_{ii}}{s_i^2 j_{ii}}}\right), & \hat{\delta}_{v_k}^{ii} > b_{ii} \\ 1, & \hat{\delta}_{v_k}^{ii} \leq b_{ii} \end{cases}$$

This filter is applied to the initial alignment procedure stimulation with velocity measurement error updates and 0.3° coarse alignment azimuth accuracy, which meets the applicability demand of the linear model. A standard KF (a), single (b), and multiple fading factor KF (c) were run with varying system noise to compare performances, and the results are shown in Figure 3. Horizontal accelerometer biasses and east gyro drift is unobservable here. The proposed multiple fading factor algorithm performs better than the other two in terms of the estimation error and stability under disturbances. As expected, the main improvement in the accuracy was, by 50%, in azimuth. In addition to the simulation, an experiment was also conducted to validate the fast and accurate estimation algorithm with higher adaptability and robustness introduced.

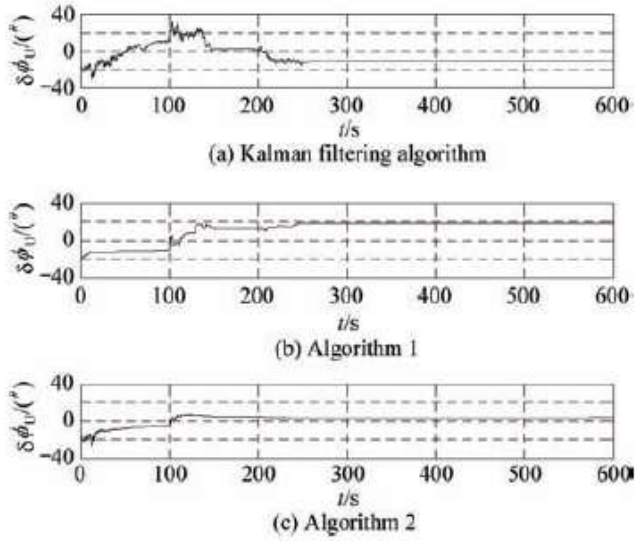

Figure 3

Even though using nonlinear model and dynamics is possible, this may still be not robust enough to account for the disturbance and model error. To tackle these model uncertainty issues, strong tracking filter (STF) can be used [20]. STUKF can be used with a single fading factor [21] or multiple fading factors MSTUKF to be able to track each state with different uncertainty.

Another work for the similar concept as above, [22] proposes a third order spherical radial reduced cubature KF with linear measurement equation to combine RCKF with STF with two multiple fading factor matrices, RMSTCKF. It is only needed to perform cubature transformation once without the need of calculating the Jacobian matrix. The cubature points are calculated as done in the course material, third order spherical.

$$[M] = \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots, \begin{pmatrix} -1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots \right\}$$

$$X = \mu + \sqrt{\Sigma}u, u \in M$$

Then the integrals are evaluated as

$$\int h(X)\mathcal{N}(X\,|\,\mu,\Sigma)dX \approx \sum_{i=1}^{2d} h\left(\mu + \sqrt{\Sigma}u^{(i)}\right) w^{(i)}, \quad with\ w = \frac{1}{2d}$$

Then, the error covariance matrix prediction is done according to (9) and then the fading factor introduced and the prediction is renewed as shown below.

$$C_k^{-*} = \lambda_{k+1} C_k^{-} \lambda'_{k+1} + \Sigma_k$$

The fading factor computation is as follows:

$$\lambda_k = \begin{bmatrix} \lambda_k^1 & & & \\ & \lambda_k^2 & & \\ & & \ddots & \\ & & & \lambda_k^n \end{bmatrix}$$

$$\lambda_k^i = max\left(a, \frac{\alpha_i tr[N_k]}{\sum_{i=1}^{n} \alpha_i M_k^{ii}}\right)$$

$$M_k = C_k^{-} H_k^T H_k$$

$$N_k = V_k - H_k \Sigma_{k-1} H_k^T - \Gamma_k$$

$$where \quad V_k = \begin{cases} e_1 e_1^T, & k = 0 \\ \frac{\rho V_{k-1} + e_k e_k^T}{1+\rho}, & k > 0 \end{cases}, and \quad e_k = y_k - H_k m_k^{-}$$

$\beta = 3$ is defined as the weakening factor and $\rho = 0.95$ is defined as the forgetting factor. It is deduced from the equations that ENU is used as the local navigation frame, N, with the states as

$$X = [\phi_E\ \phi_N\ \phi_U\ \delta V_E\ \delta V_N\ \ \nabla_x\ \nabla_y\ \epsilon_x\ \epsilon_y\ \epsilon_z]$$

Small angle approximation is once again used as $sin\alpha \approx \alpha$, $cos\alpha \approx 1$ on the DCM. The measurements are once again used as the ZVU. The initial misalignment states are chosen to be 1°, 1°, and 30° respectively in East, North, and Up axes. The results for RCKF, RSTCKF, and RMSTCKF are compared with 100 Monte Carlo simulations in terms of root mean square error as shown in Figure 4. It shows that RMSTCKF converges faster than the other two. The azimuth estimation accuracy is around 0.01°, as shown inTable 2.

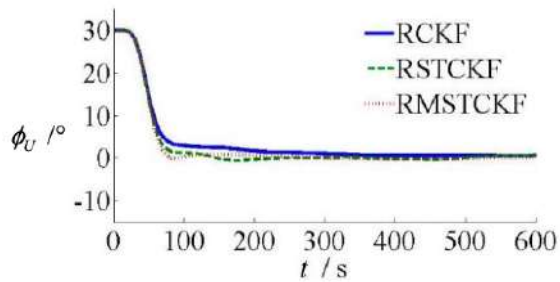

Figure 4

Misalignment angle error	RCKF	RSTCKF	RMSTCKF	Limited accuracy(')
Phi-E	-0.383	-0.390	0.386	-0.3438
Phi-N	0.413	0.418	0.425	0.3440
Phi-U	37.690	37.343	37.181	-32.4000

Table 2

When covariance matrix is changed to test robustness and not match the true one, RCKF works poorly, whereas the other two converge quickly as shown in Figure 5.

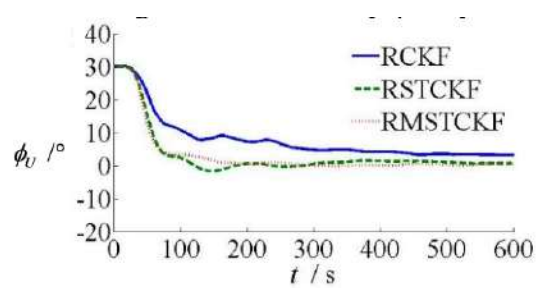

Figure 5

Disturbances are also considered to account for the real life platform conditions before launch. This can also show the robustness of the algorithm, and RCKF again works worse than the other two in terms of convergence time and accuracy as shown in Figure 6. RMSRCKF, RSTCKF and RCKF converges in 150, 220, and 500 seconds respectively.

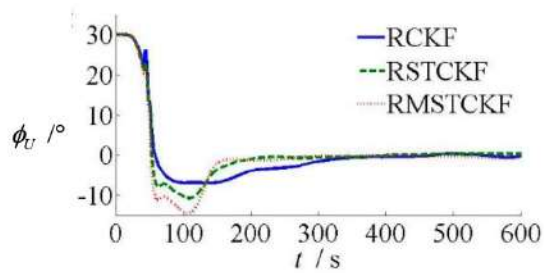

Figure 6

Two types of filters, unscented and particle, studied in the course material and project are also applicable to initial alignment. In addition to that, a combination of these two, unscented particle filter (UPF), is another proposed filtering method to be used [23]. This was one of the options for the last part of the Project 3, where UKF was to be used as a proposal to the PF. This could prevent the particle from degrading. It would also help moving the particles to the high likelihood region as known. This is a method with high estimation capability where the noise can be non Gaussian and the dynamics are not linear. It has superiority over PF, UKF, and EKF (respectively) in terms of estimation performance and having fewer limitations. The work establishes a covariance matching scheme where the gain is adaptively changed in accordance with the covariance to prevent divergence and establish tracking at a faster rate.

$$X = [\phi_E \ \phi_N \ \phi_U \ \delta V_E \ \delta V_N \ \ \epsilon_x \ \epsilon_y \ \epsilon_z \nabla_x \ \nabla_y \ ]$$

$$H = \begin{bmatrix} 0_{2x3} & I_{2x2} & 0_{2x5} \end{bmatrix}$$

$$[M] = \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots, \begin{pmatrix} -1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \ldots \right\}, \quad and \quad u^{(0)} \begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

$$\int h(X)\mathcal{N}(X\,|\,\mu,\Sigma)dX \approx \sum_{i=0}^{2d} h\left(\mu + \sqrt{\Sigma}\sqrt{d+\lambda}u^{(i)}\right) w^{(i)}$$

The weights for the mean are calculated as

$$w_m^{(i)} = \begin{cases} \frac{\lambda}{d+\lambda}, & i = 0 \\ \frac{1}{2(d+\lambda)}, & i = 1,2,...,2d \end{cases}$$

Whereas the weights for the covariance are calculated as

$$w_c^{(i)} = \begin{cases} \frac{\lambda}{d+\lambda} + (1-\alpha^2+\beta), & i = 0 \\ \frac{1}{2(d+\lambda)}, & i = 1,2,...,2d \end{cases}$$

Where α, β and κ are free parameters to be chosen and here, $\lambda = \alpha^2(d+\kappa) - d$, and $0.0001 \leq \alpha \leq 1$, $\beta = 2$, $\kappa = 3 - d$

In order to obtain adaptivity, time variant fading factors are used as in the sense of strong tracking filter. This way, a memory attenuation factor is introduced as done in the filter approaches above. If the condition

$$e_k^T e_k \leq S * tr(E(e_k e_k^T)), \quad where \quad S > 1 \; and \; e_k = y_k - h(m_k^-)$$

is not met, the following condition is checked to compute the fading factor and correct as explained above after prediction and before measurement updating:

$$\lambda_k = \begin{cases} \lambda_k, & \lambda_k \geq 1 \\ 1, & \lambda_k < 1 \end{cases}$$

$$C_k^{-*} = \lambda_k C_k^- \lambda_k' + \Sigma_k$$

$$\lambda_0 = \frac{tr(V - \Gamma_k)}{tr(U)}$$

ρ is taken as 0.95 again in this work, same as the other fading factor using algorithms introduced above. This parameter impacts the weight of the current information proportionally.

Where V and U are defined in previous section and (11) respectively and evaluated integrals as done before in a summation with cubature points and weights.

$$w_k^{(i)} = w_{k-1}^{(i)} \frac{p(y_k | x_k^{(i)}) p(x_k^{(i)} | x_{k-1}^{(i)})}{q(x_k^{(i)} | x_0^k(i), y_0^k)}$$

$$p(y_k | x_k^{(i)}) = exp[-0.5(y_k - h(m_k^{-1}))^T \Gamma^{-1} (y_k - h(m_k^{-1}))^T]$$

$$p(x_k^{(i)} | x_{k-1}^{(i)}) = exp[-0.5(y_k - h(m_k^{-1}))^T \Sigma^{-1} (y_k - h(m_k^{-1}))^T]$$

$$q(x_k^{(i)} | x_0^k(i), y_0^k) = \left(\sqrt{|P_k|}\right)^{-1} exp[-0.5(y_k - h(m_k^{-1}))^T P_k^{-1} (y_k - h(m_k^{-1}))^T]$$

The filter uses the scheme for UKF and PF which are introduced in the lecture notes and implemented in the project. The algorithm introduced (adaptive UPF) is tried with large azimuth error, where errors on three axes are 1°, 1°, and 10° respectively. UPF and the adaptive UPF are compared in terms of estimation performance and adaptive UPF resulted better as shown in Figure 7.

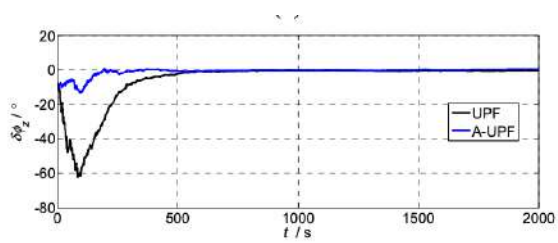

Figure 7

Adaptive UPF requires 450 seconds less than the ususal UPF to estimate azimuth. The results with noise with variance 0.02 are also shown below in Table 3. Resampling helps usage of high weighted particles. The paper also analyzes the output of three types of resampling methods: system, remainder, and polynomial resampling.

	Mean/°			Variance/°		
	Pitch Error	**Roll Error**	**Head Error**	**Pitch Error**	**Roll Error**	**Head Error**
UPF	0.0454	0.0278	2.8174	0.0648	0.0634	4.9038
Adaptive UPF	0.0105	0.0122	−0.7304	0.0629	0.0425	2.2557

Table 3

In case of model uncertainty the noise is not white, or not known if white or not, a more robust filter might be needed as opposed to KF. [24] takes three types of disturbances, sensor drift, Gaussian noise and norm bounded disturbances (i.e. uncertainty in the model, perturbation etc.), and designs a disturbance rejection and attenuation purposed filter. The first one is done through using the estimates of sensor drift to eliminate them, and the second one was accomplished by implementing a H_2/H_∞ to attenuate Gaussian and norm bounded disturbances. H_2 filer, which bounds the norm of noise signal to the error in filtering, with H_∞, which ensures the worst case of H_∞ norm from noise to error in estimation, is combined to acquire both dynamic performance and convergence of the error on estimation. The scheme is depicted in Figure 8.

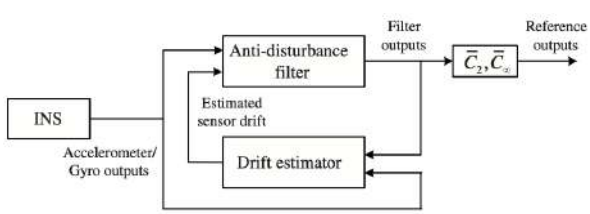

Figure 8

The filter at hand is basically being designed for an LTI system with multiple disturbances.

$$\tilde{x}(k+1) = (A - LC)\tilde{x}(k) + (B - LD)\tilde{\omega}(k) + (B_1 - LD_1)\omega_1(k) + (B_2 - LD_2)\omega_2(k)$$

$$\tilde{\omega}^n(k+1) = (W - KD)\tilde{\omega}^n(k) - KC\tilde{x}(k) + (B_3 - KD_1)\omega_1(k) - KD_2\omega_2(k)$$

$$\tilde{x}(k) = x(k) - \hat{x}(k) \quad \tilde{\omega}(k) = \omega(k) - \hat{\omega}(k)$$

For

$$\hat{x}(k+1) = A\hat{x}(k) + B[\hat{\omega}^n(k) + u(k)] + L[y(k) - \hat{y}(k)]$$

$$\hat{y}(k) = C\hat{x}(k) + D[\hat{\omega}^n(k) + u(k)]$$

$$\hat{\omega}^n(k+1) = W\hat{\omega}^n(k) + K[y(k) - \hat{y}(k)]$$

$$x(k) = \begin{bmatrix}\phi_E(k) & \phi_N(k) & \phi_U(k)\end{bmatrix} \quad y(k) = \begin{bmatrix}f_E(k) & f_N(k) & f_U(k)\end{bmatrix} \quad \omega(k) = \begin{bmatrix}\epsilon_E(k) & \epsilon_N(k) & \epsilon_U(k) & \nabla_E(k) & \nabla_N(k)\end{bmatrix}$$

Considering that the heading error being more than the leveling error, usually, can lead to modeling errors for linearization, proposed filter attenuates that error with the help of H_∞ filter. The results are shown in Figure 9 for comparison with KF.

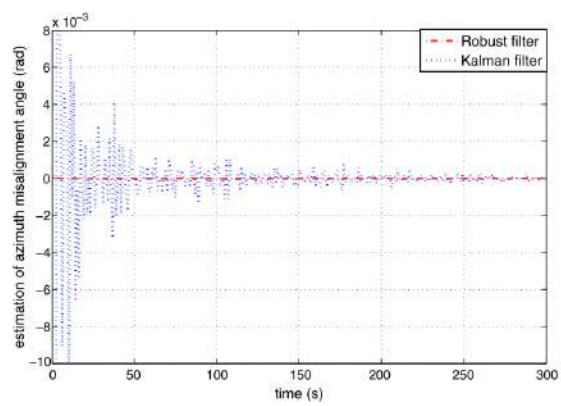

Figure 9

The same type of anti-disturbance filter was also implemented to study its comparison with UKF, and the results are as shown below in Figure 10, and Table 4 as 10^{-5} rad as unit. It is clear that it performed better [25].

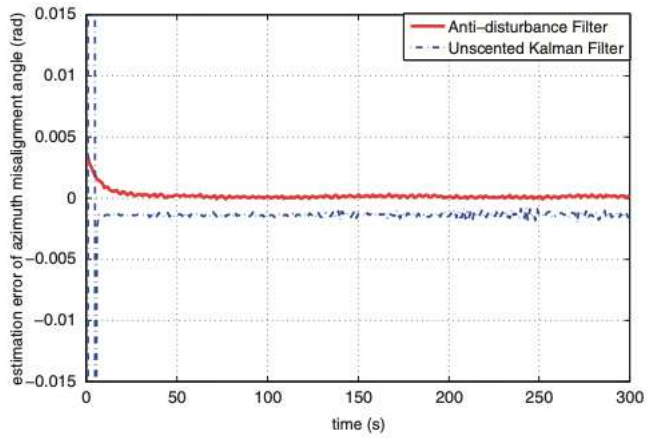

Figure 10

Method	Levelling x	Levelling y	Azimuth
In this article	4.084436	5.962061	8.382810
UKF	27.573893	75.867469	18.481666

Table 4

V. DISCUSSION & CONCLUSION

The main motivation behind initial alignment is to obtain an accurate initial attitude estimate to start navigation with better states. This would affect the success of the mission. The initial alignment procedure considered in this literature review was SINS stationary self alignment, although there are several types such as slave/master, rocking/swaying base, transfer alignment etc. The alignment procedure requires to decrease error in the fine alignment phase, which is initialized by the coarse alignment output, with the help of filtering algorithms. The challenge is usually on the azimuth error, where leveling can be done with higher accuracy. Due the disadvantages of lengthy and physically challenging optical alignment procedure, algorithm based alignment is favored.

Also, the usage of a linear KF might not always converge to the true value, especially when the small angle assumption is violated with a large initial attitude error.

The accuracy of initial alignment depends on many things including the specs of the IMU used. So, papers which use accelerometer and gyroscope with different error parameters are not completely comparable quantitatively. However, most of the papers are comparing the results of their proposed method with others when run with the same IMU parameters and initial conditions. So, it makes sense to make use of those comparisons for each. I tried to include different types of filters for this procedure, where connections to the lecture contents are feasible.

EKF is a widely used algorithm for this purpose. Then, a strong tracking is desired and obtained with the help of fading factor. This fading factor is used on multiple filters. Comparison of KF, single, and multiple fading factor KF shows that the usage of multiple fading factor performs the best among the others, where factors are adjusted for each state individually. Then, using radial third order spherical cubature points itself and, with single and multiple fading factor filters are compared. Once again it is shown that multiple fading factors performed better. The fading factor is once again applied to another course material filter, particle filter with the unscented KF as the proposal. This has performed better than the usual particle filter with the unscented KF as the proposal. There are also other types of anti-disturbance filters such as H_2/H_∞, which performs better when compared to KF and UKF. In addition to that, Monte Carlo is also used for many analysis in literature to assess the performance.

VI. REFERENCES

[1] Song, Z. The Test and Launch Control Technology for Launch Vehicles. Springer Singapore, 2018.

[2] Mrus, G. J., Zukowsky, W. S., Kokot, W., Yoder, P. R., and Wood, J. T. Automatic Theodolite for Pre-Launch Azimuth Alignment of the Saturn Space Vehicles. Applied Optics, The Optical Society, 10, 3, Mar 01, 1971, p. 504.

[3] Zanetti, R., Holt, G., Gay, R., D'Souza, C., Sud, J., Mamich, H., and Gillis, R. Design and Flight Performance of the Orion Prelaunch Navigation System. Journal of Guidance, Control, and Dynamics, American Institute of Aeronautics and Astronautics (AIAA), 40, 9, Sep, 2017, pp. 2289-2300.

[4] Proud, Ryan W. et al., Orion Guidance and Control Ascent Abort Algorithm Design and Performance Results, 2009.

[5] Norris, L., Tao, Y.-C., Hall, R., Chuang, J., and Whorton, M. Analysis of Ares 1 Ascent Navigation Options. AIAA Guidance, Navigation and Control Conference and Exhibit, American Institute of Aeronautics and Astronautics, Jun 15, 2008.

[6] Titterton, D., and Weston, J. Strapdown Inertial Navigation Technology. Institution of Engineering and Technology, 2004.

[7] Groves, P. D. Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd Edition [Book Review]. IEEE Aerospace and Electronic Systems Magazine, Institute of Electrical and Electronics Engineers (IEEE), 30, 2, Feb, 2015.

[8] Lu, H., Hao, S., Peng, Z., and Huang, G. Initial Alignment of Large Azimuth Misalignment Angle in SINS Based on Reduced Multiple Fading Factors Strong Tracking CKF. MATEC Web of Conferences, EDP Sciences, 214, , 2018, p. 03006.

[9] Jiang Cheng Fang, and De Jun Wan. A Fast Initial Alignment Method for Strapdown Inertial Navigation System on Stationary Base. IEEE Transactions on Aerospace and Electronic Systems, Institute of Electrical and Electronics Engineers (IEEE), 32, 4, , 1996, pp. 1501-1504.

[10] Yi, J., Zhang, L., Shu, R., and Wang, J. Initial Alignment for SINS Based on Low-Cost IMU. Journal of Computers, International Academy Publishing (IAP), 6, 6, Jun 03, 2011.

[11] Silva, F. O., Hemerly, E. M., and Leite Filho, W. C. On the Error State Selection for Stationary SINS Alignment and Calibration Kalman Filters - Part I: Estimation Algorithms. Aerospace Science and Technology, Elsevier BV, 61, Feb, 2017, pp. 45-56.

[12] Silva, F., Hemerly, E., and Leite Filho, W. On the Error State Selection for Stationary SINS Alignment and Calibration Kalman Filters—Part II: Observability/Estimability Analysis. Sensors, MDPI AG, 17, 3, Feb 23, 2017, p. 439.

[13] Wang, X., and Shen, G. A Fast and Accurate Initial Alignment Method for Strapdown Inertial Navigation System on Stationary Base. Journal of Control Theory and Applications, Springer Science and Business Media LLC, 3, 2, May, 2005, pp. 145–149.

[14] Xiong, J., Guo, H., and Yang, Z. H. A Two-Position SINS Initial Alignment Method Based on Gyro Information. Advances in Space Research, Elsevier BV, 53, 11, Jun, 2014, pp. 1657–1663.

[15] Lee, J. G., Park, C. G., and Park, H. W. Multiposition Alignment of Strapdown Inertial Navigation System. IEEE Transactions on Aerospace and Electronic Systems, Institute of Electrical and Electronics Engineers (IEEE), 29, 4, 1993, pp. 1323–1328.

[16] Xu, Zhenkai, Li, Yong, Rizos, Chris, Xu, Xiaosu, "Rapid Initialization of SINS for Land Vehicle-based Vertically-launched Missiles," Proceedings of the 21st International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS 2008), Savannah, GA, September 2008, pp. 554-559

[17] Lai, Jizhou et al. Improved Arithmetic Of Two-Position Fast Initial Alignment For Sins Using Unscented Kalman Filter, 2012.

[18] Oliver, Jr., T.Navigation system design with application to the Ares I Crew Launch Vehicle and Space Launch Systems, 2013.

[19] GAO, W., MIAO, L., and NI, M. Multiple Fading Factors Kalman Filter for SINS Static Alignment Application. Chinese Journal of Aeronautics, Elsevier BV, 24, 4, Aug, 2011, pp. 476–483.

[20] Zhou D H, FRANK P M. Strong tracking filtering of nonlinear time-varying stochastic systems with colored noise Application to parameter estimation and empirical robustness analysis [J]. International Journal of Control, 2, 1996.

[21] Wang L, Wu L, Guan Y, et al. Online sensor fault detection based on an improved strong tracking filter [J]. Sensors, 2, 2015.

[22] Lu, H., Hao, S., Peng, Z., and Huang, G. Initial Alignment of Large Azimuth Misalignment Angle in SINS Based on Reduced Multiple Fading Factors Strong Tracking CKF. MATEC Web of Conferences, EDP Sciences, 214, , 2018, p. 03006.

[23] Sun, J., Xu, X.-S., Liu, Y.-T., Zhang, T., and Li, Y. Initial Alignment of Large Azimuth Misalignment Angles in SINS Based on Adaptive UPF. Sensors, MDPI AG, 15, 9, Aug 31, 2015, pp. 21807–21823.

[24] Cao, S., and Guo, L. Multi-Objective Robust Initial Alignment Algorithm for Inertial Navigation System with Multiple Disturbances. Aerospace Science and Technology, Elsevier BV, 21, 1, Sep, 2012, pp. 1–6.

[25] Guo, L., Cao, S., Qi, C., and Gao, X. Initial Alignment for Nonlinear Inertial Navigation Systems with Multiple Disturbances Based on Enhanced Anti-Disturbance Filtering. International Journal of Control, Informa UK Limited, 85, 5, May, 2012, pp. 491–501.